

The JETSPIN User Manual

M. Lauricella¹, G. Pontrelli¹, I. Coluzza², D. Pisignano³, S. Succi⁴

¹Istituto per le Applicazioni del Calcolo “Mauro Picone”, Consiglio Nazionale delle Ricerche
(CNR),
Via dei Taurini 19, 00185 Rome, Italy

²Center for Theoretical Biological Physics, Rice University,
Houston, Texas, USA

³Department of Physics, University of Pisa,
Largo Bruno Pontecorvo 3, 56127 Pisa, Italy

⁴Center for Life Nano- & Neuro-Science, Istituto Italiano di Tecnologia (IIT),
Viale Regina Elena 291, 00161 Rome, Italy

version 2.0-beta.1 (pre-release) - October 2026

Disclaimer

None of the authors, nor any contributor to the JETSPIN code or its derivatives guarantee that the software and associated documentation is free from error. Neither do they accept responsibility for any loss or damage that results from its use. The responsibility for ensuring that the software is fit for purpose lies entirely with the user.

JETSPIN Acknowledgements

The software development process has received funding from the European Research Council under the European Union's Seventh Framework Programme (FP/2007-2013)/ERC Grant Agreement n. 306357 "NANO-JETS".

About JETSPIN

The code is licensed under Open Software License v. 3.0 (OSL-3.0). The full text of the licence can be found on the website: <http://opensource.org/licenses/OSL-3.0>

If results obtained with this code are published, an appropriate citation would be:

Marco Lauricella, Giuseppe Pontrelli, Ivan Coluzza, Dario Pisignano, Sauro Succi, JETSPIN: A specific-purpose open-source software for simulations of nanofiber electrospinning, *Computer Physics Communications*, 197 (2015), 227-238.

The models implemented in JETSPIN are reviewed, in the broader context of electrospinning and solution blowing, in the following works, which may also be cited:

Marco Lauricella, Sauro Succi, Eyal Zussman, Dario Pisignano, Alexander L. Yarin, Models of polymer solutions in electrified jets and solution blowing, *Reviews of Modern Physics*, 92 (2020), 035004.

Alexander L. Yarin, Filippo Pierini, Eyal Zussman, Marco Lauricella, Modelling of nanofiber formation processes, in *Materials and Electro-mechanical and Biomedical Devices Based on Nanofibers*, Springer (2024), 237-326.

The revisions of this manual for version 2.0, like the Markdown documentation in the *docs* sub-directory, were written with the assistance of an artificial-intelligence agent and reviewed against the source code.

Contents

1	Introduction	6
1.1	The JETSPIN Package	6
1.2	Structure	7
1.3	Parallelization	8
1.4	Compiling the Source Code	8
1.5	Running JETSPIN	12
1.5.1	Running on a GPU	13
1.6	Restarting JETSPIN	14
2	JETSPIN Model and Algorithms	15
2.1	Equations of Motion	15
2.2	Perturbations at the nozzle	18
2.3	Jet insertion	19
2.4	Dynamic refinement	19
2.4.1	Numerical robustness of the cross-section reconstruction	21
2.5	Random noise	23
2.5.1	Pre-generated Gaussian pool	23
2.6	External oscillating electric field	24
2.7	Dimensionless quantities	25
2.8	Solvent evaporation and solidification	26
2.8.1	JETSPIN nondimensionalization	27
2.8.2	Polymer concentration and rheological solidification	28
2.8.3	Kelvin–Voigt extension with concentration-dependent properties	28
2.8.4	Compatibility with dynamic refinement	29
2.8.5	Evaporation cutoff	29
2.9	Integration schemes	30
2.10	Multiple Timestep Algorithm for Coulomb forces	31
3	JETSPIN Data Files	33
3.1	Input file	33
3.1.1	Evaporation directives	38
3.2	Output files	39
4	Example Simulations	43
4.1	Test Case 1: 1-D case	43
4.2	Test Case 2: 1-D case with bead insertion at the nozzle activated	43
4.3	Test Case 3: Three-D simulation of electrospinning process	43
4.4	Test Case 4: Three-D simulation in presence of a gas counterflow	44
4.5	Test Case 5: Three-D simulation with dynamic refinement	45
4.6	Test Case 6: Three-D simulation of a quasi-Newtonian fluid	45
4.7	Test Case 7: Three-D simulation with evaporation and an orthogonal rotating electric field	45
4.8	Test Case 8: Yarin et al. (2001) reference parameters	45
4.9	Test Case 9: 1,000-bead CPU/GPU performance benchmark	46

4.10 Test Cases 10 and 11: Euler and RK2 GPU benchmarks	48
4.11 Test Case 12: Platen stochastic GPU benchmark	49
4.12 Test Case 13: dynamic-topology GPU baseline	49
4.13 Test Case 14: larger bounded dynamic topology	50
4.14 Test Case 15: forced array reallocation	50
4.15 Test Case 16: dynamic Maxwell evaporation on the GPU	51
4.16 Test Case 17: dynamic Kelvin–Voigt evaporation on the GPU	52
4.17 Test Case 18: high-resolution dynamic Maxwell evaporation	52
4.18 Test Case 19: high-resolution dynamic Kelvin–Voigt evaporation	53
4.19 Test Case 20: Stochastic Platen Maxwell Evaporation	53
4.20 Test Case 21: Anchored Dynamic Refinement with Platen Evaporation	53
4.21 Test Case 22: Repeated Dynamic Refinement and Capacity Growth	54
4.22 Test Case 23: Refinement with Collector Removal	55
4.23 Test Case 24: long run to a stationary bead count	56
4.24 Test Case 25: evaporative counterpart of Test Case 24	57
5 Tips	59
5.1 How to visualize a PDB file in VMD software package	59

Chapter 1

Introduction

Scope of Chapter

This chapter describes the design and directory structure of JETSPIN, how to compile and run it (serial, MPI and single-GPU OpenACC), and how to restart a run.

1.1 The JETSPIN Package

In the recent years, electrospun nanofibers have gained a considerable industrial interest due to many possible applications, such as tissue engineering, air and water filtration, drug delivery and regenerative medicine. In particular, the high surface-area ratio of the fibers offers an intriguing prospect for technological applications. As consequence, several studies were focused on the characterization and production of uni-dimensionally elongated nanostructures. A number of reviews [1, 2, 3, 4, 5] and books [6, 7, 8] concerning electrospinning have been published in the last decade.

Typically, electrospun nanofibers are produced at laboratory scale by the uniaxial stretching of a jet, which is ejected at the nozzle from a charged polymer solution. The initial elongation of a jet can be produced by applying an externally electrostatic field between the spinneret and a conductive collector. Electrospinning involves mainly two sequential stages in the uniaxial elongation of the extruded polymer jet: an initial quasi steady stage, in which the electric field stretches the jet in a straight path away from the nozzle of the ejecting apparatus, and a second stage characterized by a bending instability induced from small perturbations, which misalign the jet from its axis of elongation [9]. These small disturbances may originate from mechanical vibrations at the nozzle or from hydrodynamic-aerodynamic perturbations within the experimental apparatus. Such a misalignment provides an electrostatic-driven bending instability before the jet reaches the conductive collector, where the fibers are finally deposited. As a consequence, the jet path length between the nozzle and the collector increases, and the stream cross-section undergoes a further decrease. The ultimate goal of electrospinning process is to minimize the radius of the collected fibers. By a simple argument of mass conservation, this is tantamount to maximizing the jet length by the time it reaches the collecting plane. By the same argument, it is therefore of interest to minimize the length of the initial stable jet region. Consequently, the bending instability is a desirable effect, as it produces a higher surface-area-to-volume ratio of the jet, which is transferred to the resulting nanofibers [10].

Computational models provide a useful tool to elucidate the physical phenomenon and provide information which might be used for the design of electrospinning experiments. Numerical simulations can be used to improve the capability of predicting the key processing parameters and exert a better control on the resulting nanofiber structure. In recent years, with a renewed interest in nanotechnology, electrospinning studies attracted the attention of many researchers both from modeling, computational and experimental point of view [11, 12, 13, 14, 15, 9]. Although some author uses dissipative particle dynamics (DPD) mesoscale simulation method into electrospinning study [16], most models usually treat the jet filament as a series of discrete elements (*beads*) obeying the equations of continuum mechanics [11, 12]. Each bead is subject to different types of interactions, such as long-range Coulomb repulsion, viscoelastic drag, the external electric

field. The main aim of such models is to explain the complexity of the resulting dynamics and provide the set of parameters driving the optimal process. The effect of fast-oscillating loads on the bending instability have been explored in an extensive modeling and computational study [17].

JETSPIN delivers a FORTRAN code especially designed to simulate the electrospinning process in a variety of different conditions and experimental settings. This comprehensive platform is devised such that different cases and input variables can be described and simulated. The framework is developed to exploit several computational architectures: serial, MPI-parallel and, since version 2.0, a single GPU through OpenACC.

JETSPIN, as open-source software, can be used to carry out a systematic sensitivity analysis over a broad range of parameter values. The results of simulations provide valuable insight on the physics of the process and can be used to assess experimental procedures for an optimal design of the equipment and to control processing strategies of technological advanced nanofibers.

The JETSPIN code is freely downloadable at the web site:

<https://github.com/nanojets/jetspin>

1.2 Structure

JETSPIN is written in free-format Fortran 90 with a few Fortran 2003 features (`get_environment_variable`, allocatable components of derived types), and it consists of approximately 430 subroutines and functions. The source exploits the modular approach provided by the programming language. All the variables having in common description of certain features or method are grouped in modules. The convention of explicit type declaration is adopted, and all the arguments passed in calling sequences of functions or subroutines have defined intent. We use the `PRIVATE` and `PUBLIC` accessibility attributes in order to decrease error-proneness in programming.

The main routines have been gathered in the `main.f90` file, which drives all the CPU-intensive computations needed for the capabilities mentioned below. The variables describing the main features of nanofibers (position, velocity, etc.) are declared in the `nanojet_mod.f90` file, which also contains the main subroutines for the memory management of the fundamental data of the simulated system. Since the size system is strictly time-dependent, JETSPIN exploits the dynamic array allocation features of Fortran to assign the necessary array dimensions. In particular, the size system is modified by the routines `add_jetbead` and `erase_jetbead`, while the decision of the main array size, declared as `mxnpjjet`, is handled by the routine `reallocate_jet`. The sizes of various service bookkeeping arrays are handled within a parallel implementation strategy, which exploits few dedicated subroutines (see Sec 1.3). All the implemented time integrators are written in the `integrator_mod.f90` file, which contains the routine `driver_integrator` to select the proper integrator, as indicated in the input file. All the terms of equations of motion for the implemented model (see Sec 2.1) are computed by routines located in the `eom_mod.f90` file (`eom_ev_mod.f90` for the evaporating jet), which use `support_functions_mod.f90` for the jet geometry and `electric_field_mod.f90` for the external field; the integrators compute the Coulomb sums (`coulomb_force_mod.f90`) and call the equations of motion through `driver_eom_mod.f90`, and the jet breakup is handled in `breaking_mod.f90`. The Kelvin–Voigt model with evaporation has its integrators in `integrator_kv_ev_mod.f90`; dynamic refinement is in `dynamic_refinement_mod.f90`, with its Akima interpolation in `fit_mod.f90`; `utility_mod.f90` holds the random-number generator and the Gaussian pool, and `profiling_mod.f90` the optional region profiler. The OpenACC device step is in `device_step_mod.f90`; the module `accelerator_mod` exists in two versions, `openacc_accelerator_mod.f90` with the device kernels for the OpenACC targets and `host_accelerator_mod.f90` with stubs for the CPU targets, chosen by the Makefile target. A summarizing scheme of the main JETSPIN program in the `main.f90` file has been sketched in Fig 1.1.

The user can carry out simulations of nanofibers without a detailed understanding of the structure of JETSPIN code. All the parameters governing the system can be defined in the input file (see Sec 3.1), which is read by routines located in the `io_mod.f90` and `parse_mod.f90` files. Instead, the user should be acquainted with the model described in Chapter 2. The content of the output file is completely customizable by the input file as described in Sec 3.1, and it can report different time-averaged observables computed by routines of the module `statistic_mod` (see Sec 3.2). The routines in the `error_mod.f90` file can display

various warning or error banners on computer terminal, so that the user can easily correct the most common mistakes in the input file.

JETSPIN is supplied as a single directory (a git repository) with the sub-directories described below, besides *docs* (Markdown documentation), *manual* (this manual), *tests* (test suites and records) and *tools*. All the source code files are contained in the *source* sub-directory. The *examples* sub-directory contains different test cases that can help the user to edit new input files. The *build* sub-directory stores a UNIX **makefile** that assembles the executable versions of the code both in serial and parallel versions with different compilers. It is invoked from the repository root while using the *source* sub-directory as its working directory (see Section 1.4); it does not need to be copied. Finally, after JETSPIN was compiled, the *execute* sub-directory contains the binary executable file, where it can be run.

1.3 Parallelization

The parallel infrastructure of JETSPIN incorporates the necessary data distribution and communication structures. The parallel strategy underlying JETSPIN is the Replicated Data (RD) scheme[18], where fundamental data of the simulated system are reproduced on all processing nodes. In simulations of nanofibers, the fundamental data consist of position, velocity, and viscoelastic stress arrays of each bead in which the nanojet is discretised (see Chapter 2). Further data defining mass and charge of each bead are also replicated. However, all auxiliary data are distributed in equal portion of data (as much as possible) for each processor. Despite being available other parallel strategies such as the Domain Decomposition[19], our experience has shown that such volume of data is by no means prohibitive on current parallel computers.

By the RD scheme, we adopt in JETSPIN a simple communication strategy of the communication between nodes, which is handled by global summation routines. The module **version_mod** located in the **parallel_version_mod.f90** file contains all the global communication routines which exploit the MPI (Message Passing Interface) library. Note that a Fortran compiler and an MPI implementation for the specific machine architecture are required in order to compile JETSPIN in parallel mode. An alternative version of the module **version_mod** is located in the **serial_version_mod.f90** file, and it can be easily selected by appropriate flags in the **makefile** at compile time. By selecting this version, JETSPIN can also be run on serial computers without modification, even though the code has been designed to run on parallel computers.

The OpenACC targets build the serial version: one process on one GPU (Sec. 1.5.1). MPI runs therefore execute the CPU code. A stochastic Platen run whose model the device step supports reads the pre-generated Gaussian pool, which rank zero generates once before the time loop and broadcasts to every rank; each step reads its values by global bead index, so serial and parallel runs, and the CPU and GPU builds of the same compiler, integrate the same noise (the generator is the compiler's intrinsic one; Sec. 2.5.1). The other stochastic Platen runs draw one block of Gaussian numbers per step, also generated by rank zero in global bead order and broadcast.

The size system is strictly time-dependent as mentioned in Sec 1.2 and, therefore, the memory of various service bookkeeping arrays is dynamically distributed over all the processing nodes. In particular, the bookkeeping array size, declared as **mxchunk**, is managed by the routine **set_mxchunk**. All the beads of the discretised nanofiber are assigned at every time step to a specific node by the routine **set_chunk**, and their temporary data are stored in bookkeeping arrays belonging to the assigned node. It is worth stressing that the communication latency makes the parallelization efficiency strictly dependent on the system size. Therefore, we only advice the use of JETSPIN in parallel mode whenever the user expects a system size with at least 50 beads for each node (further details in Ref [20]).

1.4 Compiling the Source Code

The *build* sub-directory stores a UNIX **Makefile** that assembles serial and parallel executable versions with different compilers. JETSPIN may be compiled on any UNIX platform. From the root of the repository, invoke this file directly while making *source* the working directory; there is no need to copy the Makefile. Use

```
make -C source -f ../build/Makefile target
```

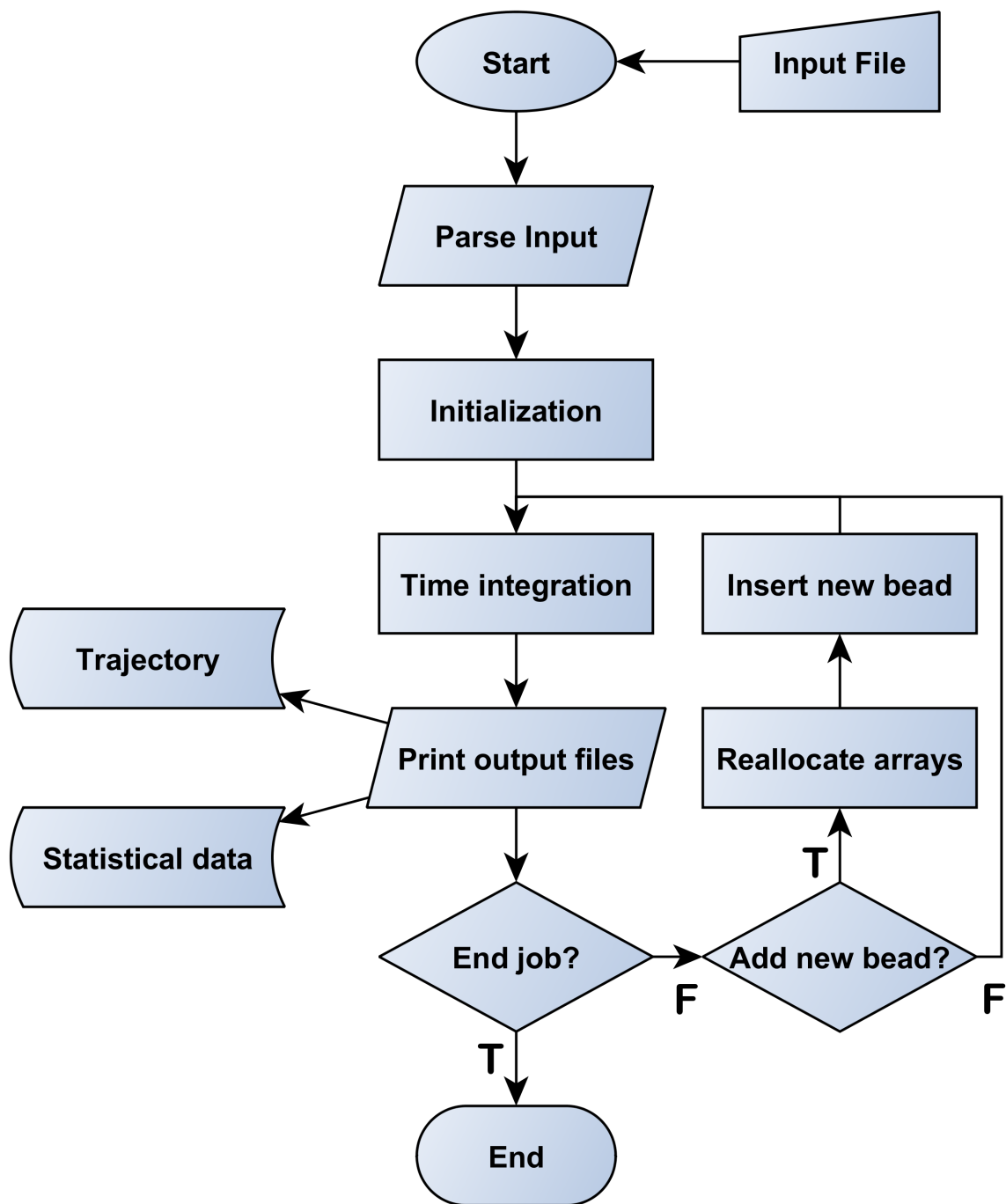


Figure 1.1: Simplified structure of the main JETSPIN program. The arrays are reallocated only when they are full; the restart, the freezing and removal of the beads at the collector, the breakup, the dynamic refinement and the job-time check are not shown.

where `target` is one of the options reported in Table 1.1. Object and module files are generated in *source*, and the resulting `main.x` executable is placed in *execute*. For example, the serial GFortran build is produced with

```
make -C source -f ../build/Makefile gfortran
```

and the corresponding Open MPI build with target `gfortran-mpi`. Build products can be removed with target `clean` using the same command form.

NVIDIA HPC SDK provides separate CPU-only and OpenACC build targets. On a module-based installation, first load the SDK and verify that its MPI wrapper uses NVFORTRAN:

```
module purge
module use /opt/nvidia/hpc_sdk/modulefiles
module load nvhpc/24.3
nvfortran --version
mpif90 --showme:command
```

The serial and MPI CPU-only builds are selected respectively with `nvfortran` and `nvfortran-mpi`. The latter requires `mpif90 -showme:command` to report `nvfortran`. Neither target enables OpenACC offloading.

The `nvfortran-openacc` target enables single-GPU OpenACC offloading. Its default values target compute capability 8.0, including NVIDIA A30 and A100 GPUs, and the CUDA 12.3 toolkit distributed with HPC SDK 24.3. It also selects the `nofma` GPU option to preserve the numerical behaviour of the nearly straight three-point curvature calculation:

```
make -C source -f ../build/Makefile nvfortran-openacc
```

The OpenACC GPU and host-validation targets expose the standard `_OPENACC` preprocessing macro. Accelerator imports, initialization, Coulomb dispatch, and the dispatch to the device step are enclosed by this macro. CPU targets preprocess the same sources without defining it, so accelerator calls are absent from their compiled translation units rather than being retained as no-op runtime branches.

Another device architecture or installed CUDA toolkit can be selected through the `GPUCC` and `CUDA_VERSION` Make variables. The compute capability is passed without the `cc` prefix; for example:

```
make -C source -f ../build/Makefile nvfortran-openacc \
  GPUCC=90 CUDA_VERSION=12.3
```

HPC SDK 25.5 ships the CUDA 11.8 and 12.9 toolkits, so it is used with `CUDA_VERSION=12.9`; such a build also runs with a driver of the CUDA 12.3 generation. NVFORTRAN 24.3 at `-O3` rewrites a division x/y as $x \cdot (1/y)$ (`-Mrecip-div`), whereas 25.5, like GFortran, rounds divisions exactly: the results of the two compilers differ in the last digits, and trajectories that are sensitive to them separate. An OpenACC run should be compared with a CPU build of the same compiler version. NVFORTRAN 24.3 is the reference compiler of the GPU records of the test cases; 25.5 passes the same checks.

Further Make variables, given on the command line like `GPUCC`, select the executable name (`EX`, default `main.x`) and its directory (`BINROOT`, default *execute*), the MPI wrapper (`MPIFC`, default `mpif90`) and the GFortran MPI option `MPI_COMPAT_FLAG` (default `-fallow-argument-mismatch`; a GFortran older than version 10 needs `-Wno-argument-mismatch`), the NVIDIA compiler (`NVFORTRAN`), and extra preprocessor flags for the `nvfortran*` targets only (`FPPFLAGS_EXTRA`, for the development macros; the oracle and comparison targets below set their own). Make also takes the variables that have a default (all but `EX` and `BINROOT`) from the environment, so that a `CUDA_VERSION` exported by another tool reaches the build. The environment variables that control an OpenACC run are described in Sec. 1.5.1.

Target `nvfortran-openacc-host` compiles the OpenACC control path for host execution. It is intended for compiler and numerical validation when an NVIDIA device is unavailable and is not a GPU performance build. Before a device run, `nvidia-smi` and `nvaccelinfo` should both confirm that the selected GPU is visible.

Target `nvfortran-openacc-force-oracle` is a model-independent, development-only GPU build. At every force evaluation of the device step, in every configuration that the step supports (Sec. 1.5.1), it downloads the current stage, evaluates the complete trusted host equations, and uploads only their derivatives.

Integration updates, dynamic topology, and the tail of the Platen step (`accelerator_platen_update` and `accelerator_platen_end_step`, with the final stress derivative) stay on the GPU. It enables the single `JETSPIN_DEV_HOST_FORCE_ORACLE` macro.

Target `nvfortran-openacc-coulomb-oracle` is the narrower diagnostic. It enables only `JETSPIN_DEV_HOST_COULOMB_ORACLE`. It transfers the state required by the direct Coulomb sum, evaluates that sum on the host in its established order, and uploads the force array. Its semantics are the same with or without evaporation. Every other force term and the integration updates remain on the GPU. The two targets are deliberately distinct, and their per-stage transfers make both unsuitable for production or performance measurements.

Target `nvfortran-openacc-host-akima` enables the development-only `JETSPIN_DEV_HOST_AKIMA` macro. It retains the historical host coefficient and interpolation routines while leaving the surrounding OpenACC refinement lifecycle enabled. Target `nvfortran-openacc-compare-akima` enables `JETSPIN_COMPARE_AKIMA`; it computes the trusted host result followed by the device result and reports maximum coefficient and interpolated-value differences for every field. Target `nvfortran-openacc-compare-refinement` enables `JETSPIN_COMPARE_REFINEMENT_ASSEMBLY` and applies the same host-then-device comparison to the reconstruction of bead volume, evaporated volume, mass, and charge that follows the Akima interpolation (see Sec. 2.4). None of these targets is intended for performance measurements.

On Windows system we advice the user to compile JETSPIN under the command-line interface Cygwin [21]. Finally, the binary executable file can be run in the *execute* sub-directory. Note that a Fortran compiler that supports the few Fortran 2003 features used by JETSPIN (Sec. 1.2) is necessary in order to compile it. Further, a Message Passing Interface Implementation should be also present, if the user wishes to compile JETSPIN in parallel mode. The Makefile supports GFortran, Intel Fortran and NVIDIA NVFORTRAN; we recommend GFortran for the CPU builds and NVFORTRAN 24.3 for the OpenACC build. As Message Passing Interface Implementation, we suggest to use that provided by the Open MPI Project.

target:	meaning:
gfortran	compile in serial mode using the GFortran compiler.
gfortran-mpi	compile in parallel mode using the GFortran compiler and the Open Mpi library.
gfortran-debugger	compile in serial mode using GFortran without optimization and with runtime checks (<code>-O0 -g -fcheck=all</code>).
gfortran-mpidebugger	as gfortran-debugger , in parallel mode with the Open Mpi library.
cygwin	compile in serial mode using the GFortran compiler under the command-line interface Cygwin for Windows.
cygwin-mpi	compile in parallel mode using the GFortran compiler and the Open Mpi library under the command-line interface Cygwin for Windows (note a pre-compiled package of the Open Mpi library is already available on Cygwin).
intel	compile in serial mode using the Intel compiler.
intel-mpi	compile in parallel mode using the Intel compiler and the Intel Mpi library.
intel-openmpi	compile in parallel mode using the Intel compiler and the Open Mpi library.
intel-debugger	compile in serial mode using the Intel compiler with runtime checks and traceback.
intel-mpidebugger	as intel-debugger , in parallel mode with the Intel Mpi library.
nvfortran	compile in serial CPU mode using NVIDIA Fortran; OpenACC offloading is disabled.
nvfortran-mpi	compile in parallel CPU mode using an MPI wrapper configured with NVIDIA Fortran; OpenACC offloading is disabled.
nvfortran-openacc	compile a single-GPU OpenACC executable; configure the device with <code>GPUCC</code> and <code>CUDA_VERSION</code> .
nvfortran-openacc-host	compile the OpenACC code path for host-only validation.
nvfortran-openacc-force-oracle	compile supported Euler, RK2, RK4, or Platen paths with the complete development-only host-force oracle.
nvfortran-openacc-coulomb-oracle	compile supported Euler, RK2, RK4, or Platen paths with only the direct Coulomb sum evaluated on the host.
nvfortran-openacc-host-akima	retain the historical host Akima routines as a development-only refinement oracle.
nvfortran-openacc-compare-akima	compare host and device Akima coefficients and interpolated values at every accepted refinement.
nvfortran-openacc-compare-refinement	compare host and device reconstruction of volume, mass, and charge at every accepted refinement.
help	return the list of possible target choices
clean	remove object and module files from the <i>source</i> sub-directory.

Table 1.1: List of targets for several common workstations and parallel computers, which can be used with the Makefile in the *build* sub-directory.

1.5 Running JETSPIN

In order to run the JETSPIN executable (`main.x`) the user should prepare the input file following the hints provided in Section 3.1. The test cases located in *examples* sub-directory can be used as starting point for editing the input file. As JETSPIN is executed, the program will look for the input file, named `input.dat`, in the same directory, where the executable was called. For instance, if JETSPIN is called in the *execute* sub-directory, the code will look for the file `input.dat`, which must have been put in the same sub-directory.

The JETSPIN code can be executed in serial mode by the command

```
./main.x
```

or in parallel mode by the command

```
mpirun -np $ncpu ./main.x
```

where `$ncpu` is the number of CPUs used in parallel (e.g. `mpirun -np 4 ./main.x` in order to run JETSPIN in parallel mode on 4 CPUs).

A successful run of JETSPIN will generate several output files, which appear in the directory, where the executable was called. The output files will be produced only if their writing was explicitly requested in input file by using proper directives (see Section 3.1). A detailed description of the generated output files is provided in Section 3.2. Briefly, the main output files are the files `statout.dat`, `traj.xyz` and `frame%06d.xyz`, which are human readable. The `statout.dat` file contains a catalogue of instantaneous or blocked averaged values of several variables, which can be used for temporal or statistical analysis in post-processing. The `traj.xyz` file provides a time ordered sequence of configurations to facilitate further analysis of the jet dynamics, while the `frame%06d.xyz` file, reports the same information in splitted files, one for each registered time frame. Further, as JETSPIN is running, a series of lines will be printed in the terminal window, providing several information as requested by suitable directives in input file (see Section 3.1). Also present will be the binary `save.dat` file, which is *not* human readable. It is written every *restart dump* steps (100,000 by default) and at the end of the run, and holds the bead state in double precision followed by the state of the run: the position in the Gaussian pool, the state of the random-number generator, the counters of the dynamic refinement, and whether the device step of the OpenACC build has engaged. Together with `input.dat` it restarts the job (see Section 1.6).

1.5.1 Running on a GPU

The executable of the `nvfortran-openacc` target (Sec. 1.4) is a serial program that uses one GPU; it is started as `./main.x`. It moves the whole time step to the GPU (the device step) for the Euler, RK2, and RK4 integrators on `system 3`, Maxwell or Kelvin-Voigt, with or without evaporation and air drag, without dynamic refinement (or the other options that tag beads), and for the Platen integrator on `system 4`, Maxwell, with or without evaporation, insertion, and dynamic refinement. Multiple-step Coulomb sums, time-dependent external fields, the drag velocity, bead tracking, fixed bead sets with *removing yes*, one-dimensional systems, MPI runs, and the developer options Lorentz force, upper potential (*wall*), variable mass and breakup are not supported yet.

The device step engages when `npjet`, the index of the nozzle bead, reaches 100 (it counts the beads in the arrays, collected ones included, not only the active ones), and stays engaged when a compaction lowers it and across a restart, since `save.dat` records it. The terminal then shows **OpenACC device step engaged (scheme, model) at step N with M active beads**, or **OpenACC device step resumed (...)** followed by **engaged before the restart** in a restarted run. Below that point the executable runs the code of the CPU build and reproduces, byte for byte, the results of a CPU build of the same compiler version. Above it the results agree with the CPU build within roundoff: the device sums accumulate in another order, so trajectories that amplify roundoff separate. The runs that the device step does not support execute the CPU code too, with only the direct Coulomb sums offloaded once the jet has at least 128 active beads. A device stage that cannot evaluate the equations of motion of a run stops it with error 22, a guard that a supported run does not reach.

The following environment variables, set to 1 unless stated otherwise, change this behaviour:

- `JETSPIN_OPENACC_COULOMB_MIN_BEADS=n` sets the 128-bead threshold of the offloaded Coulomb sums (0 offloads every call);
- `JETSPIN_OPENACC_DISABLE_PERSISTENT` or `JETSPIN_OPENACC_DISABLE_EOM` keeps the device step closed, so that the run executes the CPU code with the offloaded Coulomb sums above;
- `JETSPIN_OPENACC_DISABLE_COULOMB` keeps the one-dimensional Coulomb sum on the host;

- JETSPIN_OPENACC_SYNC makes the Platen device step of a run with insertion synchronous (the other device steps always are);
- JETSPIN_PROFILE (also `yes`, `true`, `on`) prints the wall time of the main parts of the time loop at the end of the run;
- JETSPIN_TOPOLOGY_SNAPSHOT writes the active state at every insertion and removal to `topology-state.dat`.

The last two act in the CPU builds too. Two development variables, which set the spare capacity of refining runs, are described in `docs/introduction/compiling.md`. For timing, the process should be bound to the NUMA node of its GPU (`nvidia-smi topo -m` lists it), for instance with `numactl -cpunodebind=N -membind=N ./main.x`: the placement of the host process alone changes the time of latency-bound runs by up to 8%.

1.6 Restarting JETSPIN

A simulation can be restarted after normal termination or after it was stopped by its time limit (the directives *job time* and *close time*, measured as elapsed wall-clock time; see Section 3.1). In order to restart a JETSPIN job the file `input.dat` is again required together with the file `restart.dat`, which is the exact copy of the `save.dat` file created by the previous job. In particular, the `save.dat` file was written by JETSPIN every `restart dump` steps (100,000 by default) and at the end of the previous simulation. The file `save.dat` is *not* human readable, and it should be renamed as `restart.dat`, and located in the same directory of input file. If the user attempts to restart JETSPIN without the `restart.dat` file, the job will immediately exit with error. Note that the directive “*restart yes*” needs to be added in input file in order to restart a previous JETSPIN job (see Section 3.1); the directive “*restart reset*” is still accepted but has no effect (see below). It is worth stressing that JETSPIN will append new data to the files `statout.dat` and `traj.xyz` (and to the developer binaries `statdat.dat`, `traj.dat` and `bead.dat`, if enabled), creating them if they are missing, while all the other output files will be **overwritten** (included the `save.dat` file). Thus, the user is advised to keep backup copies of the `restart.dat` file, noting the times it was written, to help you avoid going right back to the start of a simulation.

The same restarting procedure can be also used for extending a simulation beyond its initial duration. The restarted job resumes from the step and time stored in `restart.dat` and integrates up to the value of the directive `final time`, which should therefore be increased to the new total simulated time. For instance, if a simulation run with “*final time 1.d-3*” needs to be extended by a further 5×10^{-4} seconds, the user should set “*final time 1.5d-3*” and include the “*restart yes*” directive, leaving `timestep` unchanged. Note that there is no directive for the number of steps: since directives are matched by the leading characters of each word, a record such as “*timesteps 15000*” would be read as “*timestep 15000*” and would set the integration step to 15000 seconds. Further, the restarting procedure can be useful in the event of machine failure. Here, the job can be restarted in the same way from the `save.dat` file, which is written every `restart dump` steps in order to meet just such an emergency. We stress that the procedure is not foolproof in the last event, since the job could be crash while the `save.dat` file is being written.

Since version 2.0 the restart file stores the bead state in double precision, followed by the state of the run that the beads do not carry: the position in the pre-generated Gaussian pool (Sec. 2.5.1) and the number of times it has been read through, the state of the random-number generator, the counters of the dynamic refinement and of the anchor tagging, and whether the device step of the OpenACC build has engaged. This block carries a version number (currently 2); a file of an earlier version, written by development builds, restarts with the device gate closed. If the saved state of the random-number generator has a size different from this executable’s (as with another compiler), it is not restored and a warning is printed. With the same `input.dat` and the same executable, a restarted simulation therefore continues the uninterrupted one exactly, with or without evaporation; the script `tests/restart/run.sh` checks this for Tests 12, 13, 16, 17, 24 and 25 and Examples 4 and 8. The statistics accumulated since the last print are not restored, so that every printed row is reproduced when the restart interval is a multiple of the print interval; the directive “*restart reset*” has no further effect. Restart files written by JETSPIN 1.22 or earlier are still read, in single precision and without the state of the run, so that the random sequence restarts from the beginning; those versions could not restart a simulation with evaporation.

Chapter 2

JETSPIN Model and Algorithms

Scope of Chapter

This chapter describes the model and simulation algorithms incorporated into JETSPIN.

2.1 Equations of Motion

The model implemented in JETSPIN is an extension of the Lagrangian discrete model introduced by Reneker et al. [11] (in the following text referred as Ren model). The model provides a compromise of efficiency and accuracy by representing the filament as a series of n beads (jet beads) at mutual distance l connected by viscoelastic elements. The length l is typically larger than the cross-sectional radius of the filament, but smaller than the characteristic lengths of other observables of interest (e.g. curvature radius). Each i -th bead has mass m_i and charge q_i (not necessarily equal for all the beads). The beads are numbered from the nozzle ($i = 0$, Sec. 2.2) towards the leading bead of the jet ($i = n$). The nanofiber is modeled as a viscoelastic fluid following the approach developed in Ref. [22], and the stress σ_i for the element connecting bead $i - 1$ with bead i is given by the viscoelastic constitutive equation:

$$\frac{d\sigma_i}{dt} = -\frac{1}{t_0}(\sigma_i - \sigma_{HB}), \quad (2.1)$$

where t_0 is the relaxation time and σ_{HB} is the Herschel–Bulkley stress. Note that the relaxation time is give as

$$t_0 = \frac{\mu}{G}, \quad (2.2)$$

where G is the elastic modulus and μ the viscosity of the fluid jet, while σ_{HB} reads

$$\sigma_{HB} = \sigma_Y + K_{ve} \left(\frac{1}{l_i} \frac{dl_i}{dt} \right)^{n_{ve}}. \quad (2.3)$$

In the previous expression, K_{ve} is the flow consistency index with dimensions $\text{gs}^{ne-2}\text{cm}^{-1}$, σ_Y the yield stress, n_{ve} the flow behavior index, t the time, and l_i the length of the element connecting bead $i - 1$ with bead i . Note that it is possible to define an effective viscosity μ_{eff} as

$$\mu_{eff} = K_{ve} \left(\frac{1}{l_i} \frac{dl_i}{dt} \right)^{n_{ve}-1}. \quad (2.4)$$

It is worth stressing that the case $K_{ve} = \mu$, $n_{ve} = 1$ and $\sigma_Y = 0$ recovers the Maxwellian fluid model. Keeping the last case in mind, for convenience sake, we write

$$K_{ve} = k_{ve}\mu, \quad (2.5)$$

with k_{ve} the flow consistency index re-scaled by μ , and the viscoelastic constitutive equation is rearranged as

$$\frac{d\sigma_i}{dt} = -\frac{G}{\mu} \left[\sigma_i - \sigma_Y - k_{ve}\mu \left(\frac{1}{l_i} \frac{dl_i}{dt} \right)^{n_{ve}} \right], \quad (2.6)$$

The last constitutive equation allows the modeling of several fluids as, for example, Bingham fluids ($\sigma_Y > 0$), shear-thinning fluids ($n_{ve} < 1$), shear-thickening fluids ($n_{ve} > 1$).

As alternative, it is possible to activate the Kelvin–Voigt model [23] (see Tab 3.1). In the last case, Eq 2.6 is replaced by

$$\frac{d\sigma_i}{dt} = G \frac{1}{l_i} \frac{dl_i}{dt} + \mu \frac{1}{l_i} \frac{d^2 l_i}{dt^2}, \quad (2.7)$$

where the second derivative $d^2 l_i / dt^2$ is evaluated as the relative acceleration of beads $i - 1$ and i projected on the element.

Given $a_i = [V_i / (\pi l_i)]^{1/2}$ the cross-sectional radius of the element i , where V_i is the jet volume represented by bead i , the viscoelastic force $\vec{f}_{ve}$ pulling bead i back to $i - 1$ and towards $i + 1$ is

$$\vec{f}_{ve,i} = -\pi a_i^2 \sigma_i \cdot \vec{t}_i + \pi a_{i+1}^2 \sigma_{i+1} \cdot \vec{t}_{i+1}, \quad (2.8)$$

where $\vec{t}_i$ is the unit vector pointing bead i from bead $i - 1$. In the three-dimensional stochastic model (**system 4**) an element under compression ($\sigma_i \leq 0$) exerts no viscoelastic force on its beads; **system 1** and **system 3** apply the force for either sign of σ_i . The surface tension force $\vec{f}_{st}$ for the $i - th$ bead is given by

$$\vec{f}_{st,i} = k_i \cdot \pi \left(\frac{a_i + a_{i+1}}{2} \right)^2 \alpha \cdot \vec{c}_i, \quad (2.9)$$

where $(a_i + a_{i+1}) / 2$ is the mean radius of the two elements joined at bead i , α is the surface tension coefficient, k_i is the local curvature, and $\vec{c}_i$ is the unit vector pointing the center of the local curvature from bead i . Note the force $\vec{f}_{st}$ is acting to restore the rectilinear shape of the bending part of the jet.

In the electrospinning experimental configuration an intense external electric potential Φ_0 is applied between the spinneret and a conducting collector located at distance h from the injection point. As consequence, each $i - th$ bead endures the electric force

$$\vec{f}_{el,i} = q_i \frac{\Phi_0}{h} \cdot \vec{x}, \quad (2.10)$$

where $\vec{x}$ is the unit vector pointing the collector from the spinneret and assumed along the x axis. Note that in Ren model the electric field Φ_0 is assumed to be static in order to avoid the computationally expensive integration of Poisson equation, whereas in reality Φ is depending on the net charge of the nanofiber so as to maintain constant the potential at the electrodes. The latter issue was elegantly addressed by Kowalewski et al. [24], and its implementation in JETSPIN will be planned. Further, it is possible to impose a time-dependent external electric field (see Section 2.6).

A bead that reaches the collector ($x_i \geq h$) discharges on the grounded electrode: it is held at $x_i = h$, is no longer integrated, and is excluded from the Coulomb sums (released builds compute no image charges), while it stays joined to the jet by its viscoelastic element. With the directive **removing yes** the collected beads are also deleted from the jet arrays, the leading one as soon as the next one reaches the collector; without it they remain on the collector as deposited fiber, which the dynamic refinement ignores (Sec. 2.4). In versions up to 1.22 a run without removal froze nothing: its beads crossed the collector plane keeping their charge and their Coulomb interactions, and a jet with dynamic refinement could fail a few hundred steps after a remesh.

The net Coulomb force $\vec{f}_c$ acting on the $i - th$ bead from all the other beads is given by

$$\vec{f}_{c,i} = \sum_{\substack{j=0 \\ j \neq i}}^n \frac{q_i q_j}{(R_{ij} + a_{ij})^2} \cdot \vec{u}_{ij}, \quad (2.11)$$

where $R_{ij} = \left[(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2 \right]^{1/2}$, $\vec{u}_{ij}$ is the unit vector pointing the i -th bead from j -th bead, and $a_{ij} = a_{\min(i,j)}$ is the cross-sectional radius of the bead of the pair nearer to the nozzle (a_0 for the nozzle bead and, with bead insertion, also for the one or two beads closest to the nozzle). The sum includes the nozzle bead ($j = 0$) and skips the beads frozen at the collector. Note that the distance between the two charges is increased by the fiber radius a_{ij} , since the charge is assumed to lie on the outer shell of the fiber and not on its center line (the particle-like point) as implemented in the original Ren model. Although the Ren model provides a reasonable description for the spiral motion of the jet, the last term $\vec{f}_c$ introduces mathematical inconsistencies due to the discretization of the fiber into point-charges. Different approaches were developed to overcome this issue which usually imply strong approximations[25, 10, 26]. Other strategies use a less crude approximation by accounting for the actual electrostatic form factors between two interacting sections of a charged fiber[27] or involving more sophisticated methods which exploit the tree-code hierarchical force calculation algorithm[24]. In JETSPIN it is possible to activate the dynamic refinement algorithm to recover a finer discretization of the jet (see Sec 2.4).

Although usually much smaller than the other driving forces, the body force due to the gravity can be included with the directive `gravity yes` (default: no) by the usual expression

$$\vec{f}_{g,i} = m_i g \cdot \vec{x}, \quad (2.12)$$

where g is the gravitational acceleration.

As shown in experimental evidences [28], the air drag force affects the dynamics of the nanofiber. The extended stochastic model developed in Refs [29, 30, 31] was included in JETSPIN to reproduce the aerodynamic effects. In particular, the air drag is modeled by adding two force terms: a random term and a dissipative term. The dissipative air drag term is usually dependent on the geometry of the jet, which changes in time, and it combines longitudinal and lateral components. Based on experimental results[28], the longitudinal component of the air drag dissipative force term acting on bead i is

$$\vec{f}_{air,i} = -0.65\pi a_i l_i \rho_a (2a_i/\nu_a)^{-0.81} |v_{tan,i}|^{1+0.19} \cdot \vec{t}_i \quad (2.13)$$

where ρ_a and ν_a are the air density and kinematic viscosity, respectively, $\vec{t}_i$ is the unit vector pointing bead i from bead $i-1$, and $v_{tan,i} = (\vec{v}_i - u_a \vec{x}) \cdot \vec{t}_i$ is the projection along $\vec{t}_i$ of the velocity of bead i relative to the air, which moves with velocity u_a along x (directive `airdrag airvelocity`, default 0). The force is always directed along $-\vec{t}_i$. Further, it is possible to write $\vec{f}_{air,i}$ as

$$\vec{f}_{air,i} = -m_i \gamma_i l_i^{0.905} |v_{tan,i}|^{1+0.19} \cdot \vec{t}_i \quad (2.14)$$

if we introduced the dissipative coefficient γ_i equal to

$$\gamma_i = 0.65\pi^{0.905} \frac{\rho_a}{m_i} \left(\frac{2}{\nu_a} \right)^{-0.81} V_c^{0.095}, \quad (2.15)$$

where $V_c = \pi a_0^2 l_{step}$ is the nominal jet volume represented by a bead (Sec. 2.4): JETSPIN evaluates a_i in the drag force as $[V_c/(\pi l_i)]^{1/2}$ for every bead, also when the dynamic refinement or the evaporation change the volume V_i represented by the bead, so that only m_i and l_i vary from bead to bead.

Following the expression introduced by A. Yarin[32, 6], under a high-speed air drag the lateral component $\vec{f}_{lift,i}$ of the aerodynamic dissipative force related to the flow speed is given in the linear approximation (for small bending perturbations) by

$$\vec{f}_{lift,i} = -l_i \cdot k_i \rho_a v_{tan,i}^2 \pi \left(\frac{a_i + a_{i+1}}{2} \right)^2 \cdot \vec{c}_i, \quad (2.16)$$

where k_i is still the local curvature, and $\vec{c}_i$ is the unit vector pointing the center of the local curvature from bead i . The combined action of such longitudinal and lateral components provide the dissipative force term acting on the i -th bead

$$\vec{f}_{diss,i} = \vec{f}_{air,i} + \vec{f}_{lift,i}. \quad (2.17)$$

Instead, the random force term for the i -th bead has the form

$$\vec{\mathbf{f}}_{rand,i} = \sqrt{2m_i^2 D_{v,i}} \cdot \vec{\eta}_i(t), \quad (2.18)$$

where $D_{v,i} = A \gamma_i$ denotes a diffusion coefficient in velocity space, with A the amplitude given by the directive `airdrag amplitude` and γ_i the coefficient of Eq. 2.15, so that $D_{v,i}$ is inversely proportional to the bead mass m_i . This random term is computed only by the three-dimensional stochastic model (`system 4`) with the Platen integrator (`integrator 4`); there `airdrag amplitude` is mandatory (warning 33), while with other systems or integrators it is rejected (warnings 34 and 35). Here $\vec{\eta}_i$ is a three-dimensional vector, whereof each component η is an independent stochastic process that is nowhere differentiable with $\langle \eta(t_1) \eta(t_2) \rangle = \delta(|t_2 - t_1|)$, and $\langle \eta(t) \rangle = 0$. Note that for the sake of simplicity we assume $\eta = d\omega(t)/dt$, where $\omega(t)$ is a Wiener process. Further details are available in Ref [33].

The combined action of these forces governs the elongation of the jet according to the Newton's equation providing a non-linear Langevin-like stochastic differential equation:

$$m_i \frac{d\vec{v}_i}{dt} = \vec{\mathbf{f}}_{el,i} + \vec{\mathbf{f}}_{c,i} + \vec{\mathbf{f}}_{ve,i} + \vec{\mathbf{f}}_{st,i} + \vec{\mathbf{f}}_{g,i} + \vec{\mathbf{f}}_{diss,i} + \vec{\mathbf{f}}_{rand,i}, \quad (2.19)$$

where $\vec{v}_i$ is the velocity of the i -th bead. The velocity $\vec{v}_i$ satisfies the kinematic relation:

$$\frac{d\vec{\mathbf{r}}_i}{dt} = \vec{v}_i \quad (2.20)$$

where $\vec{\mathbf{r}}_i$ is the position vector of the i -th bead, $\vec{\mathbf{r}}_i = x_i \vec{\mathbf{x}} + y_i \vec{\mathbf{y}} + z_i \vec{\mathbf{z}}$. The three Eqs 2.6, 2.19 and 2.20 form the set of equations of motion (EOM) governing the time evolution of system. It is worth stressing that Eq 2.19 recovers a deterministic EOM by imposing $\rho_a = 0$ and $D_{v,i} = 0$.

2.2 Perturbations at the nozzle

The spinneret nozzle is represented by a single mass-less point of charge $\bar{q}$ fixed at $x = 0$ (nozzle bead). Its charge is the nominal bead charge $\bar{q} = \rho_V \pi a_0^2 l_{step}$ (Sec 2.7). Such charged point can be also interpreted as a small portion of jet which is fixed to the nozzle. In JETSPIN it is possible to add small perturbations to the y_n and z_n coordinates of the nozzle bead in order to model fast mechanical oscillations of the spinneret[17] (directive `perturb yes`, only with `system 3` and `system 4`). Given the initial position of the nozzle

$$y_n = A \cdot \cos(\varphi) \quad (2.21a)$$

$$z_n = A \cdot \sin(\varphi), \quad (2.21b)$$

the equations of motion for the nozzle bead are

$$\frac{dy_n}{dt} = -\omega \cdot z_n \quad (2.22a)$$

$$\frac{dz_n}{dt} = \omega \cdot y_n, \quad (2.22b)$$

where A denotes the amplitude of the perturbation (`perturb ampl`), while ω and φ are its angular frequency and initial phase, respectively. The value of `perturb freq` is used directly as the angular frequency ω in rad/s, without a factor 2π . The initial phase φ is drawn uniformly in $[0, 2\pi)$ at the start of the run by rank zero from the random seed of the input file.

2.3 Jet insertion

The jet insertion at the nozzle is modeled as follows. For the sake of simplicity, let us consider a simulation which starts with only two bodies: a single mass-less point fixed at $x = 0$ representing the spinneret nozzle, and a bead modeling an initial jet segment of mass m_i and charge q_i located at distance l_{step} from the nozzle along the x axis. Here, l_{step} denotes the length step used to discretize the jet in a sequence of beads. The starting jet bead is assumed to have a cross-sectional radius a_0 , defined as the radius of the filament at the nozzle before the stretching process. Applying the condition that the volume of the jet is conserved, the relation $\pi a_i^2 l_i = \pi a_0^2 l_{step}$ is valid for any i -th bead. Furthermore, the starting jet bead has an initial velocity v_s along the x axis equal to the bulk fluid velocity in the syringe needle. Once this traveling jet bead is a distance of $1.5 \cdot l_{step}$ away from the nozzle, a new jet bead (third body) is created at distance l_{step} from the nozzle along the straight line joining the two previous bodies. The new bead is held at distance l_{step} from the nozzle on the line joining the nozzle to the traveling bead, and its charge grows from zero to the nominal value $\bar{q} = \rho_V \pi a_0^2 l_{step}$ (Sec 2.7) as $\bar{q} [1 - \cos(\pi(d - 1.5 l_{step}) / (0.5 l_{step}))] / 2$, where d is the distance of the traveling bead from the nozzle. When the traveling bead reaches the distance $2 \cdot l_{step}$, the new bead is released with zero stress on its element, so that the stress given by **nozzle stress** applies only to the beads of the initial jet. Let us now label $i + 1$ the farthest bead from the nozzle, and i the last inserted bead. By default the i -th bead is released with the initial velocity v_s along the x axis. If the directive **dragvel yes** is given (see Tab 3.1), it is instead released with the initial velocity $v_i = v_s + v_d$, where v_d denotes the dragging velocity computed as

$$v_d = \frac{v_{i+1} - v_s}{2}. \quad (2.23)$$

Here, the dragging velocity should be interpreted as an extra term which accounts for the drag effect of the electrospun jet on the last inserted segment. Note that the actual dragging velocity definition was chosen in order to not alter the strain velocity term $(1/l_{i+1}) \cdot (dl_{i+1}/dt)$ of Eq 2.6 before and after the bead insertion. The directive **dragvel yes** disables the device step of the OpenACC build and the pre-generated Gaussian pool (Sec 2.5.1).

2.4 Dynamic refinement

In the original Ren model, each bead represent a constant quantity V_c of jet volume, so that the relationship $\pi a_i^2 l_i = \pi a_0^2 l_{step} = V_c$ is valid for any i -th bead. Despite such relationship can be sometimes a useful advantage (as example, see Sec 2.7), the radius reduction a_i of the electrospun nanofiber provides a significant increase of the discretization length l_i value along the jet path (also greater than two order of magnitude). As consequence, it was observed in literature [24] that the jet discretization close the collector becomes rather coarse to model efficiently the nanofiber. To get around this problem the dynamic refinement procedure developed in Ref [29] can be used in JETSPIN in order to maintain the the length of the elements l_i below a prescribed maximum length l_{max} , set by the directive **dynam refin threshold**. Whenever an element length l_i is larger than l_{max} , the nanofiber is remeshed from the leading bead (or from the contact bead at the collector, see below) up to the collector-side bead of the element nearest to the nozzle that is longer than l_{max} ; that element keeps its length, and the beads between it and the nozzle keep their positions, their λ values and their volumes. The base insertion spacing l_{step} does not define the refined mesh. The discretization is performed by Akima spline interpolations [34, 35] of the main quantities describing the jet beads (positions, velocities, stress, mass and charge densities, fiber radius and, with evaporation, the evaporation radius; both radii are fitted as $\ln(\pi a^2)$, see Sec 2.4.1). In order to perform the interpolation we proceed following three steps. First of all, we introduce the variable $\lambda \in [0, 1]$ to parametrize the jet path, where $\lambda = 0$ identifies the leading bead of the jet, and $\lambda = 1$ the nozzle. For each bead we compute its respective λ_i value by using the formula

$$\lambda_i = \frac{1}{l_{path}} \sum_{k=i+1}^{N_{beads}} l_k \quad (2.24)$$

where $l_{path} = \sum_{k=1}^{N_{beads}} l_k$ is the jet path length from the nozzle to the leading bead, $i = 1$ and $i = N_{beads}$ denote the jet bead at the nozzle and the leading bead, respectively ($i = 0$ denotes the nozzle), and l_k is the length of the element joining bead $k - 1$ with bead k (Sec 2.1). The set of λ_i values represent the mesh used to build the cubic spline. Given a generic quantity y , the data $y_i = y(\lambda_i)$ are tabulated and used in order to compute the coefficients of the *Akima spline*, following the algorithm reported in Ref [35]. Then, we enforce a target parametrization of the remeshed portion of the nanofiber, $\lambda_a \leq \lambda \leq \lambda_e$, where λ_a is 0 at the leading bead (or the value of the contact bead) and λ_e the value at its nozzle-side end. Without interior anchors, the new mesh λ_i^* can be represented schematically as

$$\lambda_i^* = \lambda_a + \frac{i}{N_{beads}^*} (\lambda_e - \lambda_a), \quad i = 0, 1, \dots, N_{beads}^*, \quad (2.25)$$

where $N_{beads}^* = \lceil (\lambda_e - \lambda_a) l_{path} / l_{max} \rceil$. In the implemented algorithm, each anchored segment of arc length L_s is divided in the same way into $\lceil L_s / l_{max} \rceil$ equal intervals. Finally, the new values $y(\lambda_i^*)$ are computed for any i -th bead by the *spline interpolation*. The procedure is repeated for all the interpolated fields in order to provide the respective quantities in the new mesh λ_i^* . It is worth to dressing that the volume $V_i = \pi a_i^2 l_i$ represented by each bead is not anymore constant, and, consequently, $\pi a_i^2 l_i \neq \pi a_0^2 l_{step}$. When the remeshed portion ends at the nozzle-side end of the jet, so that no kept bead follows it, the volume of its last bead is computed with the length of the element on its collector side, or with l_{step} if the portion holds a single bead. It is also possible to keep selected beads in their positions before and after the refinement procedure in order to reduce possible errors introduced by the interpolation procedure. In other words, these anchor beads are maintained as knots of both the old and new meshes (see directive **dynam refin anchor** in Tab 3.1). If that spacing is omitted it defaults to five times the discretization resolution; an explicit value set below one resolution is likewise raised to that same $5 \times$ floor. In the one-bead startup they are tagged as material is inserted. For a pre-extended initial jet, interior anchors are initialized at the requested spacing before the first timestep. The path endpoints are already common to both meshes and are not tagged as interior anchors. An anchor is an ordinary material bead and does not imply nanoparticle mass unless the separate variable-mass model is enabled. Its role is to provide a persistent interpolation knot; it is not spatially frozen during time integration. In the following text we will refer to this emended version of the Ren model as DyRen model. Further details on the implemented dynamic refinement procedure are available in Ref [29].

The refinement is tested and accepted according to the following rules. No refinement is attempted before the time given by **dynam refin start**, nor, when bead insertion is active, while the jet contains fewer than ten active beads. The directive **dynam refin every** sets the minimum interval between two accepted refinements, not a fixed sampling period: once that interval has elapsed, the threshold is tested at every time step until a refinement is accepted, and only then is the interval counter reset. A threshold crossing alone is not sufficient: the refinement is accepted only if the estimated target mesh contains more points than the current active mesh. The number of accepted refinements is available as the output key **nref** (see Tab 3.2). After an accepted refinement the beads at or beyond the collector are frozen on it, as at every time step, with or without bead removal. In versions up to 1.22 the time step froze them only with removal, so that a jet could cross the collector plane between two refinements and be pulled back at the next one. Without removal the beads collected on the collector form a deposited fiber that stays in the jet arrays; the refinement ignores it: the acceptance test and the remeshed path start at the contact bead, the last collected bead joined to the free jet, and the deposited beads are kept unchanged at their indices. A run without removal therefore accepts the same refinements as the same run with removal (Test Case 23 with and without removal: same events, same steps).

For the serial Maxwell/Platen OpenACC path exercised by Test Case 21, the time integrator, Gaussian pool, insertion logic, and bead state remain in a persistent device data region. After the configured minimum interval, a device reduction returns only the last segment exceeding the threshold, the total path length, and the nozzle correction used by the mesh-size test. No bead array is transferred during an ordinary check. When a denser mesh is actually possible, the active topology and evaporation state are downloaded once. The host computes the normalized source and target coordinates. The GPU then computes the interval slopes, Akima tangents, cubic coefficients, and all field interpolations. Slopes, interior tangents, coefficient sets, and target knots are independent parallel work; only the constant-size endpoint-slope extrapolation is serial. The interpolated cross-section radius then drives a second device kernel that reconstructs bead volume and evaporation volume, rescales each for reference-volume conservation, and converts the interpolated

mass/charge densities back to per-bead quantities. The host retains only the data-dependent, rare bookkeeping that these device kernels build on and consume: the normalized target-mesh construction and anchor save/restore/final-assembly rules, after which the completed mesh is uploaded once and device integration resumes. The normal Test 21 mesh reserves one allocation increment. A capacity-growth validation reduces that reserve and forces the accepted mesh beyond the old bound. In this case the topology and evaporation mappings are released before the host arrays are replaced; the Platen and Coulomb workspaces are resized and the completed state is bound once. The pre-generated Gaussian pool does not depend on capacity and stays mapped unchanged.

This allocation increment is the `dynam refin capacity i` input directive (default 100 beads if omitted; a value below 1 gives warning 107 and is reset to 100). It sets both the headroom reserved for a pre-extended anchored jet and the growth increment applied whenever an accepted refinement event exceeds the current capacity.

Test Case 22 exercises this event lifecycle three times. Developer-only 20-entry reserve and growth overrides force three successive capacity increases, while the normal production increment remains 100. Each event preserves every active anchor and separately conserves reference volume, evaporated volume, mass, and charge. This repeated test verifies the mapping lifecycle together with the device Akima spline.

Test Case 23 adds collector removal to the same lifecycle. Removal advances the active lower bound and clears the collected bead through the device topology primitive; it does not download the full jet. The validation requires removal both before and after the third Akima event, so repeated device interpolation and capacity replacement are checked with a rebased active interval. Complete intermediate state transfers remain confined to accepted refinement events.

The native GPU path reproduces the CPU events, the nine removals, and the final 526 active elements step for step, and its statistics agree with the CPU within 8.0×10^{-10} relatively in all 81 rows, including the rows written after the jet has reached the collector. Before the persistent-path fixes of 30 September 2026 the device kept the full charge on the bead being inserted at the nozzle and let a bead frozen at the collector keep moving, and the native events and removal schedule differed from the CPU ones. Matching topology and conservation – not bit-for-bit trajectory identity – remain the acceptance criteria for this test, since the device kernels still evaluate in a different floating-point order.

A development host-Akima target retains the historical coefficient and interpolation routines as an oracle. A second comparison target executes both paths for each field. Across the 11 fields and three Test Case 23 events, the largest coefficient relative difference is 1.03×10^{-14} ; interpolated values differ by at most 1.82×10^{-12} absolutely and 1.89×10^{-15} relatively.

A third development target applies the same host-then-device oracle pattern to the volume-reconstruction/conservation/der kernel: it backs up the pre-reconstruction state, evaluates the trusted host reference, restores that state, then runs the device kernel so it remains authoritative, and compares. All three Test Case 23 events agree with the host reference at or near roundoff (worst absolute 8.9×10^{-16} , worst relative 1.4×10^{-16}). Building this oracle exposed two ordering mistakes before they reached the standard path – the device kernel invoked twice, and the host reference evaluated from already-converted state – each of which corrupted the event’s mass/charge invariant by tens of percent, which is why every new device stage in this port is validated with such an A/B comparison before being trusted.

2.4.1 Numerical robustness of the cross-section reconstruction

A long production run growing from a single nozzle bead through several dozen consecutive accepted refinement events (an untracked development case combining the Test 23 physics with a much longer integration window) exposed two distinct numerical weaknesses of the reconstruction described above, neither of which is visible in the short Test 21–23 validation windows.

The first concerns the evaporated-volume floor. Every ordinary per-timestep integrator already enforces $\bar{V}_i/\bar{V}_i^0 \geq \text{evlim}$ (Sec. 2.8), but the accepted-event reconstruction of $\bar{V}_i$ from the interpolated evaporation radius did not. An Akima undershoot on that field could leave the ratio far below `evlim` – observed directly, to about one seventh of it, uniformly across nearly the whole active jet in a single event – inflating the polymer mass fraction c_p well past unity and driving the Maxwell evaporative stress equation toward overflow. The floor is now restored by a water-filling redistribution after the existing conservation rescale: beads below their own floor are clamped up to it, and the resulting deficit is taken from the still-compliant beads in proportion

to their surplus, iterated in case that step creates a new violator, so that the segment’s evaporated-volume total is preserved to roundoff rather than broken, as a naive clamp alone was found to do. If the floor is mathematically incompatible with the segment total, the event is aborted rather than silently violating either constraint.

The second weakness is independent of evaporation and reproduces with it disabled: the cross-section radius a_i (fit independently by Akima and then squared into $V_i = \pi a_i^2 l_i$, and through the bead density into its mass) can compound a nonphysical thinning across *successive* accepted events – each one re-fitting from the previous event’s own output – well beyond the order-of-magnitude fibre thinning electrospinning genuinely produces, eventually collapsing bead mass toward zero. This was isolated to the radius/area reconstruction specifically: the interpolated position field (and the element lengths it implies) shrinks only modestly and non-monotonically over the same run, while stress and velocity only show large excursions in the single event immediately preceding a failure, consistent with a late consequence of the near-zero mass (via $F = ma$) rather than an independent defect of their own interpolation.

Five defenses were added to the cross-section fit, none of which alone has eliminated the failure on the long run that exposed it, each validated against Tests 21–23 with zero regression. First, the fit is performed on $\ln(\pi a_i^2)$ rather than the raw radius or area: the reconstructed volume is linear in area but quadratic in radius, so fitting area removes one amplification of any interpolation error, and fitting its logarithm additionally guarantees a strictly positive recovered value by construction – a raw area fit was observed to undershoot past zero – while linearising the roughly exponential thinning trend, which is far better conditioned for the Akima tangent estimate than the area’s own steeper curvature. Second, an isolated single-event interpolation spike is removed by a three-point median filter over the fitted target segment, excluding anchor points, which must remain exact. Third, each non-anchor target point is clamped to the range spanned by its two immediate bracketing source points before the event, a local approximation of the shape-preserving interpolation guarantee classic Akima does not provide; this was confirmed unable, alone, to catch a drift already compounded through earlier events, since that drift is by then baked into the “legitimate” source data too. Fourth, interior points are instead bounded between their two bracketing *anchor* beads, with a generous margin, since anchors are restored exactly at every event; this was found insufficient in isolation because an anchor can itself already be tagged from an already-degraded bead – anchors are frozen at tagging time, not validated against anything. Fifth, the Akima knot tangents for this field are limited to satisfy the classic Fritsch–Carlson sufficient condition for a monotone cubic Hermite interpolant, applied only to the cross-section and density fields (never position, velocity, or stress, which may have genuine local extrema); this delayed the failure the most of any single change tried, but the compounding drift still eventually recurs across enough events, possibly a cumulative resampling effect distinct from classic Akima overshoot and not yet isolated. A sixth, absolute-floor variant of the evlim water-filling (default 1 nm) is also present but was found too permissive to engage on this input, since electrospinning legitimately reaches sub-micron fibre radii; it remains as a defense against a genuine sign crossing the other layers might miss.

The practical driver of the compounding drift on this specific long run turned out to be its refinement *cadence*, not a defect the five interpolation-side defenses above could fully absorb. The run combined a tight threshold (0.10 cm, five times the base resolution) with a short minimum interval between accepted events (10^{-5} s), so growing the jet from a single nozzle bead accumulated several dozen accepted refinement events – each re-fitting the cross-section from the previous event’s own output – within under 2×10^5 timesteps. Reusing Example 5’s established, coarser cadence instead (0.4 cm threshold, i.e. twenty times the resolution, and a 10^{-3} s minimum interval) let the same input, on GFortran CPU, run to full completion: all 10^8 (one hundred million) timesteps, physical time $t = 0.5$ s, 499 accepted refinement events, closing with zero errors. The jet bridges the full nozzle-to-collector distance and reaches its first collector removal at step 95528, under 0.1% into the run, and the active-bead count is already in its final stationary regime from that point on: the first accepted refinement event (step 249999) and the last (step 99849999) report the identical active-bead pattern, with no drift across the full run and the minimum bead radius staying at its initial value at every one of the 499 events. This is the first time this development probe demonstrated, over its complete intended duration, the stationary insertion/removal oscillation it was designed to reach. (The OpenACC/GPU path has been validated to 2.5×10^6 of the 10^8 steps at this cadence but not yet driven to full completion; see `docs/STATE.md`.) The same cadence is now used by the tracked Test Cases 24 and 25, which differ from the probe: they adopt the canonical JETSPIN electrostatics and a 16 cm collector distance. Test Case 24, without evaporation, runs cleanly, and so does its evaporative counterpart, Test Case

25, with a 50% initial polymer fraction. With the 6% fraction of Ref. [36] the evaporative case stops on a numerical instability that originates in the evaporation parameter regime rather than in the refinement (see Sec. 4.24). The cadence remains a workaround for the cross-event compounding, not a complete cure. The full investigation is recorded in `docs/refinement-robustness-investigation.md`. A corresponding advisory, warning 108, is issued whenever the configured or auto-derived `dynam refin threshold` is set below twenty times the discretization resolution. It does not change behaviour, since the Tests 21–23 references use a threshold below that recommendation (0.10 cm, five times the resolution of 0.02 cm) and remain validated short-window cases (Example 5 and Tests 24 and 25 use exactly twenty times the resolution, 0.4 cm, and raise no warning), but it flags the combination that produced this failure mode for any input exercising many consecutive refinement events over a long integration window.

This investigation itself targeted the serial GFortran path only, and fitting $\ln(\pi a_i^2)$ and the three despiking/local-source/anchor-envelope steps already applied on the OpenACC path from the start, since they wrap the fit regardless of which backend performs it. The `evlim` floor, the radius floor, and the Fritsch–Carlson monotonicity limiter have since been ported to their OpenACC counterparts as well: the two water-filling floors are applied by bringing just the small accepted-event segment to the host, calling the identical, already-validated host subroutine, and returning the corrected values to the device – an exact reuse, not a reimplementation, since this correction is rare, small, and inherently sequential, the same rationale already applied elsewhere in this port to the normalized target-mesh construction. The monotonicity limiter was ported natively into the device Akima kernel’s own tangent computation, using the same single-thread device region that kernel already uses for its endpoint-slope extrapolation. Validated on a native NVIDIA A30: Tests 21–23 pass on all five available comparison backends, including a device-versus-host Akima coefficient/value comparison at the same roundoff level as the pre-existing documented baseline, and a GPU run of the evaporative development probe described above ran past 2.5 million steps (12 accepted refinement events) with no error and no recurrence of the `evlim/cp` blowup.

2.5 Random noise

In JETSPIN, it is possible to add a random noise to Eq. 2.19 (directive `noise yes`, only with `system 4` and `integrator 4`, warnings 94 and 95) in order to model the instability at the nozzle triggering the bending motion. This procedure can be used as an alternative to (or in junction with) the strategy presented in Section 2.2.

In particular, collected in $\vec{f}_{old,i} = \vec{f}_{el,i} + \vec{f}_{c,i} + \vec{f}_{ve,i} + \vec{f}_{st,i} + \vec{f}_{g,i} + \vec{f}_{diss,i} + \vec{f}_{rand,i}$ all the force terms, we modify Eq. 2.19 as

$$m_i \frac{d\vec{v}_i}{dt} = \vec{f}_{old,i} - m_i \gamma_{rand} \vec{v}_i + \sqrt{2m_i^2 D_{rand}} \vec{\eta}_i(t), \quad (2.26)$$

following a standard Langevin approach, where D_{rand} is a diffusion coefficient in the velocity space (in cgs units, cm^2/s^3) and as γ_{rand} a dissipative term (s^{-1}). We highlight that D_{rand} sets the width of the fluctuations. In particular, it is possible to demonstrate [37] that the variance $\sigma_{rand}(t)^2$ of each Cartesian component $v_{i,\alpha}$ ($\alpha = x, y, z$) of the velocity $\vec{v}_i$ (cm^2/s^2) due to only the random and dissipative terms at time t , computed over an ensemble of stochastic trajectories, is equal to $\sigma_{rand}(t)^2 \equiv \langle [v_{i,\alpha} - \langle v_{i,\alpha} \rangle]^2 \rangle = (D_{rand}/\gamma_{rand}) (1 - e^{-2\gamma_{rand}t})$, and, consequently, $\lim_{t \rightarrow \infty} \sigma_{rand}(t)^2 = D_{rand}/\gamma_{rand}$, which is sometimes called fluctuation-dissipation Langevin relation. In `input` file, the user should specified both D_{rand} (`noise diffusivity`) and the variance per Cartesian component σ_{rand}^2 (`noise variance`). The γ_{rand} is automatically set to be equal to $\gamma_{rand} = D_{rand}/\sigma_{rand}^2$ by exploiting the Langevin relation.

2.5.1 Pre-generated Gaussian pool

When the stochastic Platen integrator (`integrator 4`) runs a three-dimensional jet (`system 4`) with a model that the OpenACC build can keep on the device (fixed bead sets and jets with insertion and removal, with or without dynamic refinement and evaporation), its Gaussian numbers are not drawn during the run, in every build and for any number of MPI ranks. Rank zero generates a pool of values once, before the time loop, and broadcasts it, and the OpenACC build copies it to the device once; CPU and GPU, serial and parallel runs read the same values. Every time step reads the next $6n_{\text{active}}$ values, two draws for each of

the three components of every active bead, indexed by the global bead number. The pool is not used with multiple-step Coulomb sums, a time-dependent field (**external potential type** other than 0), **dragvel yes**, bead tracking (**print binary f style 4**), a fixed bead set with **removing yes** and no insertion, or a developer option (see **device_step_supported** in **source/device_step_mod.f90**). In these Platen runs rank zero draws the Gaussian numbers step by step and broadcasts them, and **noise pool** has no effect.

For a fixed topology the pool holds only the values the run needs, up to the requested size:

$$\min \left(N_{\text{steps}}, \left\lfloor \frac{N_{\text{pool}}}{6 n_{\text{active}}} \right\rfloor \right) \cdot 6 n_{\text{active}}$$

values for a run of N_{steps} time steps, so that a run shorter than one period never repeats its noise; a dynamic topology (bead insertion) takes exactly N_{pool} values. The pool is read cyclically, so the noise sequence repeats with a period of

$$N_{\text{period}} = \frac{N_{\text{pool}}}{6 n_{\text{active}}} \quad (2.27)$$

time steps. With the default $N_{\text{pool}} = 10^8$ and 270 active beads the period is about 62,000 steps, so a 10^8 -step run reuses the pool about 1,600 times; the noise is not independent beyond one period. The directive **noise pool** sets N_{pool} between 10^6 and 2×10^9 . A larger pool lengthens the period proportionally at a cost of 8 bytes per value on the host and again on the GPU (4×10^8 values need about 3 GiB on each side). Whether the period is adequate is a choice left to the user. The run log reports the pool once, as **Gaussian history pool: values= N_v covers N steps at n beads**, with N_v the number of values held; for a dynamic topology n is the initial capacity rather than the active count. The pool position is saved in the restart file: a restarted run regenerates the pool from the seed of the input file and continues after the same number of consumed values (Sec. 1.6). A pool of another size, set by **noise pool** or, for a fixed jet, by a longer **final time**, holds the same sequence up to the shorter size, so that the restarted run continues as an uninterrupted run with the new pool would.

2.6 External oscillating electric field

Oscillating orthogonal electric field can bias the electrospinning process, particularly the radius of the electrospun fibers. Further, it can be used as a control mechanism to achieve uniform distributions of polymer filaments at the collector. In JETSPIN it is possible to simulate external oscillating electric field. Denoted the electric field

$$\vec{E} = (E_x, E_y, E_z), \quad (2.28)$$

the directive **external potential type** (see Table 3.1) can be used to select the waveform of the oscillation. The user can select three types of waveform: a rectangular wave oriented along x-axis; a rectangular wave produced by a RC circuit; a rotating electric field of a sinusoidal wave. For the static field (type 0) and the two rectangular waves (types 1 and 2) the amplitude is $E_x = \Phi_0/h$, with Φ_0 given by **external potential**. In the first case the electric field is given by

$$\vec{E} = (E_x \Theta(\sin \omega t), 0, 0), \quad (2.29)$$

where $\omega = 2\pi f$, f is the frequency given by **external potential freq**, and Θ denotes the step function, equal to 1 for a positive argument, 1/2 for a vanishing one and 0 for a negative one; the phase given by **external potential phase** is not applied to the rectangular waves. If the waveform is modeled as produced by an RC circuit, the rectangular value is replaced by the exponential increase $E_x [1 - \exp(-t'/RC)]$ during the half-periods in which the field is switched on, and by the exponential decay $E_x \exp(-t''/RC)$ during those in which it is switched off, where t' and t'' are the times elapsed since the field was switched on and off, respectively. Here, RC denotes a decay time (seconds) given by **external potential time**. The last case describes an orthogonal rotating electric field. Here the electric field takes the form

$$\vec{E} = (E_x, E_y \cos(\omega t + \varphi), E_z \sin(\omega t + \varphi)), \quad (2.30)$$

where $(E_x, E_y, E_z) = (\Phi_x, \Phi_y, \Phi_z)/h$, with the three components Φ_x, Φ_y, Φ_z given by **external potential vector**, and φ is the initial phase given by **external potential phase**. A time-dependent field (type other than 0) is not run by the device step of the OpenACC build and does not use the pre-generated Gaussian pool (Sec 2.5.1).

Details on the main effects of orthogonal rotating electric fields are available in Ref. [38].

2.7 Dimensionless quantities

In JETSPIN all the variables are automatically re-scaled and stored in dimensionless units. In order to adopt a dimensionless form of the equations of motion, we use the dimensionless scaling procedure proposed by Reneker et al.[11]. We define a characteristic length

$$L_0 = \sqrt{\frac{\bar{q}^2}{\pi a_0^2 G}} = l_{step} \sqrt{\frac{\pi a_0^2 \rho_V^2}{G}}, \quad (2.31)$$

where we write the charge q as $\pi a_0^2 l_{step} \rho_V$, denoting ρ_V the electric volume charge density of the filament. Further, we divide the time t and the stress σ by their respective characteristic scales reported in Tab 2.1. Applying the condition that the volume of the jet is conserved $\pi a_i^2 l_i = V_i$ for any i -th bead, we rewrite the EOM as:

$$\frac{d\vec{\mathbf{r}}_i}{dt} = \vec{v}_i \quad (2.32a)$$

$$\frac{d\bar{\sigma}_i}{dt} = - \left[\bar{\sigma}_i - \bar{\sigma}_Y - \bar{k}_{ve} \left(\frac{1}{\bar{l}_i} \frac{d\bar{l}_i}{dt} \right)^{n_{ve}} \right] \quad (2.32b)$$

$$\begin{aligned} \frac{d\vec{v}_i}{dt} = & \Phi_i \cdot \vec{\mathbf{x}} + \sum_{\substack{j=0 \\ j \neq i}}^n \frac{Q_{ij}}{(\bar{R}_{ij} + \bar{a}_{ij})^2} \cdot \vec{\mathbf{u}}_{ij} - F_{ve,i} \bar{V}_i \frac{\bar{\sigma}_i}{\bar{l}_i} \cdot \vec{\mathbf{t}}_i + F_{ve,i} \bar{V}_{i+1} \frac{\bar{\sigma}_{i+1}}{\bar{l}_{i+1}} \cdot \vec{\mathbf{t}}_{i+1} \\ & + A_i \frac{\bar{k}_i}{4} \left(\frac{\sqrt{\bar{V}_i}}{\sqrt{\bar{l}_i}} + \frac{\sqrt{\bar{V}_{i+1}}}{\sqrt{\bar{l}_{i+1}}} \right)^2 \cdot \vec{\mathbf{c}}_i + F_g \cdot \vec{\mathbf{x}} - \Gamma_i \bar{l}_i^{0.905} |\bar{v}_{tan,i}|^{1+0.19} \cdot \vec{\mathbf{t}}_i - \\ & - \Lambda_i \frac{\bar{l}_i \bar{k}_i \bar{v}_{tan,i}^2}{4} \left(\frac{\sqrt{\bar{V}_i}}{\sqrt{\bar{l}_i}} + \frac{\sqrt{\bar{V}_{i+1}}}{\sqrt{\bar{l}_{i+1}}} \right)^2 \cdot \vec{\mathbf{c}}_i - \Gamma_{rand} \vec{v}_i + \sqrt{2(\Theta_{v,i} + \Theta_{rand})} \cdot \vec{\eta}_i(\bar{t}) \end{aligned} \quad (2.32c)$$

where we used the dimensionless derived variables and groups defined in Tab 2.1, the beads and elements are numbered as in Sec 2.1, and $\bar{a}_{ij}$ is the dimensionless cross-sectional radius of the bead of the pair nearer to the nozzle. It is worth stressing that the viscoelastic and surface tension force terms are slightly different from the dimensionless form provided by Reneker et al. [11], since we are considering the DyRen model presented in Sec 2.4. In particular, we do not use the relation $\pi a_i^2 l_i = \pi a_0^2 l_{step}$ since it is valid only for the Ren model (see discussion in Sec 2.4). Note that, whenever the Kelvin-Voigt model is activated, Eq 2.32b is replaced by

$$\frac{d\bar{\sigma}_i}{dt} = \frac{1}{\bar{l}_i} \frac{d\bar{l}_i}{dt} + \frac{1}{\bar{l}_i} \frac{d^2 \bar{l}_i}{dt^2}. \quad (2.33)$$

Finally, the dimensionless EOM of the nozzle are given as

$$\frac{d\bar{y}_i}{dt} = -K_s \cdot \bar{z}_i \quad (2.34a)$$

$$\frac{d\bar{z}_i}{dt} = K_s \cdot \bar{y}_i, \quad (2.34b)$$

with the dimensionless parameter K_s defined in Tab2.1.

Characteristic Scales	
$L_0 = l_{step} \sqrt{\frac{\pi a_0^2 \rho_v^2}{G}}$	$t_0 = \frac{\mu}{G}$
$\sigma_0 = G$	
Dimensionless Quantities	
$\bar{l}_i = \frac{l_i}{L_0}$	$\bar{R}_{ij} = \frac{R_{ij}}{L_0}$
$\bar{\sigma}_i = \frac{\sigma_i}{\sigma_0}$	$\bar{v}_i = v_i \frac{t_0}{L_0}$
$\bar{k}_i = k_i L_0$	$\bar{v}_{tan,i} = v_{tan,i} \frac{t_0}{L_0}$
$\bar{V}_i = \frac{V_i}{L_0^3}$	$\bar{k}_{ve} = k_{ve} t_0^{(1-n_{ve})}$
$\bar{a}_i = \frac{a_i}{L_0}$	
Dimensionless Groups	
$\Phi_i = \frac{q_i \Phi_0 \mu^2}{m_i h L_0 G^2}$	$Q_{ij} = \frac{q_i q_j \mu^2}{m_i L_0^3 G^2}$
$F_{ve,i} = \frac{L_0 \mu^2}{m_i G}$	$A_i = \frac{\alpha \mu^2}{m_i G^2}$
$F_g = \frac{g \mu^2}{L_0 G^2}$	$K_s = \omega \frac{\mu}{G}$
$H = \frac{h}{L_0}$	$L_{step} = \frac{l_{step}}{L_0}$
$\Gamma_i = \gamma_i \cdot t_0 L_0^{0.905} \left(\frac{L_0}{t_0} \right)^{0.19}$	$\Theta_{v,i} = D_{v,i} \cdot \frac{t_0^3}{L_0^2}$
$\Gamma_{rand} = \gamma_{rand} \cdot t_0$	$\Theta_{rand} = D_{rand} \cdot \frac{t_0^3}{L_0^2}$
$\Lambda_i = \frac{\rho_a L_0^3}{m_i}$	

Table 2.1: Definitions of the characteristic scales, dimensionless derived variables, and groups employed in the text.

2.8 Solvent evaporation and solidification

JETSPIN implements the solvent-evaporation and rheological-solidification model introduced by Yarin, Koombhongse and Reneker [36]. The dimensional mass-transfer correlation is

$$Sh = 0.495 Re^{1/3} Sc^{1/2}, \quad Re = \frac{2a_i |\mathbf{v}_i|}{\nu_a}, \quad Sc = \frac{\nu_a}{D_a}, \quad (2.35)$$

where a_i is the local jet radius, ν_a is the kinematic viscosity of air, and D_a is the diffusion coefficient of solvent vapour in air. The speed $|\mathbf{v}_i|$ in Re is that of the bead in the laboratory frame: unlike the aerodynamic drag, it does not subtract the air velocity of `airdrag airvelocity`. For a discrete jet element of length l_i and volume $V_i = \pi a_i^2 l_i$, the corresponding evaporation equation is

$$\frac{dV_i}{dt} = -0.495 D_a Re_i^{1/3} Sc^{1/2} [c_{s,eq}(T) - c_{s,\infty}] \pi l_i. \quad (2.36)$$

This is the discrete form of Eq. (34) of Yarin et al. [36]; in the discrete model l_i is the distance between adjoining beads.

The notation used for humidity in Ref. [36] can be ambiguous because $c_{s,\infty}$ is described there in terms of relative humidity, whereas Eq. (33) gives the saturation-vapour ratio. JETSPIN makes the convention

explicit. The input quantity RH is a relative humidity in the interval $[0, 1]$, and the far-field vapour ratio used in Eq. 2.36 is

$$c_{s,\infty} = RH c_{s,\text{eq}}(T). \quad (2.37)$$

If $c_{s,\text{eq}}(T)$ is not supplied explicitly, JETSPIN evaluates it from the temperature using the water-vapour correlation of Seaver, Galloway and Manuccia used by Yarin et al., as the ratio $c_{s,\text{eq}}(T) = p_{\text{sat}}(T)/1013.25 \text{ mbar}$ of the saturation pressure of water vapour to one atmosphere, where

$$\frac{p_{\text{sat}}(T)}{\text{mbar}} = \sum_{k=0}^6 a_k \vartheta^k, \quad \vartheta = \frac{T}{\text{K}} - 273.15, \quad (2.38)$$

with $a_0 = 6.107799961$, $a_1 = 4.436518521 \times 10^{-1}$, $a_2 = 1.428945805 \times 10^{-2}$, $a_3 = 2.650648731 \times 10^{-4}$, $a_4 = 3.031240396 \times 10^{-6}$, $a_5 = 2.034080948 \times 10^{-8}$ and $a_6 = 6.136820929 \times 10^{-11}$. If D_a is not supplied, the same atmospheric-pressure parametrization is used,

$$D_a = 0.211 \left(\frac{T}{273.15 \text{ K}} \right)^{1.94} \text{ cm}^2/\text{s}, \quad (2.39)$$

corresponding to air at one atmosphere. Both defaults are values for water; for another solvent $c_{s,\text{eq}}(T)$ and D_a should be specified explicitly in the input file. The computed values are printed as warnings 102 and 105, respectively.

2.8.1 JETSPIN nondimensionalization

The evaporation equation is integrated together with the equations of motion in the standard JETSPIN dimensionless variables. With

$$t_0 = \frac{\mu_0}{G_0}, \quad L_0 = \sqrt{\frac{q_0^2}{\pi a_0^2 G_0}}, \quad (2.40)$$

JETSPIN uses

$$\bar{t} = \frac{t}{t_0}, \quad \bar{l}_i = \frac{l_i}{L_0}, \quad \bar{\mathbf{v}}_i = \frac{t_0}{L_0} \mathbf{v}_i, \quad \bar{V}_i = \frac{V_i}{L_0^3}, \quad (2.41)$$

and

$$\bar{\nu}_a = \nu_a \frac{t_0}{L_0^2}, \quad \bar{D}_a = D_a \frac{t_0}{L_0^2}. \quad (2.42)$$

Both Re and Sc are invariant under this rescaling. Consequently, Eq. 2.36 becomes

$$\frac{d\bar{V}_i}{d\bar{t}} = -0.495 \bar{D}_a Re_i^{1/3} Sc^{1/2} [c_{s,\text{eq}}(T) - c_{s,\infty}] \pi \bar{l}_i, \quad (2.43)$$

which is the expression evaluated in `eom_ev_mod.f90`. The arrays `jetvl` and `jetve` contain, respectively, the reference volume $\bar{V}_i^0$ of the material element before solvent loss and its instantaneous volume $\bar{V}_i$ after evaporation.

Yarin et al. also introduce the evaporation-specific Deborah number

$$De = \frac{\mu_0/G_0}{a_0^2/D_a} = \frac{D_a t_0}{a_0^2} = \bar{D}_a \left(\frac{L_0}{a_0} \right)^2, \quad (2.44)$$

and the group

$$\delta = \frac{L_0 a_0}{t_0 \nu_a}. \quad (2.45)$$

JETSPIN does not need to store De and δ as independent input parameters: it stores $\bar{D}_a$ and $\bar{\nu}_a$ and evaluates the local Re and Sc directly. Thus the implementation uses the standard JETSPIN L_0, t_0 scaling while remaining equivalent to the evaporation nondimensionalization of Yarin et al.

2.8.2 Polymer concentration and rheological solidification

Conservation of polymer mass gives

$$c_{p,i} = c_{p,0} \frac{V_i^0}{V_i}. \quad (2.46)$$

The viscosity is updated according to Eqs. (37) and (41) of Yarin et al.:

$$\frac{\mu_i}{\mu_0} = 10^{B(c_{p,i}^m - c_{p,0}^m)}. \quad (2.47)$$

For the Yarin–Koombhongse–Reneker model Ref. [36] the relaxation time obeys

$$\frac{\theta_i}{\theta_0} = \frac{c_{p,i}}{c_{p,0}}, \quad (2.48)$$

and, since $G_i = \mu_i/\theta_i$,

$$\frac{G_i}{G_0} = \frac{10^{B(c_{p,i}^m - c_{p,0}^m)}}{c_{p,i}/c_{p,0}}. \quad (2.49)$$

JETSPIN retains a generalized exponent t_{ev} through $\theta_i/\theta_0 = (c_{p,i}/c_{p,0})^{t_{\text{ev}}}$; setting **evaporation tconstant** 1 gives exactly Eq. 2.48 and is the Yarin-2001 choice. The default value is one.

For Maxwell rheology, with $\bar{\sigma}_i = \sigma_i/G_0$, the resulting constitutive equation is

$$\frac{d\bar{\sigma}_i}{dt} = \frac{1}{\theta_i/\theta_0} \left[\frac{\mu_i}{\mu_0} \frac{1}{\bar{l}_i} \frac{d\bar{l}_i}{dt} - \bar{\sigma}_i \right], \quad (2.50)$$

which is Eq. (39) of Yarin et al. written in the JETSPIN scaling. JETSPIN integrates the general form

$$\frac{d\bar{\sigma}_i}{dt} = \frac{1}{\theta_i/\theta_0} \left[\bar{\sigma}_Y + \bar{k}_{ve} \frac{\mu_i}{\mu_0} \left(\frac{1}{\bar{l}_i} \frac{d\bar{l}_i}{dt} \right)^{n_{ve}} - \bar{\sigma}_i \right], \quad (2.51)$$

with the yield stress $\bar{\sigma}_Y$ (**yield stress**, zero by default) and the Herschel–Bulkley consistency $\bar{k}_{ve}$ and flow index n_{ve} of Sec. 2.7 (both equal to one without **hbfluid yes**); only the consistency term is multiplied by μ_i/μ_0 . Eq. 2.50 is the case $\bar{\sigma}_Y = 0$, $\bar{k}_{ve} = n_{ve} = 1$.

2.8.3 Kelvin–Voigt extension with concentration-dependent properties

The concentration laws in Eqs. 2.47 and 2.49 describe material properties and can also be combined with the Kelvin–Voigt constitutive model available in JETSPIN. This combination is an extension of the Yarin concentration-dependent rheology; it should not be attributed to Ref. [36], which used a Maxwell fluid.

In dimensionless variables define

$$r_\mu = \frac{\mu_i}{\mu_0}, \quad r_G = \frac{G_i}{G_0} = \frac{r_\mu}{(c_{p,i}/c_{p,0})^{t_{\text{ev}}}}. \quad (2.52)$$

The Kelvin–Voigt constitutive identity is written as

$$\bar{\sigma}_i = r_G \varepsilon_i + r_\mu \dot{\varepsilon}_i, \quad (2.53)$$

where the overdot denotes differentiation with respect to dimensionless time. Differentiating Eq. 2.53 gives

$$\dot{\bar{\sigma}}_i = r_G \dot{\varepsilon}_i + r_\mu \ddot{\varepsilon}_i + \dot{r}_G \varepsilon_i + \dot{r}_\mu \dot{\varepsilon}_i. \quad (2.54)$$

Thus a time-dependent concentration requires the two product-rule terms that are absent when G and μ are constant.

No additional material parameters are introduced. From polymer-mass conservation,

$$\dot{c}_{p,i} = -c_{p,i} \frac{\dot{V}_i}{V_i}, \quad (2.55)$$

and therefore

$$\dot{r}_\mu = r_\mu \ln(10) B m c_{p,i}^{m-1} \dot{c}_{p,i}, \quad (2.56)$$

$$\dot{r}_G = r_G \left[\ln(10) B m c_{p,i}^{m-1} - \frac{t_{ev}}{c_{p,i}} \right] \dot{c}_{p,i}. \quad (2.57)$$

JETSPIN reconstructs the instantaneous elastic strain from the current stress state,

$$\varepsilon_i = \frac{\bar{\sigma}_i - r_\mu \dot{\varepsilon}_i}{r_G}, \quad (2.58)$$

so that no extra per-bead state variable is required.

The extension retains the historical JETSPIN Kelvin–Voigt kinematics: $\dot{\varepsilon}_i = (1/l_i) dl_i/d\bar{t}$, while the acceleration term used for $\ddot{\varepsilon}_i$ is the existing $(1/l_i) dv_{\parallel,i}/d\bar{t}$ approximation. Consequently, when $\dot{c}_p = 0$, Eq. 2.54 reduces exactly to the pre-existing JETSPIN Kelvin–Voigt differential equation. At the Yarin solidification cutoff, $\dot{V}_i = \dot{c}_{p,i} = 0$, so r_μ and r_G remain frozen. Euler, Heun (RK2) and classical RK4 integrations are supported for the coupled Kelvin–Voigt/evaporation model; **integrator 4** stops the run at the first step (Sec. 2.9). With Maxwell rheology, evaporation supports all four integrators, including the stochastic Platen scheme (**integrator 4**) with **system 4**.

2.8.4 Compatibility with dynamic refinement

Solvent evaporation can be combined with JETSPIN dynamic refinement for both Maxwell and Kelvin–Voigt rheology. The refinement procedure treats the reference material volume **jetv1** and the instantaneous post-evaporation volume **jetve** independently. Before an Akima remeshing their totals over the refined portion of the jet are stored; after interpolation the two volume fields are reconstructed and each is renormalized to its own pre-refinement total. Thus refinement preserves both the reference material represented by the beads and the current volume after solvent loss. After the renormalization the reference volume is raised where needed to a minimum bead radius of 1 nm, and the instantaneous volume to the floor $V_i \geq \text{evlim } V_i^0$ of Eq. 2.62, by a redistribution that preserves the two totals (Sec. 2.4.1); if a floor cannot be met with the total of the refined portion, the run stops with error 21 (radius) or error 20 (evlim).

After remeshing the local polymer concentration continues to be evaluated from Eq. 2.46, using the newly remapped local ratio V_i^0/V_i . Because the two fields are interpolated separately, the local concentration profile can be slightly smoothed by remeshing even though the corresponding total volumes are conserved. Backup/restore and parallel broadcast operations likewise include the instantaneous volume whenever evaporation is active.

Dynamic refinement may increase the maximum bead capacity during a run. The Kelvin–Voigt/evaporation integrator therefore resizes its Runge–Kutta workspace whenever the refinement changes the required bead or chunk capacity. This keeps the three-way combination of evaporation, Kelvin–Voigt rheology and dynamic refinement valid after a remeshing that grows the discretization.

Evaporation also reduces the mass represented by a material element. JETSPIN therefore uses

$$\frac{m_i}{m_i^0} = \frac{V_i}{V_i^0}, \quad (2.59)$$

in the acceleration terms. In particular, the external-electric, Coulomb, viscoelastic, surface-tension and, for Maxwell rheology, aerodynamic accelerations are evaluated with the instantaneous mass; the Kelvin–Voigt equations, with or without evaporation, have no air-drag term. The Coulomb accelerations use the instantaneous mass also in every summation of the multiple timestep algorithm (Sec. 2.10). This is the discrete counterpart of the modified momentum balance in Eqs. (35a)–(35d) of Yarin et al.

2.8.5 Evaporation cutoff

Yarin et al. stop the evaporation calculation when the solvent mass fraction

$$c_{s,i} = 1 - c_{p,i} \quad (2.60)$$

reaches $c_{s,i} = 0.1$. Using Eq. 2.46, the equivalent condition on the volume represented by a bead is

$$\frac{V_i}{V_i^0} = \frac{c_{p,0}}{1 - 0.1} = \frac{c_{p,0}}{0.9}. \quad (2.61)$$

The JETSPIN integrators enforce the corresponding floor

$$\frac{V_i}{V_i^0} \geq \text{evlim} = \min \left(1, \frac{c_{p,0}}{0.9} \right) \quad (2.62)$$

at every intermediate and final integration stage. Once this limit is reached, V_i , $c_{p,i}$, μ_i , θ_i , and G_i remain constant, as in the solidification cutoff adopted in Ref. [36]. For $c_{p,0} \geq 0.9$ the floor is one and no solvent evaporates. The initial polymer fraction must satisfy $0 < c_{p,0} < 1$, which the input reading does not check: $c_{p,0} = 0$ makes the ratio $c_{p,i}/c_{p,0}$ of Eq. 2.48 undefined (0/0).

2.9 Integration schemes

In order to integrate the homogeneous differential equations of motion we discretize time as $t_n = n\Delta t$ with $n = 1, \dots, N$, where Δt is the timestep (`timestep`) and $N = \lceil T/\Delta t \rceil$ is the number of steps, T being the `final time`. A ratio $T/\Delta t$ within a relative 10^{-12} of an integer counts as that integer, so that every compiler makes the same number of steps (at least one). The last step is a full Δt : a final time that is not a multiple of Δt is overshoot to $N\Delta t$. JETSPIN offers four integration schemes, selected by the directive `integrator` as described in Sec 3.1: the first-order accurate Euler scheme (`integrator 1`), the second-order accurate Heun scheme, sometimes called second-order accurate Runge-Kutta (`integrator 2`), the classical fourth-order accurate Runge-Kutta scheme [39] (`integrator 3`), and the stochastic scheme by Platen described below (`integrator 4`). The Kelvin–Voigt model (`kvfluid yes`), with or without evaporation, supports only the integrators 1–3. The Maxwell model, with or without evaporation, supports all four, the Platen scheme with `system 4` only: with `system 1` or `3` it stops the run at the first step (message “ERROR - ftype is wrong!”). The range of `integrator` is not checked when the input file is read: a value outside 1–4, or `integrator 4` with the Kelvin–Voigt model, stops the run at the first step (message “ERROR - ktype error”).

The stochastic terms of the three dimensional model, namely the random airdrag force (`airdrag amplitude`) and the velocity noise (`noise yes`), require `system 4` (warnings 35 and 95) and the explicit strong order scheme by Platen [40, 41, 42] (`integrator 4`, warnings 34 and 94), whereof the order of strong convergence was evaluated equal to 1.5; in JETSPIN this order refers to the update of the velocities (see below). With `airdrag yes`, `system 4` and `integrator 4`, the directive `airdrag amplitude` is mandatory (warning 33). The deterministic airdrag force alone can be integrated by any of the schemes; the one dimensional model (`system 1`) and the Kelvin–Voigt model have no airdrag term.

The Platen scheme avoids the use of derivatives by corresponding finite differences in the same way as Runge-Kutta schemes do for ODEs in a deterministic setting, and it is briefly summarized as follows.

Let us consider a Brownian motion vector process $\mathbf{X} = \{\mathbf{X}_t, t\}$ of d -dimensional satisfying the stochastic differential equation

$$d\mathbf{X} = \mathbf{a}(t, X^1, \dots, X^d) dt + \mathbf{b} d\Omega \quad (2.63)$$

where $\mathbf{a}$ and $\mathbf{b}$ are vectors of d -dimensional usually called drift and diffusion vector coefficients, and $\Omega(t)$ denotes a Wiener process. Denoted Y_t^k the approximation for the k -th component of the vector $\mathbf{X}$ at time t , the integrator has the following form:

$$Y_{t+\Delta t}^k = Y_t^k + b^k \Delta\Omega + \frac{1}{2\sqrt{\Delta t}} \left[a^k(\tilde{\mathbf{Y}}_+) - a^k(\tilde{\mathbf{Y}}_-) \right] \Delta\Psi + \frac{1}{4} \left[a^k(\tilde{\mathbf{Y}}_+) + 2a^k + a^k(\tilde{\mathbf{Y}}_-) \right] \Delta t, \quad (2.64)$$

with the vector supporting values

$$\tilde{\mathbf{Y}}_{\pm} = \mathbf{Y}_t + \mathbf{a}\Delta t \pm \mathbf{b}\sqrt{\Delta t}. \quad (2.65)$$

The general scheme contains further terms, with the differences of $\mathbf{b}$ at the supporting values $\tilde{\mathbf{r}}_{\pm}$ and $\tilde{\Phi}_{\pm} = \tilde{\mathbf{r}}_{+} \pm \mathbf{b}(\tilde{\mathbf{r}}_{+})\sqrt{\Delta t}$, which vanish for an additive noise. They are not needed in JETSPIN, whose diffusion coefficient does not depend on the velocities (it depends on the state only through the bead mass, which changes only by evaporation): $\mathbf{b}$ is evaluated at the beginning of the step.

Here, $\Delta\Omega$ and $\Delta\Psi$ indicate normally distributed random variables constructed from two independent $N(0, 1)$ standard Gaussian distributed random variables (U_1, U_2) by means of the following linear transformation:

$$\begin{aligned}\Delta\Omega &= U_1\sqrt{\Delta t} \\ \Delta\Psi &= \frac{1}{2}\Delta t^{3/2}\left(U_1 + \frac{1}{\sqrt{3}}U_2\right).\end{aligned}\tag{2.66}$$

JETSPIN applies Eq 2.64 to the velocities only, the components that carry the noise; in the supporting values $\tilde{\mathbf{r}}_{\pm}$ the positions and the stresses are advanced by an Euler step, without noise. The positions are then advanced by a Heun step,

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{2}(\mathbf{f}_1 + \mathbf{f}_2),\tag{2.67}$$

where $\mathbf{f}_1$ is the derivative at the beginning of the step (the velocity $\mathbf{v}_n$) and $\mathbf{f}_2$ is evaluated at the Euler-predicted positions $\mathbf{x}_n + \Delta t\mathbf{f}_1$ and at the new velocities $\mathbf{v}_{n+1}$. The stresses are advanced by a Heun step as well, with $\mathbf{f}_2$ evaluated at the new positions and velocities and at the Euler-predicted stresses. With solvent evaporation (Sec 2.8) the volumes of the beads follow the same Heun step as the positions.

Note that JETSPIN integrates the equations in the dimensionless units of Sec 2.7: the timestep Δt is automatically divided by the time scale $t_0 = \mu/G$.

2.10 Multiple Timestep Algorithm for Coulomb forces

For simulations involving a large number of beads in the calculation of the long-range Coulomb interactions JETSPIN offers the possibility of using a multiple timestep algorithm which improves the efficiency. The method is based on that described by Streett *et al* in Ref [43] and modified for the special context of electrospinning process.

The algorithm is activated by **multiple step yes**. A primary cutoff r_{pcut} for the Coulomb pair interactions is given by **primary cutoff** (mandatory, warning 74). The cutoff r_{pcut} is used to define a standard Verlet neighbor list [44], without a skin: the pairs of beads closer than r_{pcut} when the list is updated stay in the list until the next update, and the other pairs stay outside it even if they come closer. Long-range Coulomb forces derived from beads in the neighbor list are usually larger than those due to remaining (far) beads not contained in the list. Therefore, it is possible to assess the Coulomb forces if the forces derived from the beads in the neighbor list are calculated at every force evaluation, while those from the remaining beads (outer forces) are calculated much less frequently, and are merely extrapolated over the interval. JETSPIN handles this procedure as follows. The Verlet neighbor list is updated at regular intervals of n_{mult} timesteps, set by **multiple step every** n_{mult} (mandatory, warning 76; n_{mult} must be greater than 2, warning 75). The interval between updates should be usually of the order of ~ 50 timesteps. Whenever the Verlet neighbor list is updated, the Coulomb forces are computed by the complete direct summation, and the outer forces are stored. The complete summation is used again at the following force evaluations, up to the first one at a different time, which gives the outer forces at a second time. From then on, the Coulomb force acting on a bead is assessed as sum of the forces derived from the beads in the neighbor list, calculated explicitly, and of the outer forces, calculated by linear extrapolation of the two complete summations: their derivative is the difference of the two outer forces divided by the time between them, and it is centred at the midpoint of the two times. The integrators of Sec 2.9 pass to every stage of a timestep the time at the beginning of the step: the complete summation is then used at all the stages of the timestep of the update and at the first stage of the next one, and the extrapolated outer forces are constant within a timestep. Only the Kelvin–Voigt integrators with evaporation pass the time of each stage, so that their second complete summation is the first stage at a later time. The scheduled update, at every timestep multiple of n_{mult} , waits until the derivative of the previous update has been taken. Every direct summation used by the algorithm is complete, in both

the one and three dimensional models: the primary cutoff only selects the pairs of the neighbor list. With solvent evaporation all the summations use the instantaneous masses of the beads (Eq 2.59). It is readily apparent how this scheme can lead to a significant saving in execution time.

Note that the accuracy of the algorithm is a function of the multiple step interval, and decreases as n_{mult} increases. Further, the neighbor list removes the need to scan over all the beads in the simulation at every timestep. It is worth to highlight that the algorithm is not time reversible.

The neighbor list is also updated, at the next Coulomb force evaluation, after every insertion, removal and remeshing of the jet, after a bead is frozen at the collector (whether it is removed or not), when the arrays of the jet grow, and at the start of a run, also of a restarted one. We remark that it is also possible to define in input file a maximum displacement Δr_{pcut} (**maximum displ**, optional) in order to achieve a better control on the accuracy of the multiple timestep algorithm. The maximum displacement Δr_{pcut} works as follows: whenever at least two beads have traveled for a distance larger than $\Delta r_{pcut}/2$ from their positions at the last update of the list, the Verlet neighbor list is updated without waiting for the scheduled update. The test is made at every Coulomb force evaluation, and every bead is tested, on every rank, so that serial and parallel runs take the same decision. Without **maximum displ** no displacement test is made.

When **erms** is printed (Tab 3.2), once per interval, at the last timestep before a scheduled update, JETSPIN also computes the complete direct summation and compares the extrapolated outer forces with the exact ones; **erms** is the mean deviation per bead, averaged over the intervals checked since the previous print. The check does not change the forces of that evaluation, which are computed as at every other evaluation of the interval, so that printing **erms** does not change the trajectory.

The time covered by the extrapolation, $n_{mult} \Delta t$, must stay short compared with the time in which the beads move by a significant fraction of the cutoff, since the outer forces are extrapolated linearly over the whole interval and the list has no skin; **maximum displ** bounds this motion independently of n_{mult} . In versions up to 1.22 the second direct summation of the one dimensional model was truncated at the cutoff, while the summation at the update of the list was not: the extrapolated far field had the wrong sign and grew over the interval. This defect, not the choice of the parameters, made a one dimensional variant of Test Case 2 with an update every 100 steps unstable. Without the multiple timestep algorithm the primary cutoff truncates only the direct summation of the one dimensional model (Sec 3.1), which changes the physics of the jet.

With the corrections of version 2.0 (NVFORTRAN 24.3 and GFortran builds), the one dimensional variant of Test Case 2 with an update every 100 steps, a cutoff of 1 cm and a maximum displacement of 0.1 cm follows the direct summation within 1.4×10^{-7} in the tip position over 2×10^6 steps, with the same 130 insertions, and two MPI ranks reproduce the serial run exactly; with version 1.22 the same input left the tip at about 1.5 cm while the direct summation had carried it to 26 cm after 700,000 steps, and stopped on the numerical instability check at step 1,344,736. Test Cases 5 and 6 (cutoff 1 cm, update every 100 steps) follow the direct summation within 6×10^{-8} and 4.4×10^{-6} over 10^6 steps, with the same events. The extrapolation of the outer forces is the only approximation: Test Case 8 with evaporation and the algorithm switched on, with a cutoff longer than the jet, gives the rows of the direct summation exactly over 300,000 steps. With a cutoff of 1 cm, about 1,470 beads and a jet whipping towards the collector, the same run keeps 1.1% fewer active beads and makes 146 removals instead of 129 with an update every 100 steps, 0.1% fewer and 131 removals with an update every 3 steps, and 1,990 removals without **maximum displ**: such a jet needs a short interval or a larger cutoff.

In the OpenACC build a run with the multiple timestep algorithm executes the code of the CPU build, since the device step does not support it (Sec 1.5.1); only the complete direct summations of a jet without evaporation can be offloaded to the GPU.

Chapter 3

JETSPIN Data Files

Scope of Chapter

This chapter describes all the input and output files of JETSPIN.

3.1 Input file

In order to run JETSPIN simulations an input file has to be prepared, which is a free-format and case-insensitive. The input file has to be named `input.dat`, and it contains the selection of the model system, integration scheme directives, specification of various parameters for the model, and output directives. The input file does not require a specific order of key directives, and it is read by the input parsing module. Every line is treated as a command sentence (record). Records beginning with the symbol `#` (commented) and blank lines are not processed, and may be added to aid readability. Each record is read in words (directives and additional keywords and numbers), which are recognized as such by separation by one or more space characters.

The parser (subroutine `read_input` in `io_mod.f90`, with the string utilities of `parse_mod.f90`) follows a few simple rules that the user should keep in mind. Each record is read up to 150 characters, stripped of leading blanks and converted to lower case; records beginning with `#` or `!` are skipped. A keyword is recognized when it matches the beginning of any word of the record, and the top-level directives are tested in a fixed order, the first match being accepted. Hence abbreviated forms are equivalent (e.g. `dynam refin every` and `dynamic refinement every`), but an invented keyword may silently match another directive: `timesteps 15000` would be read as `timestep 15000`. A numeric value is the first number found in the record, scanning from the left, and directives expecting several values take the following numbers in order. There are no end-of-line comments: comments should be placed on separate records beginning with `#`. An unknown directive stops the program with an error banner (`ERROR - unknown directive in input file`), as does the end of the file before the `finish` record; records after `finish` are ignored. For most directives an unknown sub-keyword prints warning 61, and the program stops after reading the input file (`ERROR - incomplete input file`); `primary`, `maximum`, `multiple`, `restart`, `dragvel`, `evaporation`, and `dynam` without `refin` ignore it silently instead, and `external potential word f`, `print xyz word f` and `print binary word f` are read as `external potential f`, `print xyz f` and `print binary f` (e.g. `print xyz rescale 1000` writes `traj.xyz` every 1000 s). An unknown key in `print list` or `printstat list` gives warning 48 or 58, followed by warning 60 with the list of the valid keys, and the program stops after reading the input file.

As in the example input files *examples/input-N/input.dat* described in the chapter of the example simulations, the last record is a `finish` directive, which marks the end of the input data. Before the `finish` directive, a wide list of directives may be inserted (see Tab 3.1). The directive `system` should be used to set the nanofiber model. In JETSPIN two models are available: 1) the one dimensional model similar to the model of Chapter 2 but assuming the nanofiber to be straight along the $\vec{x}$ axis, and, therefore, neglecting the surface tension force $\vec{f}_{st}$; 2) the three dimensional model described in Chapter 2. Internally these options

are handled by the integer variable `systype`, which assumes the values explained in Tab 3.1. Further details of the 1-D model can be found in Refs [22, 45]. A series of variables are mandatory and have to be defined. For example, `timestep`, `final time`, `initial length`, etc. (see underlined directives in Tab 3.1). A missed definition of any mandatory variable will call an error banner on the terminal. The user can use two possible ways to define the initial geometry of a nanofiber both given the mandatory directive `initial length`: 1) the discretization step length of nanofiber is specified by the directive `resolution`, which causes automatically the setting of the jet segments number (the number of segment in which the jet is discretised); 2) the jet segments number is declared by the directive `points`, while the value of the discretization step length is automatically set by the program (as in Example **Test Case 3** in Sec 4.3). Exactly one of the two directives must be given (warning 7 if neither is present, warning 22 if both are), and the resulting step length must not be smaller than the `nozzle cross` radius (warning 62, error 8). A few other values are checked when the input is read: the `elastic modulus` must be at least 10^{-4} (warning 88), the `initial length` must not be smaller than the `nozzle cross` radius (warnings 42–44), and the `perturb ampl` must not exceed the `initial length` (warnings 45–47). The values of `system` and `integrator` are not range-checked at input: an unsupported value, or combination (e.g. `integrator 4` with `kvfluid yes`), stops the run at the first step with `ERROR - ftype is wrong!` or `ERROR - ktype error.`, respectively. The directive `primary cutoff` sets a distance in centimeters. With `multiple step yes` it is the radius of the neighbor list of the multiple timestep algorithm for the Coulomb forces (Sec 2.10), and it is mandatory; every direct Coulomb sum used by the algorithm is then complete. Without that algorithm it only truncates the direct Coulomb sum of the one dimensional model (`system 1`), skipping the pairs farther apart than the cutoff (and the image terms of the developer-only `mirror` option); the three dimensional direct sum ignores it. If it is omitted, no cutoff is applied. (Up to version 1.22 the cutoff was converted from centimeters only with the multiple timestep algorithm, and it truncated the one dimensional direct sum also under that algorithm, which corrupted the extrapolation.) The directive `restart yes` can be used to restart a JETSPIN job (for further information see Section 1.6).

The user should pay special attention in choosing the time integration step given by the directive `timestep`, whose detailed considerations are provided in Ref [20]. The reader is referred to Tab 3.1 for a complete listing of all directives defining the electrospinning setup parameters. Not all these quantities are mandatory: an omitted optional quantity is zero, or the feature is off, unless Tab 3.1 gives another default. Several directives become mandatory once a feature is activated, as stated in the table; if one of them is missing, the program prints a warning and stops after reading the input file (`ERROR - incomplete input file`). **Note all the quantities have to be expressed in centimeter–gram–second unit system** (e.g. charge in statcoulomb, electric potential in statvolt, etc.).

The optional `job time` and `close time` directives ensure a controlled close down procedure when a job runs out of time. The time specified by the `job time` directive indicates the total time allowed for the job, in seconds, or in minutes, hours or days when followed by `m`, `h` or `d`; `job time indef`, like an omitted directive, sets no limit. The unit is best written as a separate word (`job time 12 h`): a `d` attached to the number is read as the exponent mark of a double-precision number, so that `job time 2d` means 2 seconds. (It should be set equal to the time specified to the operating system when the job is submitted.) The `close time` directive represents the time JETSPIN will require to write and close all the data files at the end of processing (default: 0). This means the effective processing time limit is equal to the `job time` minus the `close time`: once it is reached, at the end of a time step, the integration loop stops, the output files are closed and `save.dat` is written as at the final time (no extra statistics record is forced: the last record is that of the last multiple of `print time`). The elapsed time is the wall-clock time since the program started, as a batch system counts it (up to version 1.22 it was the processor time of rank zero, which does not count waits and could fall behind the batch limit).

A few directives are read only by developer builds, in which the logical parameter `ldevelopers` of `nanojet_mod.f90` is set to `.true.` (it is `.false.` in released builds), and they are not listed in Tab 3.1: `lorentz`, `magnetic field`, `variable mass`, `wall`, `mirror`, `breakup`, `ultimate strength`, `print pdb tagbead`, and the `type` option of `dragvel yes`. A released build stops at the first seven with `ERROR - unknown directive in input file`, gives warning 61 at `print pdb tagbead` and stops after reading the input file (`ERROR - incomplete input file`), and reads `dragvel yes type i` as `dragvel yes`.

directive:	meaning:
------------	----------

airdrag yes	active the inclusion of the airdrag force in the model (default: no) (any integrator; the 1d-model and kvfluid yes have no airdrag term; with system 4 and integrator 4 airdrag amplitude becomes mandatory)
airdrag airdensity f	mass density of the air surrounding the nanofiber (mandatory with airdrag yes , warning 38)
airdrag airviscosity f	kinematic viscosity of the air surrounding the nanofiber (mandatory with airdrag yes , warning 39)
airdrag airvelocity f	velocity of the air along the x axis (the jet axis), subtracted from the bead velocity in the airdrag term (default: 0, warning 49)
airdrag amplitude f	amplitude of the randomic force rescaled by γ (only with system 4 and integrator 4, warnings 35 and 34; mandatory there with airdrag yes , warning 33) (note that the dissipation coefficient γ is automatically computed by JETSPIN using the input data by Eq. 2.15)
close time f	Set job closure time to f seconds (default: 0)
collector distance f	distance of collector along the x axis from the nozzle (note the nozzle is assumed in the origin)
$\frac{\text{density mass}}{\text{density charge}} f$	mass density of the nanofiber
dragvel yes	electric volume charge density of the nanofiber give each bead inserted at the nozzle the dragging velocity of Subsec 2.3 (default: no, the bead starts with the nozzle velocity along the x axis)
dynam refin yes	active the dynamic refinement during the simulation (default: no; dynam refin every and dynam refin threshold become mandatory)
dynam refin no	disable the dynamic refinement
dynam refin threshold f	maximum target element length; segments longer than f trigger the refinement (must be $\geq 2\times$ the discretization resolution, values below $5\times$ are raised to $5\times$; recommended $\geq 20\times$, below that a warning is issued – see Sec. 2.4.1)
dynam refin every f	minimum interval in seconds between two accepted refinements; once it has elapsed the threshold is tested at every step until a refinement is accepted
dynam refin start f	the refinement is applied only if the simulation time is greater than or equal to f seconds
dynam refin anchor f	during the refinement procedure all the anchor beads inserted at distance f preserve their positions (they are maintained as knots of both the old and new meshes) (default, if omitted: $5\times$ the discretization resolution; values below $1\times$ resolution are raised to that same $5\times$ floor)
dynam refin capacity i	capacity increment incnpjet in beads: the minimum initial array capacity, the growth at an accepted refinement event and the neighbor-list growth of the multiple timestep algorithm (default: 100; a non-positive value gives warning 107 and is reset to 100)
evaporation yes	activate solvent evaporation and concentration-dependent rheology (default: no; polymer frac , airviscosity , temperature , umidity , bconstant and mconstant become mandatory)
evaporation polymer frac f	initial polymer mass fraction $c_{p,0}$ (mandatory, warning 97)
evaporation airviscosity f	kinematic viscosity of air ν_a in cm^2/s (mandatory, warning 98)
evaporation temperature f	temperature in kelvin used for the saturation-vapour correlation (mandatory, warning 99)
evaporation umidity f	relative humidity RH in the interval $[0, 1]$ (mandatory, warnings 100, 106) (historical spelling retained for input-file compatibility)
evaporation diffusivity f	solvent-vapour diffusion coefficient D_a in cm^2/s

evaporation relumidity f	(if omitted it is evaluated by Eq. 2.39) optional saturation-vapour ratio $c_{s,eq}(T)$
evaporation bconstant f	(if omitted it is evaluated from evaporation temperature) parameter B in the concentration-dependent viscosity law (mandatory, warning 103)
evaporation mconstant f	exponent m in the concentration-dependent viscosity law (mandatory, warning 104)
evaporation tconstant f	exponent in the relaxation-time law; use $f = 1$ for Yarin et al. (2001) (default: 1)
<u>elastic modulus</u> f	elastic modulus of the nanofiber (at least 10^{-4} , warning 88)
external potential f	electric potential between the nozzle and collector (the resulting electric field is oriented along x -axis; not used for the field when external potential vector is given)
external potential freq f	frequency of the waveform describing the electric potential (mandatory when external potential type is not 0, warning 71)
external potential phase f	initial phase in radians of the rotating components of type 3 (ignored by types 1 and 2; default: 0)
external potential type i	set the waveform describing the time evolution of the external electric potential. The <i>integer</i> can be equal to 0 for a static field, 1 for a rectangular waveform, 2 for a rectangular waveform of a RC circuit, 3 for orthogonal rotating electric field (default: 0)
external potential time f	time constant (relaxation time) of the RC circuit (mandatory with type 2, warning 78)
external potential vector $3f$	potentials for the $x y z$ components of the electric field: each component is the value divided by collector distance ; the y and z components rotate at freq (read in V with units real ; the output key v still prints the scalar external potential) (mandatory with type 3, warning 90, and accepted only with it, warning 92)
<u>final time</u> f	set the end time of the simulation: the run makes $final\ time / timestep$ steps, rounded up
<u>finish</u>	close the input file (last data record)
gravity yes	active the inclusion of the gravity force in the model (default: no)
hbfluid yes	allow the set of parameters needed to define the Herschel–Bulkley stress. Note that if it is set no, JETSPIN will use the simple Maxwellian fluid model (default: no)
hbfluid consistency f	flow consistency index rescaled by μ (in μ unit) as defined in Eq 2.5 (mandatory with hbfluid yes , warning 40; without it the value is 1)
hbfluid index f	flow behavior index (mandatory with hbfluid yes , warning 41; without it the value is 1)
<u>initial length</u> f	length of the nanofiber at the initial time (not smaller than nozzle cross , warnings 42–44)
<u>integrator</u> i	set the integration scheme. The <i>integer</i> can be '1' for Euler, '2' for Heun, '3' for Runge-Kutta, and '4' for the stochastic Platen scheme (required, with system 4, by noise yes and airdrag amplitude).
inserting yes	inject new beads as explained in Subsec 2.3
job time f	Set job time to f seconds, or minutes, hours, days if followed by the word m , h , d (e.g. job time 12 h); elapsed wall-clock time; job time indef : no limit (default: no limit)
kvfluid yes	activate the Kelvin–Voigt model which is used instead of Maxwell model (default: no). It is implemented for system 1 or 3 (warning 89) with integrators 1–3 (integrator 4 stops the run at the first step),

	not together with hbfluid yes (warning 87); its equations have no airdrag term, so airdrag yes has no effect
maximum displ f	set the maximum displacement of the neighbor list (see Sec 2.10): the list is also rebuilt when at least two beads moved more than $f/2$ since the last rebuild (only with multiple step yes ; default: off)
multiple step yes	activate the multiple timestep algorithm for Coulomb forces (default: no; multiple step every and primary cutoff become mandatory)
multiple step every i	set the interval of multiple-steps (see Sec 2.10): the neighbor list is rebuilt every i steps, and at the next Coulomb evaluation after an insertion, a removal, a remesh, a bead frozen at the collector or an array growth (mandatory with multiple step yes , warning 76; it must be greater than 2, warning 75)
noise yes	active the inclusion of a random noise in the model (default: no) (system 4 and integrator 4 are required, warnings 95 and 94)
noise variance f	variance of random noise in velocity space in cm^2/s^2 (mandatory with noise yes , warning 93)
noise diffusivity f	diffusivity of random noise in velocity space in cm^2/s^3 (mandatory with noise yes , warning 96)
noise pool f	number of pre-generated Gaussian values read cyclically by the stochastic Platen runs that use the pool (see Sec. 2.5.1), between 10^6 and 2×10^9 (default: 10^8 ; a value outside gives warning 109 and the run stops); the noise repeats after $f/(6 n_{\text{active}})$ timesteps
<u>nozzle cross</u> f	cross section radius of the nanofiber at the nozzle
nozzle stress f	viscoelastic stress of the nanofiber at the nozzle
nozzle velocity f	bulk fluid velocity of the nanofiber at the nozzle
perturb yes	active the periodic perturbation at the nozzle (perturb freq and perturb ampl become mandatory)
perturb freq f	frequency of the perturbation at the nozzle
perturb ampl f	amplitude of the perturbation at the nozzle (not larger than initial length , warnings 45–47)
points i	number of segment in which the jet is discretised (exactly one of points and resolution is mandatory, warnings 7, 22)
primary cutoff f	distance in cm: radius of the neighbor list of the multiple timestep algorithm (see Sec 2.10; mandatory with multiple step yes , warning 74); otherwise it truncates only the 1d-model direct Coulomb sum (default: no cutoff)
print binary f [style i]	developer binary trajectory traj.dat every f seconds, in reduced units, layout style 1–9 (default: 1; warning 63 for a style outside 1–9, warning 77 for a style not allowed with the active options)
print binary removed	developer binary record bead.dat of the beads removed at the collector (written only with removing yes)
print list $s_1 \dots$	print on terminal statistical data as indicated by symbolic strings (see Tab 3.2 for detailed informations)
print time f	print data on terminal and output file every f seconds (default: 1000 timesteps, warning 59)
print pdb frame f	print the jet geometry in a single PDB file frame%06d.pdb , with its PSF file frame%06d.psf , every f seconds
print pdb rescalepdb f	PDB coordinates in centimeters multiplied by f (default: 1)
print xyz f	print the trajectory traj.xyz in XYZ file format every f seconds
print xyz frame f	print the jet geometry in a single XYZ file format every f seconds (default name of files: frame%06d.xyz)

<code>print xyz maxnum i</code>	print <code>traj.xyz</code> with i beads starting from the nearest bead to the collector (default: 100; frame files hold all the beads)
<code>print xyz rescalexyz f</code>	XYZ coordinates (<code>traj.xyz</code> and <code>frame%06d.xyz</code>) in centimeters multiplied by f (default: 1)
<code>printstat binary</code>	developer binary dump <code>statdat.dat</code> of all the observables of Tab 3.2 at every <code>print time</code>
<code>printstat list $s_1 \dots$</code>	print on output file statistical data as indicated by symbolic strings (see Tab 3.2 for detailed informations)
<code>removing yes</code>	remove the beads collected at the collector (a bead that reaches the collector is frozen there in any case)
<code>restart yes</code>	restart JETSPIN from a previous job (see Section 1.6)
<code>restart dump i</code>	print the <code>save.dat</code> file every i steps (default: 100000); <code>save.dat</code> is also written at the end of the run
<code>restart reset</code>	accepted for compatibility: the statistical data accumulated since the last print are not restored in any case
<code>resolution f</code>	discretization step length of nanofiber (exactly one of <code>resolution</code> and <code>points</code> is mandatory, warnings 7, 22)
<code>seed i</code>	seed control to the internal random number generator (default: 1)
<code>surface tension f</code>	surface tension coefficient of the nanofiber
<code><u>system</u> i</code>	set the nanofiber model. The <i>integer</i> can be equal to '1' for select the 1d-model, '3' for the 3d-model, and '4' for the 3d-model with the stochastic airdrag and noise terms (integrator 4)
<code><u>timestep</u> f</code>	set the time step for the integration scheme
<code>units s</code>	<code>cgs</code> (default) or <code>real</code> : with <code>real</code> the charge density is read in C/L and the external potential (scalar and vector) in V, then converted
<code><u>viscosity</u> f</code>	viscosity of the nanofiber
<code>yield stress f</code>	yield stress (default: 0)

Table 3.1: Here, we report the list of directives available in JETSPIN. Note i , f , and s denote an integer number, a floating point number, and a string, respectively. The underlined directives are mandatory; others become mandatory with the feature they belong to, as noted. An omitted optional value is zero, or the feature is off, unless another default is given in parentheses.

3.1.1 Evaporation directives

The evaporation directives listed in Table 3.1 activate and parameterize the model described in Section 2.8:

evaporation yes activate solvent evaporation and the concentration-dependent rheology (default: no; **evaporation no** is also accepted). With **evaporation yes** the directives **polymer frac**, **airviscosity**, **temperature**, **umidity**, **bconstant** and **mconstant** are mandatory: a missing one gives the warning listed below and the run stops.

evaporation polymer frac f initial polymer mass fraction $c_{p,0}$ (mandatory, warning 97). It must satisfy $0 < c_{p,0} < 1$, which is not checked; for $c_{p,0} \geq 0.9$ no solvent evaporates (Eq. 2.62).

evaporation airviscosity f kinematic viscosity of air ν_a in cm^2/s (mandatory, warning 98). It is independent of **airdrag** **airviscosity**, which enters only the aerodynamic drag.

evaporation temperature f temperature in kelvin (mandatory, warning 99, also when **relumidity** and **diffusivity** are both given).

evaporation umidity f relative humidity RH in the interval $[0, 1]$ (mandatory, warning 100; a value outside the interval gives warning 106 and the run stops). The historical spelling **umidity** is retained for input-file compatibility.

evaporation diffusivity f vapour diffusion coefficient D_a in cm^2/s . If omitted, Eq. 2.39 for water vapour is used at one atmosphere, and the value is printed as warning 105.

evaporation relumidity f optional value of the saturation-vapour ratio $c_{s,\text{eq}}(T)$. If omitted, it is evaluated for water from **evaporation temperature** (Eq. 2.38), and the value is printed as warning 102. The historical keyword is retained for input-file compatibility.

evaporation bconstant f parameter B in Eq. 2.47 (mandatory, warning 103).

evaporation mconstant f exponent m in Eq. 2.47 (mandatory, warning 104).

evaporation tconstant f exponent t_{ev} in the generalized relaxation-time law; use $f = 1$ to reproduce Yarin et al. (2001) (default: 1).

3.2 Output files

A series of specific directives causes the writing of output files (see Tab 3.1). In JETSPIN five kinds of output files can be written: 1) the file **statout.dat** containing time-dependent statistical data of simulated process; 2) the file **traj.xyz** reporting the nanofiber trajectory in XYZ format file; 3) the files **frame%06d.xyz** reporting the nanofiber geometries in splitted XYZ format files (a single file for each frame); 4) the files **frame%06d.pdb** reporting the nanofiber geometries in splitted PDB (Protein Data Bank) format files (a single file for each frame). Note that in the last case a topological file **frame%06d.psf** in PSF (Protein Structure File) format will be printed alongside each pdb file. These two files (PDB and PSF) can be opened by VMD-Visual Molecular Dynamics [46]; 5) the binary restart file **save.dat**, written every **restart** dump steps and at the end of the run (Sec 1.6). The developer binaries **traj.dat**, **bead.dat** and **statdat.dat** are written only with the directives **print binary**, **print binary removed** and **printstat binary** (see Tab 3.1). With the environment variable **JETSPIN_TOPOLOGY_SNAPSHOT=1**, the text file **topology-state.dat** records, at every insertion or removal, a line **event step inpjnet npjnet active add** followed by one row per active bead (in reduced units). A restarted run appends to **statout.dat** (after a new three-line header block), **traj.xyz**, **statdat.dat**, **traj.dat** and **bead.dat**, creating them if they are missing.

Various statistical data can be written on the text file **statout.dat**, and the user can select them in input using the directive **printstat list** followed by appropriate symbolic strings, whose the list with corresponding meanings is reported in Tab 3.2. The selected observables will be printed at time interval on the same line following the order specified in input file, preceded by the integration-step number: each record is the step (format **i10**) followed by one column per key (format **g20.10**). A three-line header beginning with **#** names the columns at the top of the file (and again where a restarted run starts appending); on the terminal it is repeated every 100 records. In a new run the first record is written before the first integration step (step 0), then one every **print time**. (The file **statdat.dat** is a different, developer-only binary dump of all observables, written only with the directive **printstat binary**.) In the same way, a list of statistical data can be printed on computer terminal using the directive **print list** followed by the symbolic strings of Tab 3.2.

At normal termination, the terminal also reports the wall-clock duration of the temporal integration loop and its throughput in integration steps per second. The timer starts immediately before the loop and stops immediately after it; input parsing, initial allocation, restart loading, and output-file setup are therefore excluded. Integration, bead insertion and removal, statistics, and scheduled output executed inside the loop are included. In MPI builds, ranks are synchronized at both boundaries and rank zero reports the elapsed duration of the slowest participating rank. This loop-only wall time is the recommended measurement for CPU, MPI, and accelerator performance comparisons. The throughput counts the steps of the current run only (after a restart, the steps made before it are excluded).

Besides the records of **print list**, rank zero prints a few log lines on the terminal (the complete list is in docs/data/output.md): **Gaussian history pool:** **values=V** covers **S** steps at **N** beads before the

loop of a run that reads the pre-generated Gaussian pool (Sec 2.5.1); in OpenACC builds, `OpenACC device step engaged` (*scheme, model*) at `step N` with `M` active beads when the device step engages (`resumed` instead of `engaged` after a restart of an engaged run); `Topology event: step=N add=T/F remove=R active=M` at every step with an insertion or a removal; `Dynamic refinement event: ...`, followed by diagnostic lines, at every accepted refinement; at the end of the run, the totals `Topology additions`, `Topology removals` and `Array reallocations` (which include the steps made before a restart) and the final `Topology active beads`, then `Program closed correctly` and `CPU time = T seconds on N CPU(s)`, where T is the processor time of rank zero since the program started (not the wall-clock time) and N the number of MPI ranks.

For performance diagnosis, setting the environment variable `JETSPIN_PROFILE=1` prints an optional wall-clock breakdown for the integrator, Coulomb calculation, bead topology operations, breakup check, statistics, scheduled output, and restart output. Profiling is disabled by default. The Coulomb value is a nested part of the integrator total and must not be added to it when interpreting percentages. The profiler prints cumulative wall time, percentage of the complete temporal-loop time, and call count on the terminal; it does not write a profiling file. These are explicitly instrumented regions rather than an automatic measurement of every Fortran subroutine. Region timings are inclusive, so the integrator value includes the Coulomb, equation-of-motion evaluation, and Runge–Kutta update regions.

In MPI execution, the detailed region breakdown is local to rank zero and is not reduced to the maximum over all ranks. The synchronized complete-loop wall time should be used for MPI scaling. Since clock reads add a small overhead, performance comparisons should enable profiling in both runs or disable it in both runs.

The file `traj.xyz` is written as a continuous series of XYZ format frames taken at time interval, so that can be read by suitable visualization programs (e.g. VMD-Visual Molecular Dynamics [46], UCSF Chimera [47], etc.) to generate animations. The number of elements contained in the file is kept constant equal to the value specified by the directive `print xyz maxnum`, since few programs (e.g. VMD) do not manage a variable number of elements along the simulations. If the actual number of beads is lower than the given constant `maxnum`, the extra elements are printed in the origin point. On the other hand, the number of beads printed in the files `frame%06d.xyz` and `frame%06d.pdb` is exactly the actual number of beads used to discretized the jet at the given frame. Note that the unit of measurement used in the files `traj.xyz` and `frame%06d.xyz`, and in the files `frame%06d.pdb`, is the centimeter multiplied by f , which was given in input file by the directive “`print xyz rescalexyz f`” and “`print pdb rescalepdb f`” (see Table 3.1), respectively (default: $f = 1$, i.e. centimeters). In versions up to 1.22, `traj.xyz` was written in reduced (internal) length units unless `rescalexyz` was given, and the frame files applied the length unit twice when a rescale factor was given. The frame files are numbered by the step divided by the frame interval, so that the first frame of a new run is `frame000001` (a restarted run continues the numbering). Since the coordinates are written with the Fortran formats `f10.5` (XYZ) and `f8.2` (PDB), a rescale factor may be needed to resolve a thin jet, or to keep large coordinates within the field width.

keys:	meaning:
t	unscaled time
ts	scaled time
x, y, z	unscaled coordinates of the farest bead
xs, ys, zs	scaled coordinates of the farest bead
st	unscaled stress of the farest bead
sts	scaled stress of the farest bead
vx, vy, vz	unscaled velocities of the farest bead
vxs, vys, vzs	scaled velocities of the farest bead
yz	unscaled normal distance from the x axis of the farest bead
yzs	scaled normal distance from the x axis of the farest bead
mass	last inserted mass at the nozzle
q	last inserted charge at the nozzle
cpu	processor time of rank zero spent in the last print interval (the first record: since the program started)
cpur	estimated remaining processor time: <code>cpu</code> times the records left,

	counted as all the records from $t = 0$ to the final time minus those written by the current run (after a restart it overestimates)
cpue	processor time of rank zero elapsed since the first record of the current run (these three keys use the processor time, <code>cpu_time</code> , not the wall-clock time used by <code>job_time</code>)
n	number of beads used to discretise the jet
f	index of the first bead
l	index of the last bead
<u>curn</u>	current at the nozzle
<u>curc</u>	current at the collector
<u>vn</u>	velocity modulus of jet at the nozzle
<u>vc</u>	velocity modulus of jet at the collector
<u>svc</u>	strain velocity at the collector
<u>mfn</u>	mass flux at the nozzle
<u>mfc</u>	mass flux at the collector
<u>rc</u>	radius of jet at the collector
<u>rrr</u>	radius reduction ratio of jet
<u>lp</u>	length path of jet
<u>rlp</u>	length path of jet divided by the collector distance
mxst	maximum stress along the jet during the print interval
mxsx	x position of that maximum stress
nref	number of accepted dynamic-refinement events
angl	bending angle of the farthest bead, $2 \arctan \left(\sqrt{y^2 + z^2} / x \right)$, in degrees
v	external electric potential at the current time (the scalar external potential times the waveform of types 1 and 2; it does not reflect external potential vector)
vol	volume of the farthest bead (post-evaporation volume if evaporation is active)
volr	evaporated volume fraction $1 - V_i/V_i^0$ of the farthest bead (zero without evaporation)
<u>visc</u>	viscosity of the jet collected at the collector
<u>gc</u>	elastic modulus of the jet collected at the collector
<u>evrc</u>	evaporated volume fraction of the jet collected at the collector
<u>emfc</u>	evaporated solvent mass flux at the collector
nms	mean number of timesteps per neighbor-list build of the multiple timestep algorithm during the print interval (see Sec 2.10; zero without it, or when the list was not built in the interval)
erms	error of the extrapolated outer Coulomb accelerations of the multiple timestep algorithm (cm/s^2): once per multiple-step interval (at the last step before a scheduled update) they are compared with those from the direct sum; the modulus of the difference, averaged over the beads, is averaged over the checks of the print interval. Computed only when erms is selected; the check does not change the forces; zero without multiple step

Table 3.2: In JETSPIN a series of instantaneous and statistical data are available to be printed by selecting the appropriate key. Here, the list of symbolic string keys is reported with their corresponding meanings. Note by *scaled* we mean that the observable was rescaled by the characteristic values provided in Tab2.1. The underlined keys correspond to data which are averaged on the time interval given by the directive `print time` in input file. Note by *farest* (or *farthest*) *bead* we mean the leading bead of the jet (index `inpjet`), which is the bead farthest from the nozzle along the jet. A bead that reaches the collector is frozen there at $x = h$ (since version 2.0 also without `removing yes`) and remains the leading bead until it is removed: once the jet has reached the collector, these keys describe the first deposited bead without `removing yes`, and the last bead that reached the collector with it. The collector keys (`curc`, `vc`, `svc`, `mfc`, `rc`, `rrr`, `visc`, `gc`, `evrc`, `emfc`) are accumulated only when beads are removed at the collector, so that they are zero in a print interval without removals, and always zero without `removing yes`; `curn` and `mfn` are accumulated at each insertion and need `inserting yes`. Note all the quantities are expressed in centimeter–gram–second unit system, excepted the dimensionless scaled observables.

Chapter 4

Example Simulations

Scope of Chapter

This chapter describes the standard test cases for JETSPIN, whose input files are in the *examples* sub-directory (*input-N/input.dat*); the *log.txt* files of Examples 1–7 were written by versions up to 1.22 and are kept for reference only. Examples 1–8, shortened to 1,000 steps, form the smoke and regression suites (*tests/smoke/run.sh*, *tests/regression/run.sh*, against versioned baselines, serial and on two MPI ranks); Test Cases 9–25 have their own scripts and records under *tests*, described in their sections.

4.1 Test Case 1: 1-D case

This input file (located in the *input-1* sub-directory) provides the dimensionless parameter values $Q = 12$, $V = 2$ (printed as *Phi*) and $F_{ve} = 12$ in the definition of Reneker et al. ($F_{ve} \simeq 1727.5$ with the definition of Tab. 2.1; the program prints both), which have been already used as reference case in Refs [11, 22]. In the OpenACC build this one-dimensional case runs the CPU build’s code on the host and launches no kernel: one-dimensional systems do not take the device step, and the jet keeps its single bead.

4.2 Test Case 2: 1-D case with bead insertion at the nozzle activated

This input file (located in the *input-2* sub-directory) keeps the material parameters, the jet radius at the nozzle, the collector distance and the external potential of Test Case 1, but starts from an initial segment of 0.1 cm (0.319 cm in Test Case 1), which sets the length scale: the dimensionless parameters are $Q \simeq 122.1$, $V \simeq 6.38$ (printed as *Phi*) and $F_{ve} \simeq 122.1$ in the definition of Reneker et al. (1727.5 with the definition of Tab. 2.1). Here we have activated the injection of new beads by the directive *inserting yes* and their removal at the collector by *removing yes*; the run uses the fourth-order Runge–Kutta integrator (*integrator 3*) with a timestep of 10^{-7} s up to a final time of 2 s. In the OpenACC build this one-dimensional case runs the CPU build’s code, since one-dimensional systems do not take the device step; its Coulomb sum is computed on the GPU, with copies at every call, only while the jet has at least 128 active beads.

4.3 Test Case 3: Three-D simulation of electrospinning process

The electrospinning of polyvinylpyrrolidone (PVP) nanofibers is a prototypical process, which has been largely investigated in literature.[6, 8, 5] By the input located in the *input-3* sub-directory, we simulate the electrospinning process of a PVP solution (molecular weight = 1300 kDa) in a mixture of ethanol and water (17:3 v:v), at a concentration of about 2.5 wt%, with operating conditions typical of PVP electrospinning: an applied voltage of 9 kV and a collector placed at distance 16 cm from the nozzle, whose needle has radius 250 micron (see e.g. Ref. [48]). As rheological properties of such system we consider the zero-shear viscosity

ρ (kg/m ³)	ρ_q (C/L)	a_0 (cm)	v_s (cm/s)	α (mN/m)	μ (Pa·s)	G (Pa)	V_0 (kV)	ω (s ⁻¹)	A (cm)
840	$1.47 \cdot 10^{-2}$	$5 \cdot 10^{-3}$	0.28	21.1	2.0	5000	9.0	10^4	10^{-3}

Table 4.1: Simulation parameters for the simulation of PVP nanofibers. The headings used are as follows: ρ : density, ρ_q : charge density, a_0 : fiber radius at the nozzle, v_s : bulk fluid velocity in the syringe needle, α : surface tension, μ : viscosity, G : elastic modulus, V_0 : applied voltage bias, ω : frequency of perturbation, A : amplitude of perturbation. The bulk fluid velocity v_s corresponds to a constant flow rate of 2 mL/h in a needle of radius 250 micron. The charge density and the elastic modulus are those of the input file ($4.4 \cdot 10^4$ statC/cm³ and $5 \cdot 10^4$ g/(cm·s²)); earlier versions of this manual gave $2.8 \cdot 10^{-7}$ C/L and 50000 Pa in this table.

$\mu_0 = 0.2$ g/(cm·s) [49, 50], the elastic modulus $G = 5 \cdot 10^4$ g/(cm·s²) [51], and the surface tension $\alpha = 21.1$ g/s² [49]. We use for the simulation a viscosity value μ which is two order of magnitude larger than the zero-shear viscosity μ_0 reported, since the strong longitudinal flows we are dealing with can lead to an increase of the extensional viscosity from μ_0 , as already observed in literature.[11, 32] The mass density is equal to 0.84 g/cm³, while the charge density ($4.4 \cdot 10^4$ statC/cm³, that is $1.47 \cdot 10^{-2}$ C/L) and the elastic modulus are model parameters of this example, taken from the input file. The parameters of the simulation are those of the input file `examples/input-3/input.dat`, summarized in Tab 4.1; this section does not compare the simulated jet with experiments.

The OpenACC build runs the CPU build’s code, byte for byte, until the jet arrays reach index 100, and then the common RK device step (Heun, Maxwell, no air drag); with seed 317 it engages at step 1,114,161. Over the first 10^7 steps (0.1 s) the axial position of the leading bead, the nozzle velocity, the bead count and the currents are identical to those of the NVFORTRAN CPU build in every printed row, the off-axis distance differs by at most 6.7×10^{-5} , and every insertion and removal falls at the CPU’s step up to step 2,364,190; both runs make 878 insertions and 767 removals. With about 110 beads one NVIDIA A30 is slower than one CPU core (2179 s against about 1800 s for the 10^7 steps), because the RK device step is still synchronous and not fused.

4.4 Test Case 4: Three-D simulation in presence of a gas counter-flow

The input in the `input-4` sub-directory repeats the PVP parameters of Test Case 3 (Sec. 4.3) and enables the aerodynamic drag (`system 4, airdrag yes`) with a gas velocity of -1000 cm/s, opposed to the main jet direction; its random term requires the stochastic Platen integrator (`integrator 4`). Since version 2.0 the case reads its stochastic forcing from the pre-generated Gaussian pool (Sec. 2.5.1), in the CPU build as in the OpenACC one; before, it drew a Gaussian block at every step. The noise realization therefore differs from earlier records, and the regression baselines of the case were regenerated. Seed ensembles over the first 5 million steps (means over 0.03–0.05 s) show that the physics is unchanged: 8 CPU seeds with the pool and 8 with the former step-by-step noise agree within about one standard error for every observable (off-axis distance 2.959 cm, 114.6 active beads, cone angle 21.0 degrees, path length 177.5 cm).

The OpenACC build runs the CPU code, byte for byte, until the jet has 100 beads (about 1.12 million steps, 0.011 s), and then the common Platen device step; the jet reaches the collector at about 0.015 s and then carries about 115 beads. The device step inserts a bead in the step that grows the arrays, as the CPU build does. In a development build of version 2.0 the first insertion after the engagement came one step late, the run left the CPU trajectory there, and the A30 seed ensemble gave an off-axis distance lower by 4.5×10^{-4} cm than the CPU one; now 24 A30 seeds and 24 CPU seeds agree within about one standard error for the off-axis distance, the bead count, the cone angle and the path length. One NVIDIA A30 runs the 5 million steps in 539 s, against 1454 s on one CPU core.

4.5 Test Case 5: Three-D simulation with dynamic refinement

The input in the *input-5* sub-directory repeats the PVP parameters of Test Case 3 (Sec. 4.3) with the fourth-order Runge–Kutta integrator and activates the dynamic refinement of Sec. 2.4 (**dynamic refinement yes**): segments longer than the threshold of 0.4 cm trigger a remeshing of the jet, at most once every 10^{-3} s. Coulomb forces use the multiple timestep algorithm of Sec. 2.10 (**multiple step every 100**). Test Cases 24 and 25 use the same refinement cadence. The OpenACC build runs this case with the CPU build’s code, since its device step does not cover multiple-step Coulomb sums and refinement with the Runge–Kutta schemes yet.

Over 10^6 steps (0.01 s) the multiple timestep algorithm agrees with the direct summation (**multiple step no**) within 6×10^{-8} in every printed row, with the same 88 insertions, and two MPI ranks agree with the serial run within 2×10^{-8} (NVFORTRAN 24.3 and GFortran builds; see Sec. 2.10 for the corrections of version 2.0).

4.6 Test Case 6: Three-D simulation of a quasi-Newtonian fluid

The input in the *input-6* sub-directory activates the Kelvin–Voigt constitutive model (**kvfluid yes**, Eq. 2.7) in place of the Maxwell model, and provides the reference case for the quasi-Newtonian/Kelvin–Voigt integration path. It keeps the geometry of Test Case 3 (Sec. 4.3) with the Heun integrator (**integrator 2**), a viscosity of 10 P, a negligible elastic modulus (10^{-2} dyn/cm²), twice the charge density (88000 statC/cm³), and the multiple timestep algorithm for Coulomb forces (Sec. 2.10). The OpenACC build runs this case with the CPU build’s code, since its device step does not cover multiple-step Coulomb sums yet.

Over 10^6 steps (0.01 s) the multiple timestep algorithm agrees with the direct summation within 9×10^{-8} in the axial position and 4.4×10^{-6} in the off-axis distance, with the same 146 insertions and 35 removals (one of them a step later), and two MPI ranks agree with the serial run within 6×10^{-6} . Versions up to 1.22 departed from the direct summation by up to 25% in the off-axis distance within the same window, with one more insertion, because of the defects corrected in version 2.0 (Sec. 2.10); their per-rank arrays were also one element short, and once the jet filled its arrays a GFortran build stopped near step 668,500 on an out-of-bounds write, which other builds made silently.

4.7 Test Case 7: Three-D simulation with evaporation and an orthogonal rotating electric field

Test Case 7 combines the three-dimensional Maxwell model with an orthogonal rotating electric field and the solvent evaporation and solidification model described in Sec. 2.8. The setting **evaporation tconstant 1** reproduces the relaxation-time law of Eq. (38) of Yarin, Koombhongse and Reneker [36]. The evaporation cutoff is the Yarin criterion $c_s = 0.1$, which in JETSPIN is imposed through the equivalent volume ratio $V_i/V_i^0 = c_{p,0}/0.9$.

The example in *examples/input-7/input.dat* is intended as a regression and reference case for the coupled JETSPIN implementation. Its initial polymer fraction is $c_{p,0} = 0.025$ and therefore it should not be interpreted as a quantitative reproduction of the 6% aqueous-PEO case used in the original Yarin et al. comparison. The orthogonal rotating field is a separate JETSPIN extension and is not part of the 2001 evaporation model.

The OpenACC build runs this case with the CPU build’s code, since its device step does not cover time-dependent external fields; the three-dimensional Coulomb sums go to the GPU, with copies at every call, only while the jet has at least 128 active beads.

4.8 Test Case 8: Yarin et al. (2001) reference parameters

The directory *examples/input-8* contains a dedicated reference case based on the 6 wt% aqueous PEO calculations of Ref. [36]. The values used there are $a_0 = 0.015$ cm, $\rho = 1$ g/cm³, $\alpha = 70$ dyn/cm, $\mu_0 = 10^4$ P, $\theta_0 = 10^{-2}$ s, a charge density corresponding to 1 C/L, $h = 20$ cm, and an applied field of 1.5 kV/m. In

JETSPIN cgs units these correspond to $G_0 = \mu_0/\theta_0 = 10^6$ Ba, a charge density 2.99792458×10^6 statC/cm³, and an applied potential of approximately 1 statV across the 20 cm gap. The ambient conditions are $T = 293.15$ K and $RH = 0.165$, while the solidification parameters are $B = 7$, $m = 0.1$ and $t_{ev} = 1$.

The paper reports $K_s = 100$. Since $K_s = \omega\theta_0$, Test Case 8 uses $\omega = 10^4$ s⁻¹. The small perturbation amplitude used to seed bending is a JETSPIN numerical choice rather than a value specified in the dimensional parameter list of Ref. [36]. The initial bead spacing is similarly chosen for compatibility with the JETSPIN injection algorithm: the test uses $l_{step} = 0.0325$ cm, the electrostatic cutoff length estimated in the same paper. Therefore Test Case 8 is intended as an order-of-magnitude reference and regression test of the evaporation/rheology implementation, rather than as a pointwise reproduction of the original discretized trajectory.

The test intentionally omits the **evaporation diffusivity** directive, so that Eq. 2.39 is exercised. At 293.15 K the expected value is $D_a \simeq 0.242001758$ cm²/s. The solidification cutoff is

$$\frac{V_i}{V_i^0} = \frac{0.06}{0.9} = 0.0666666667, \quad (4.1)$$

for which $c_p = 0.9$. With $B = 7$ and $m = 0.1$, the corresponding reference ratios are

$$\frac{\mu}{\mu_0} \simeq 43.9783350, \quad \frac{\theta}{\theta_0} = 15, \quad \frac{G}{G_0} \simeq 2.9318890. \quad (4.2)$$

These values provide simple numerical targets for future regression checks of the evaporation cutoff and concentration-dependent rheology.

The OpenACC build runs the CPU build's code, byte for byte, until the jet arrays reach index 100, and then the common RK4 device step with evaporation: it engages at step 13,699, with 89 active beads. Over the full 10^6 steps (up to about 7,000 beads) one NVIDIA A30 takes 521 s against 1957 s on one CPU core. Insertions fall at the CPU's steps up to step 26,122 and removals up to step 34,783; then the two runs separate chaotically, with events a few tens of steps apart either way, and both make 7,090 insertions and 130 removals. NVFORTRAN 25.5 and 24.3 round divisions differently, and over the first 20,000 steps their runs separate after the 128th topology event, with the same totals; the reference values are those of NVFORTRAN 24.3.

4.9 Test Case 9: 1,000-bead CPU/GPU performance benchmark

Test Case 9 is a performance-oriented three-dimensional Maxwell simulation derived from Test Case 7. It initializes 1,000 beads uniformly over the full 16 cm distance from the nozzle to the collector and executes 1,000 integration steps. Insertion, removal, multiple-step Coulomb summation, and dynamic refinement are disabled, so the workload remains fixed. The leading bead starts on the collector; since version 2.0 it is frozen there, like any bead that reaches the collector, with or without removal (before, it crossed the collector plane by about 6×10^{-4} cm over the 1,000 steps). The leading bead of Test Cases 10–12, 15 and 20 also starts on the collector and is frozen in the same way. With this rule the A30 rows equal the CPU rows in every printed row; the versioned records were regenerated, and the normalized differences quoted below predate this change. In version 2.0 Test Cases 9–11 take the common OpenACC device step from the first step, with 22, 10 and 14 kernel launches per step for RK4, Euler and RK2 (one of them freezes the beads that reach the collector).

The material, aerodynamic, gravity, and perturbation parameters are retained from Test Case 7. The orthogonal rotating field is replaced by a constant axial electric potential. Evaporation is disabled so that the case isolates the non-evaporative force and integration paths. The case therefore exercises the direct $O(N^2)$ Coulomb kernel without evaporation coupling or topology changes.

Test Case 9 is intended for performance comparison rather than as an additional physical validation baseline. CPU and accelerator runs should be compared using the terminal value **Time-integration loop wall time**, which excludes input processing, initial allocation, and accelerator initialization. Its input is stored in **examples/input-9/input.dat**.

After compiling separate CPU and accelerator executables, copy each executable and **examples/input-9/input.dat** into a separate run directory. Execute the two cases sequentially on an otherwise idle system and compare

their `Time-integration loop wall time` values. The compiler version, GPU model, GPU compute capability, and build options should accompany every reported benchmark.

An initial reference measurement with NVFORTRAN 24.3 and one NVIDIA A30 reported 40.659856 s for the CPU executable and 3.785055 s for the OpenACC executable, corresponding to a speedup of 10.742. Both runs retained exactly 1,000 beads and agreed within the numerical regression tolerances. These timings are indicative and should be remeasured on each target system.

The most recent timings were measured in October 2026 with NVFORTRAN 24.3, the process bound to the NUMA node of its GPU, and the present three-dimensional Coulomb kernel, which spreads the sources of each target bead over the vector lanes of one gang and no longer uploads array descriptors at every call. The Test Case 9 loop took 0.508 s on one A30, against 2.121 s with the previous kernel on the same node and 24.94 s for the NVFORTRAN CPU executable. Test Cases 10, 11 and 12 took 0.122, 0.223 and 0.354 s, against 0.589, 1.078 and 1.659 s with the previous kernel; Test Cases 9–14 run 4.2 to 4.8 times faster than with it. Only the summation order changes, and the outputs still passed their recorded comparisons. The other timings of this section and of Test Cases 10–14 were measured with earlier kernels.

An optional internal subroutine profiler is enabled at run time with `JETSPIN_PROFILE=1 ./main.x`; no rebuild is required. The report lists the integrator total, the Coulomb force, the equation-of-motion evaluation, the Runge–Kutta update, the bead topology operations (add, remove and erase bead, breakup check), statistics, scheduled output, and restart output. The Coulomb, equation-of-motion and Runge–Kutta regions are nested within the integrator total, and their values must not be added to it. The equation-of-motion and Runge–Kutta regions are instrumented only in the CPU code of the Euler, RK2 and RK4 integrators (`integrator_mod.f90`); the OpenACC device step has no such regions, so a run that takes it, as Test Case 9 does from its first step, reports zero for them and, within the integrator, times only the Coulomb region. The equation-of-motion and Runge–Kutta times quoted below were measured with the accelerator paths of the porting milestones, which ran inside those integrators.

The report is written only to the terminal and gives cumulative wall time, percentage of the complete temporal loop, and call count for each explicitly instrumented region. It is not an automatic profile of every Fortran subroutine. The fourth-order Runge–Kutta integration used by this test makes four Coulomb evaluations per step, yielding 4,000 calls. Clock reads add a small overhead, so profiling must have the same enabled or disabled state in both runs being compared. In MPI execution, the detailed table is local to rank zero rather than a maximum over ranks; the synchronized complete-loop timer remains the appropriate MPI scaling measurement.

In an initial profiled run, the NVFORTRAN CPU loop required 39.813621 s, of which 38.232985 s (96.03%) was spent in 4,000 Coulomb evaluations. The A30 loop required 3.320607 s, with 2.164045 s (65.17%) in the same Coulomb region. The corresponding speedups were approximately 11.99 for the complete loop and 17.67 for Coulomb alone. Repeating this profile after each porting milestone provides a direct view of time moving from the Coulomb kernel to remaining host work, device transfers, or other force terms.

The complete sampled CPU and A30 OpenACC numerical outputs are versioned in `tests/performance/test9/` (regenerated on 7 October 2026 for the collector rule; CPU and A30 records identical). The accompanying comparison script checks later CPU and GPU results with relative tolerance 10^{-6} and absolute tolerance 10^{-9} . These records remain outside the automatic regression matrix because of the benchmark’s longer execution time.

The development targets `nvfortran-openacc-force-oracle` and `nvfortran-openacc-coulomb-oracle` also cover this non-evaporative RK4 case. The first evaluates all four trusted host force stages and uploads their derivatives; the second evaluates only the four direct Coulomb sums on the host. Against the versioned NVFORTRAN CPU output, the standard GPU path, complete-force oracle, and Coulomb-only oracle produced worst normalized differences of $7.97 \cdot 10^{-5}$, $3.00 \cdot 10^{-7}$, and $8.52 \cdot 10^{-5}$, respectively. Thus Coulomb accumulation order is not the dominant Test Case 9 RK4 difference. Oracle builds are not performance configurations.

A subsequent profiler refinement measured 2.026551 s (56.28%) in Coulomb, 1.420609 s (39.45%) in the equation-of-motion evaluations, and 0.014712 s (0.41%) in the separate Runge–Kutta vector updates on the A30. Each equation-of-motion and Coulomb region was called 4,000 times. This breakdown selected the three-dimensional equation-of-motion chain as the next meaningful accelerator target; porting the vector updates alone could not materially improve this workload. The trajectory still matched the versioned A30 reference exactly at the saved output precision.

The following paragraphs record the porting milestones of August 2026, which the common device step of version 2.0 has superseded; their normalized differences refer to the records of that time. The first equation-of-motion offload milestone used one explicit OpenACC kernel per RK4 stage for the Test 9 force assembly. The local three-point curvature construction also executed entirely on the device: every bead read only itself and its two neighbours, and no curvature array was transferred from the host. Because the initially straight jet makes the circumcentre arithmetic sensitive to floating-point contraction, the OpenACC target uses the NVFORTRAN `nofma` GPU option. In an updated paired measurement built from the same source with NVFORTRAN 24.3, the CPU and A30 complete-loop times were 38.616119 s and 3.159748 s, respectively, for a speedup of 12.22. The CPU/A30 regional times were 37.059132/2.250507 s for Coulomb (16.47 speedup), 1.398234/0.736914 s for equation-of-motion evaluation including curvature (1.90 speedup), and 0.018556/0.030343 s for the Runge–Kutta updates. Coulomb and equation-of-motion evaluation each had 4,000 calls. The slower device-side RK updates represented less than one percent of the A30 loop. The complete saved trajectory passed the A30 baseline comparison with relative tolerance 10^{-6} and absolute tolerance 10^{-9} ; the worst normalized difference was $7.82 \cdot 10^{-5}$. That first EOM milestone used call-scoped transfers and was restricted to the fixed Test 9 configuration; other configurations retained the CPU path.

The subsequent persistent-data milestone mapped the complete Test 9 state, Coulomb force, equation-of-motion derivatives, and Runge–Kutta scratch arrays once and retained them on the A30 across timesteps. Cross sections and all four Runge–Kutta updates were computed on the device, and the Coulomb result was consumed directly by the equation-of-motion kernel. The CPU statistics of that time still required one update of the seven primary state arrays after each timestep, corresponding to 56,056 bytes per step for the 1,001 array entries (indices 0 to 1,000) of the 1,000-bead jet.

In a representative profiled run, the persistent complete loop took 2.451978 s, compared with 3.159748 s for the call-scoped A30 implementation and 38.616119 s for the paired NVFORTRAN CPU run. The persistent Coulomb, equation-of-motion, and Runge–Kutta regions took 2.006970 s, 0.112892 s, and 0.191860 s, respectively. The Runge–Kutta region included the once-per-step full-state update to the host. The complete-loop speedup over the CPU was 15.75. The trajectory passed the unchanged A30 baseline with relative tolerance 10^{-6} and absolute tolerance 10^{-9} ; its worst normalized difference was $7.8 \cdot 10^{-5}$.

In the following device-resident-statistics milestone, the path length and maximum stress were accumulated on the accelerator after each step. Their scalar counters remained on the device. The complete primary state was copied to the host only at the five scheduled 200-step samples. The final restart recorded the last synchronized timestep and reused the step-1000 host snapshot rather than downloading it again. Thus only five full-state synchronization events remained. A representative A30 run took 2.468580 s for the complete loop, of which 0.056676 s was attributed to the small statistic kernels. The trajectory passed the unchanged A30 baseline with relative tolerance 10^{-6} and absolute tolerance 10^{-9} ; the worst normalized difference was $7.97 \cdot 10^{-5}$.

The next milestone fused the path-length and maximum-stress reductions with the final Runge–Kutta state update. Each device thread reconstructed the updated position of the next bead from the same Runge–Kutta coefficients, avoiding a neighbour race. Two small follow-up kernels preserved the rule that the last bead index wins when maximum stresses are equal. In a representative A30 run, the statistics region decreased from 0.056676 s to 0.036373 s, and the complete loop took 2.457896 s. An extended comparison matched maximum stress and its position exactly; path length differed by at most $2 \cdot 10^{-8}$ cm because of reduction ordering.

In the selective-output milestone, ordinary statistical output began to download only the seven state values of the selected bead and four scalar accumulators, as the device step of version 2.0 still does. The sampling interval is computed from `print time / timestep`; the value of 200 steps belongs to the standard Test 9 input and is not hardcoded. OpenACC transfer tracing reported 280 bytes for the five point samples, 140 bytes for their statistics, and one 56,056-byte complete-state download for the final restart, or 56,476 bytes in total. XYZ, PDB, binary trajectory, and periodic restart output still request a complete snapshot at their configured cadence.

4.10 Test Cases 10 and 11: Euler and RK2 GPU benchmarks

Test Cases 10 and 11 retain the fixed 1,000-bead geometry, physical parameters, constant axial electric field, and 1,000-step duration of Test Case 9. Test Case 10 selects the explicit Euler integrator, while Test Case 11

selects the second-order Heun (RK2) integrator. Their inputs are stored in `examples/input-10/input.dat` and `examples/input-11/input.dat`.

Both integrators take the common OpenACC device step from the first step (10 and 14 kernel launches per step, one of which freezes the beads that reach the collector). Primary state, Coulomb force, equation-of-motion derivatives, and intermediate arrays remain on the accelerator between timesteps. The final state update is fused with the path-length and maximum-stress reductions. Ordinary statistical samples transfer only the selected bead and reduced scalars; complete state is synchronized for output formats that need it and for restart data.

A paired reference measurement used NVFORTRAN 24.3 and one NVIDIA A30. Euler required 9.787760 s on CPU and 0.741555 s with OpenACC, corresponding to a speedup of 13.20. RK2 required 19.464682 s on CPU and 1.254380 s with OpenACC, corresponding to a speedup of 15.52. Both six-sample trajectories passed the CPU/GPU comparison with relative tolerance 10^{-6} and absolute tolerance 10^{-9} . The worst normalized differences were $4.00 \cdot 10^{-7}$ for Euler and $8.99 \cdot 10^{-7}$ for RK2. Since version 2.0 the leading bead is frozen on the collector (Sec. 4.9) and the A30 rows equal the CPU rows; the versioned outputs, regenerated on 7 October 2026, and the comparison helper are stored in `tests/performance/integrators/`. These timings are specific to the measured system and are not portable performance guarantees. With the present Coulomb kernel the A30 loops took 0.122 s (Euler) and 0.223 s (RK2), against 0.589 s and 1.078 s with the previous kernel (Sec. 4.9).

The common development targets `nvfortran-openacc-force-oracle` and `nvfortran-openacc-coulomb-oracle` cover these non-evaporative stages as well. For Euler their worst normalized differences from the CPU record were $7.00 \cdot 10^{-7}$ and $5.00 \cdot 10^{-7}$; for RK2 they were $6.00 \cdot 10^{-10}$ and $1.00 \cdot 10^{-7}$. Their per-stage transfers make them numerical diagnostics, not performance builds.

4.11 Test Case 12: Platen stochastic GPU benchmark

Test Case 12 combines the fixed 1,000-bead geometry and constant axial field of Test Case 9 with the Platen integrator and stochastic air-drag parameters of Test Case 4. Insertion, removal, evaporation, dynamic refinement, and multiple-step Coulomb summation are disabled. The input is stored in `examples/input-12/input.dat`.

Before timing begins, rank zero generates the Gaussian pool of Sec. 2.5.1, filling each step's slice in the draw order of the per-step block (bead, component, draw). For 1,000 steps the pool contains 6,006,000 double precision values, or 48,048,000 bytes. CPU integration reads it directly and the OpenACC build copies it to the accelerator once during initialization. Random values are neither generated nor transferred during the temporal loop. A run that consumes the whole pool wraps to its beginning, which preserves CPU/GPU reproducibility but repeats the noise periodically; Test Case 12 remains below the limit and does not wrap.

The persistent accelerator path performs the three Platen force evaluations, the positive and negative predictors, stochastic velocity update, Heun position and stress updates, and statistics on the device. An initial paired measurement with NVFORTRAN 24.3 and one NVIDIA A30 required 29.050473 s on the CPU and 1.874810 s with OpenACC, a speedup of 15.50. All six rows and fourteen columns of the two sampled outputs were identical at the written precision. The complete-force and Coulomb-only development oracles cover this same Platen sequence. Both reproduce the CPU record exactly in all written rows and columns; their intentional per-stage transfers exclude them from timing measurements. In version 2.0 the step is the common Platen device step with the fused two-kernel tail and a kernel that freezes the beads reaching the collector (Sec. 4.9): 13 kernel launches per step, against 18 for the earlier fixed-set path, which did not freeze them, and the CPU and A30 rows are identical. The versioned results, regenerated on 7 October 2026, and the comparison helper are stored in `tests/performance/integrators/`. With the present Coulomb kernel the A30 loop took 0.354 s, against 1.659 s with the previous kernel (Sec. 4.9).

4.12 Test Case 13: dynamic-topology GPU baseline

Test Case 13 initializes 1,024 beads uniformly between the nozzle and collector and enables both `inserting yes` and `removing yes`. It retains the material parameters and constant axial field of Test Case 9. An intentionally artificial nozzle velocity of 2,000 cm/s forces multiple topology events during a short 1,000-step run. Its input is stored in `examples/input-13/input.dat`.

The CPU build and the OpenACC device step make 13 insertions and 13 removals at the same steps, finish with 1,024 active beads, and remain between 1,023 and 1,025 active beads; the original call-scoped and persistent paths that preceded the device step made the same 13 insertions and 13 removals. A structured **Topology event:** line is printed whenever insertion or removal occurs.

The current dynamic milestone reserves capacity for 1,280 beads and keeps the RK4 state, Coulomb data, derivatives, and scratch arrays persistently mapped while the active lower and upper bead indices change. Collector detection and clamping run on the device. Nozzle distance checks, blocked-bead release, new-record initialization, the **npjet** update, and the placement and charge smoothing of the blocked bead before every force evaluation also execute on the device. One serial kernel decides release, insertion, capacity exhaustion, and removal, and the host receives the outcome as one 20-byte record per step, plus the two new tail records only when insertion occurs. Removed records are synchronized individually for output and cleared on both host and device. This run stays within its reserve; capacity growth, which reallocates the arrays on the host and rebuilds the device mapping, is exercised by Test Case 15. In version 2.0 this is the common device step of all RK runs. An initial call-scoped NVFORTRAN 24.3 and NVIDIA A30 measurement required 40.659696 s on CPU and 3.834931 s with OpenACC. The bounded persistent path with host insertion required 2.829376 s, and device-side insertion reduced this to 2.623558 s, or 381.162 steps/s, before the nozzle-insertion fix described below. With the present Coulomb kernel Test Cases 13 and 14, like Test Cases 9–12, run 4.2 to 4.8 times faster than with the previous one (Sec. 4.9).

The A30 run reproduces the 26 CPU topology events step for step, and its statistics agree with the CPU within the paired tolerance 3×10^{-4} . Before the nozzle-insertion fix, made during the development of version 2.0, the insertions of the persistent path fell progressively earlier (step 593 instead of 600 for the seventh): the persistent path placed the blocked nozzle bead only when it was created and smoothed its charge on the stale host copy, so the device Coulomb sum saw a full charge at the nozzle. That drift had been attributed to GPU RK4 rounding. With the environment variable **JETSPIN_TOPOLOGY_SNAPSHOT=1**, JETSPIN writes the complete active state at every topology event to **topology-state.dat** for such comparisons, and **JETSPIN_OPENACC_DISABLE_PERSISTENT=1** keeps the device step closed, so that the OpenACC build runs the CPU code (with the Coulomb sums offloaded from 128 active beads).

The directory **tests/performance/dynamic/** holds the reference event stream **topology-events.txt** (the historical streams of the persistent paths are in the other **topology-events*.txt** files) and the comparison tools. The command **tests/performance/dynamic/compare.sh CPU/run.log GPU/run.log CPU/statout.dat GPU/statout.dat** checks the **Topology event:** lines of both logs against the reference stream and compares the two statistics files with relative tolerance 3×10^{-4} and absolute tolerance 10^{-6} (the environment variables **RTOL** and **ATOL** change them); **compare_state.py** compares, event by event, two **topology-state.dat** files written with **JETSPIN_TOPOLOGY_SNAPSHOT=1**.

4.13 Test Case 14: larger bounded dynamic topology

Test Case 14 starts with 1,500 beads and reserves 1,756 slots before the persistent OpenACC mapping. Insertion, removal, RK4, Coulomb, and EOM remain device-resident without reallocation during the run. The case stresses the bounded-capacity path and is the no-overflow control for the capacity growth of Test Case 15. On an NVIDIA A30 the run reproduces the CPU topology stream step for step (18 additions, 19 removals, 1,499 final active beads); before the nozzle-insertion fix made during the development of version 2.0, the A30 made 19 additions.

The input is stored in **examples/input-14/input.dat**.

4.14 Test Case 15: forced array reallocation

Test Case 15 is a dynamic allocation check. It starts with 100 beads and no spare initial capacity. The nozzle velocity is set to 200,000 cm/s to produce repeated insertions in the short run. The first insertion exceeds **mxnpjet** and calls **reallocate_jet**. The program reports **Array reallocations:** at shutdown; the acceptance condition is a value greater than zero and at least one topology addition. Each subsequent reallocation grows the capacity by 100 slots; the initial allocation remains governed by the normal 100-slot increment. In the OpenACC build the common device step runs from the first step (100 beads), so that

device capacity growth and remapping are exercised. Acceptance checks completion, topology additions, and a positive reallocation count rather than bead-for-bead trajectory identity. The input is stored in `examples/input-15/input.dat`.

The relaxed acceptance criterion was originally attributed to the high insertion speed amplifying GPU rounding. That explanation was incorrect. The divergence was caused by a defect in the accelerator path: on a reallocation the persistent device mapping of the Coulomb force array was not released before the host array changed, so the device either failed with a *partially present* error or silently reused the previous buffer. The GPU run produced NaN from its first printed line with the bead count frozen at 103, against 211 beads and a final abscissa of 35.99 cm on the CPU. Once the mapping was released correctly, the two runs had the same bead count and final abscissas equal to about eight significant digits (35.99200794 and 35.99201023 cm); this comparison predates the collector rule, and since the rule all statistics rows are identical (below). This test is consequently a genuine regression guard for that defect, and a future tightening of its acceptance criterion is justified. On an NVIDIA A30 both builds make 111 additions and two reallocations, every insertion at the same step, and all statistics rows are identical (within 4.3×10^{-5} before the collector rule of version 2.0: the test does not remove beads, and its leading bead then crossed the collector plane, keeping its charge and its Coulomb interactions, and reached 35.99 cm; it is now frozen on the collector at 16 cm). During the development of version 2.0 two insertions fell one step later on the GPU, because the device path first grew the capacity and inserted at the next step, whereas the CPU reallocates and inserts in the same step.

4.15 Test Case 16: dynamic Maxwell evaporation on the GPU

Test Case 16 exercises Maxwell rheology, Yarin evaporation, nozzle insertion, collector removal, and capacity growth in one serial three-dimensional run. It starts with 100 beads and deliberately retains the small initial capacity so that the 1,000-step run invokes the reallocation path.

The stored input selects RK4; changing only `integrator` to 1 or 2 selects the Euler or RK2 validation. With target `nvfortran-openacc`, Euler executes one Maxwell force and stress evaluation, RK2 executes two, and RK4 executes four. Their intermediate and final updates, direct Coulomb summation, evaporation, charge smoothing and restoration, nozzle geometry, and final statistics reductions execute on the GPU. No state, force, stress, or derivative array is transferred between stages. Ordinary timesteps return only one 20-byte topology record. Bead records are moved for actual topology or output events, while the complete active state is synchronized for capacity reallocation and the final checkpoint. The run takes the common OpenACC device step (`source/device_step_mod.f90`) from the first step: the device step engages once the jet holds 100 beads and stays engaged, and this case starts with 100. The step is still synchronous and its kernels are not yet fused.

On an NVIDIA A30 the accepted 1,000-step topology for Euler, RK2, and RK4 contains 111 additions, 122 removals, two reallocations, and 89 final active beads, equal to the CPU totals. For Euler, RK2, and RK4, three-step pre-event `statout.dat` comparisons have zero difference with relative tolerance 10^{-12} and absolute tolerance 10^{-13} ; their written XYZ geometries are also byte-identical. Since 2026-10-07 the 111 insertions and 122 removals occur at the same 222 steps on the CPU and the GPU, for each integrator; before, the GPU made the first insertion at step 5 instead of 4, because its topology path grew the full arrays and inserted one step later. A pointwise CPU/GPU trajectory is not used as the acceptance criterion: the target-centric direct Coulomb kernel has a different summation order, and the transverse components, of order 10^{-12} cm, differ by up to their own size.

The rheology-independent development target `nvfortran-openacc-force-oracle` evaluates each complete trusted Maxwell force stage on the host and uploads its derivatives while state updates remain on the GPU. The narrower `nvfortran-openacc-coulomb-oracle` target evaluates only the direct Coulomb sum on the host. Both reproduce the accepted 1,000-step topology for Euler, RK2, and RK4, but their deliberate transfers make them unsuitable for production or performance runs. The input is stored in `examples/input-16/input.dat`.

The local script `tests/performance/dynamic/validate_evaporation.sh` builds independent NVFORTRAN CPU and standard OpenACC executables and repeats the complete paired Euler, RK2, and RK4 validation for Test Cases 16 and 17. The standard GPU build used by the script does not enable either diagnostic oracle macro.

4.16 Test Case 17: dynamic Kelvin–Voigt evaporation on the GPU

Test Case 17 is the Kelvin–Voigt counterpart of Test Case 16. It starts with 100 beads and combines Yarin evaporation, nozzle insertion, collector removal, and forced capacity growth over 1,000 integration steps. The stored input selects RK4; changing only `integrator` to 1 or 2 selects the equivalent Euler or RK2 validation.

With target `nvfortran-openacc`, Euler executes one Kelvin–Voigt force and stress evaluation, RK2 executes two, and RK4 executes four. All intermediate and final updates, direct Coulomb summation, evaporation, charge smoothing and restoration, nozzle geometry, and final statistics reductions execute on the GPU. Kelvin–Voigt stress uses relative bead acceleration together with the full product-rule derivative of the concentration-dependent viscosity and modulus. The jet state, Coulomb force, stress, and derivative arrays remain resident between stages. As in Test Case 16, the run takes the common OpenACC device step from the first step, synchronous and not yet fused.

Ordinary timesteps return only one 20-byte topology record with the insertion, removal, and resize decisions. Actual topology and output events transfer their required records; the complete active state is synchronized only for capacity reallocation and the final checkpoint. A runtime transfer audit confirms two full-state rebinds for the two reallocations and no per-stage array transfers.

The accepted 1,000-step CPU and NVIDIA A30 runs for Euler, RK2, and RK4 all produce 111 additions, 122 removals, two reallocations, and 89 final active beads. Three-step pre-event CPU/GPU comparisons are identical with relative tolerance 10^{-12} and absolute tolerance 10^{-13} . Since 2026-10-07 the 111 insertions and 122 removals occur at the same 222 steps on the CPU and the GPU, for each integrator; before, the GPU made the first insertion at step 5 instead of 4, because its topology path grew the full arrays and inserted one step later. Pointwise trajectories still separate at roundoff level, so equal topology, pre-event agreement, clean completion, and the transfer audit are used instead of pointwise CPU/GPU trajectory identity.

The historical CPU Kelvin–Voigt evaporation equation omits aerodynamic drag and lift even when `airdrag yes` is present. The GPU implementation preserves this established semantics. The rheology-independent `nvfortran-openacc-force-oracle` target evaluates every force stage through the trusted host equations and uploads its derivatives. The narrower `nvfortran-openacc-coulomb-oracle` target moves only the direct Coulomb sum to the host. Both leave Runge–Kutta updates on the device and are useful for numerical isolation, but unsuitable for performance measurements. The input is stored in `examples/input-17/input.dat`.

The paired CPU/GPU checks for all three deterministic integrators are automated by `tests/performance/dynamic/valid`; see Test Case 16 for the validation criteria and build policy.

4.17 Test Case 18: high-resolution dynamic Maxwell evaporation

Test Case 18 is the high-resolution counterpart of Test Case 16. It uses the Maxwell RK4 integrator with Yarin evaporation, nozzle insertion, and collector removal enabled. The initial jet length is 16 cm with `points` 800, giving a nominal spacing of 0.02 cm, that is 200 micrometres, between adjacent nodes. As in Test Case 16, dynamic refinement is disabled and the case is serial.

Because the initial capacity already contains 800 intervals, the case measures the dynamic topology path at a finer spatial resolution rather than forcing frequent capacity reallocations. As in Test Case 16, the OpenACC build takes the common device step from the first step.

The NVFORTRAN CPU reference completed in about 14.2 s with 500 topology additions, 899 removals, three reallocations, and 401 active beads. Since 2026-10-07 an NVFORTRAN 24.3 run on an NVIDIA A30 gives the same totals, every insertion at the CPU's step; before, its topology path inserted one step later whenever the arrays had to grow, and the run gave 499 additions, 898 removals and 401 active beads.

The transverse components, of order 10^{-12} cm, still differ by up to their own size, a sign of the case's sensitivity to the summation order of the direct Coulomb force. Test Case 18 is therefore a performance and porting probe rather than a strict pointwise regression case, and it is not part of the short smoke and regression matrices. The input is stored in `examples/input-18/input.dat`.

4.18 Test Case 19: high-resolution dynamic Kelvin–Voigt evaporation

Test Case 19 is the Kelvin–Voigt counterpart of Test Case 18. It uses the Kelvin–Voigt RK4 integrator with Yarin evaporation, nozzle insertion, and collector removal enabled. The initial jet length is 16 cm with `points` 800, the same 0.02 cm nominal spacing as Test Case 18. Dynamic refinement is disabled and the case is serial. As in Test Case 16, the OpenACC build takes the common device step from the first step.

The NVFORTRAN CPU reference completed in about 14.5 s with 500 topology additions, 76 removals, five reallocations, and 1,224 active beads. Since 2026-10-07 an NVFORTRAN 24.3 run on an NVIDIA A30 gives the same events, step for step, with statistics rows within 10^{-5} relatively; before, its topology path inserted one step later whenever the arrays had to grow, and the run ended with 498 additions, 77 removals and 1,221 active beads. NVFORTRAN 25.5, which rounds divisions differently, gives 500 additions, 61 removals and 1,239 active beads on CPU and GPU alike: at step 62 a bead stops a relative 1.8×10^{-8} short of the collector instead of reaching it, and the removals that follow differ. The reference values are those of NVFORTRAN 24.3.

Because the direct Coulomb force is sensitive to both bead spacing and bending dynamics, CPU and GPU trajectories for this case should be compared within a tolerance rather than as bitwise-identical states. Like Test Case 18 it is a porting and performance probe rather than a strict pointwise regression case, and it is not part of the short smoke and regression matrices. The input is stored in `examples/input-19/input.dat`.

4.19 Test Case 20: Stochastic Platen Maxwell Evaporation

Test Case 20 is a fixed-topology CPU/GPU validation of the Maxwell evaporation equations with the stochastic Platen integrator (`system` 4, `integrator` 4). It distributes 1,000 beads over 16 cm, disables insertion, removal, and dynamic refinement, and executes 100 timesteps. The input is stored in `examples/input-20/input.dat`.

Before loop timing, the CPU generates a pool of 600,600 Gaussian double precision values, each step’s slice in bead, component, and draw order (Sec. 2.5.1). OpenACC transfers this pool to the accelerator once. The three drift evaluations, positive and negative predictors, stochastic velocity update, Heun position, volume and stress updates, and statistical reductions then execute in persistent device data regions. No Gaussian value or integration-stage array crosses the host/device boundary during an ordinary timestep.

With a build of 14 August 2026, NVFORTRAN 24.3 and one NVIDIA A30, the 100-step integration loop required 3.177595 s on the CPU and 0.224058 s with OpenACC, a speedup of 14.18; this figure predates the rewrite of the evaporative Coulomb kernel and the common device step and has not been measured again. The standard OpenACC output, the complete host-force oracle output, and the Coulomb-only oracle output each reproduce all six CPU rows and fourteen columns exactly at the written precision. The oracle targets are correctness diagnostics and are not performance builds. Since 6 October 2026 the step is the common Platen device step with the fused two-kernel tail (12 kernel launches per step instead of 19); since 7 October 2026 a thirteenth kernel freezes the leading bead on the collector (Test Case 9). Under this collector rule of version 2.0 a bead that reaches the collector is always frozen there (Sec. 2.1); the leading bead of Test Case 20 starts on the collector and is held at 16 cm, so the rows differ from those written before the rule, while the CPU and A30 rows remain identical.

4.20 Test Case 21: Anchored Dynamic Refinement with Platen Evaporation

Test Case 21 validates the hybrid OpenACC port of dynamic Akima refinement. It combines Maxwell rheology, the stochastic Platen integrator, Yarin evaporation, aerodynamic drag, nozzle insertion, and numerical anchor beads. The input is stored in `examples/input-21/input.dat`.

The initial jet contains 400 elements at a resolution of 0.02 cm and occupies the interval from the nozzle to 8 cm, while the collector remains at 16 cm. The free half of the domain permits field-driven stretching. Interior beads are tagged as anchors every 0.10 cm. These beads are ordinary material points that remain knots of the old and refined interpolation meshes; no variable mass or nanoparticle model is enabled.

The refinement target is also 0.10 cm. The reduced charge density limits sensitivity to the order of the direct Coulomb sum, whereas the increased external potential makes the short validation reach the refinement threshold. These choices accelerate a numerical validation and are not proposed as a reference experimental parameter set. The minimum interval between accepted remeshes is 5×10^{-5} s, leaving a post-refinement trajectory before the run ends.

The OpenACC path keeps the Platen stages, pre-generated Gaussian pool, insertion, and state on the device. Repeated refinement checks return only three reduction results, in one transfer. At an accepted event the active state is downloaded, the host walks the mass boundaries and constructs normalized source and target knots, and Akima coefficient construction and all 11 spline interpolations execute on the GPU. The GPU then reconstructs the bead and evaporated volumes from the interpolated radii, rescales them to conserve the reference volumes, and converts the interpolated mass and charge densities back to per-bead quantities. The host keeps the anchor bookkeeping and the final assembly of the mesh, the conservative radius-floor and evaporation-limit corrections, and the invariant checks, after which the remeshed state is uploaded once. The initial allocation reserves 100 additional entries, so the validated event does not reallocate a mapped array. An additional capacity-growth mode reduces only this developer reserve to 50 entries. It releases the old persistent mappings before the host allocation changes, expands the Platen/Coulomb workspaces, and binds the completed mesh once; the Gaussian pool is not rebuilt. The same device Akima kernels are used in both modes.

With GFortran the event occurs at step 7,136 and changes the active count from 413 to 452, ending with 453 elements. With NVFORTRAN 24.3 the CPU event occurs at step 7,131 and changes 412 elements to 453, ending with 455. The complete host-force oracle and the native path on one NVIDIA A30 both reproduce the NVFORTRAN CPU event and topology; their statistics agree with the CPU within 1.4×10^{-7} and 2.4×10^{-10} relatively. CPU and GPU read the same Gaussian pool (Section 2.5.1); what remains is the different floating-point evaluation order of the device kernels. Before the persistent-path fixes of 30 September 2026 the device kept the full charge on the bead being inserted at the nozzle; the fourth insertion then fell one step earlier, the pool position was offset, and the native event moved to step 7,125.

All paths start with 79 interior anchors, finish with `nref=1`, and preserve anchor positions and fields within the 10^{-12} checker tolerance. Reference and evaporation-volume conservation errors are of order 10^{-16} , with no capacity reallocation. The run pre-generates the 10^8 -value Gaussian pool once and uploads it once. A transfer audit observed, on ordinary steps, one 20-byte topology record and, during threshold checks, the three reduction results returned in one transfer; at the Akima event, one full-state download and upload, one upload of the source and target coordinates, one upload of each of the 11 source fields and one download of each interpolated field; and the independent final-shutdown download.

In the capacity-growth mode capacity grows from 450 to 555 entries on all three paths. Events, topology, and the whole statistics file are byte-identical to the standard mode on both builds: the Gaussian pool does not depend on capacity and is neither rebuilt nor transferred again. A transfer audit again found only the accepted-event and final full-state downloads, plus one topology/evaporation rebind and one workspace recreation at the event.

The script `tests/refinement/run.sh` defaults to GFortran. Its `nvfortran`, `openacc`, and `force-oracle` arguments run the corresponding event-level validation paths. Appending `capacity-growth` enables the forced-capacity validation.

4.21 Test Case 22: Repeated Dynamic Refinement and Capacity Growth

Test Case 22 extends the hybrid refinement path of Test Case 21 from one accepted Akima event to three. It retains Maxwell rheology, Yarin evaporation, stochastic Platen aerodynamic drag, nozzle insertion, and ordinary numerical anchor beads. The initial jet contains 400 elements over 8 cm of a 16 cm nozzle-to-collector domain. The timestep is 2.5×10^{-8} s and the run ends after 15,800 steps.

The dedicated validation runner sets developer-only initial and growth reserves of 20 entries. This forces every accepted remesh beyond the current capacity, whereas production runs retain the normal 100-entry `incnpjet` policy. Each event must release the old topology, evaporation, Platen, and Coulomb mappings before the host allocations change. The new state is then bound once; the Gaussian pool stays mapped

unchanged. As in Test Case 21, the host walks the mass boundaries and constructs the target mesh, while Akima coefficient construction, spline interpolation, the volume reconstruction with its conservation rescale, and the conversion of mass and charge densities execute on the GPU; the host keeps the anchor bookkeeping and final assembly, the conservative radius-floor and evaporation-limit corrections, and the invariant checks.

With NVFORTRAN 24.3, the CPU events occur at steps 14,268, 14,971, and 15,698; NVFORTRAN 25.5 gives the same steps. The active element counts change from 413 to 459, from 459 to 495, and from 496 to 535. Capacity grows successively from 420 to 481, then to 517, and finally to 557. The complete-force oracle and the native NVIDIA A30 path reproduce these CPU event steps, topologies, and capacities.

The three-event contract is intentionally a same-compiler NVFORTRAN CPU/GPU comparison. A different compiler can perturb the stochastic bending trajectory enough to accept a different number of marginal remeshes within the same final time. Test Case 21 remains the portable single-event GFortran validation.

All 81 active anchors retain their positions, velocities, stresses, reference radii, and evaporation radii at every remeshing event. Reference volume, evaporated volume, mass, and charge are conserved to approximately 10^{-16} . In all 80 statistics rows the native A30 path agrees with the CPU within 8.0×10^{-10} relatively and the complete-force oracle within 3.2×10^{-8} ; CPU and GPU read the same Gaussian pool (Section 2.5.1).

An OpenACC transfer audit records four complete state downloads: three accepted refinement events and the final shutdown. It records exactly three topology rebinds and three evaporation-state rebinds; the Gaussian pool is uploaded once at startup and never transferred again. Ordinary timesteps and unsuccessful threshold scans do not transfer the complete bead state.

The commands `tests/refinement/run_test22.sh nvfortran`, `tests/refinement/run_test22.sh openacc`, and `tests/refinement/run_test22.sh force-oracle` run the CPU, native GPU, and diagnostic oracle validations, respectively.

4.22 Test Case 23: Refinement with Collector Removal

Test Case 23 extends the repeated-refinement path through the collector. It retains Maxwell rheology, Yarin evaporation, stochastic Platen aerodynamic drag, nozzle insertion, and ordinary numerical anchor beads. The initial jet contains 400 elements over the first 8 cm of a 12 cm nozzle-to-collector domain. The applied potential is scaled to retain the physical field strength of Test Case 22. The timestep is 2.5×10^{-8} s and the run ends after 16,000 steps.

The dedicated runner sets developer-only initial and growth reserves of 20 entries. Three accepted Akima events therefore cause three capacity changes. Collector removal changes the lower active bound before the third remesh and continues afterwards. This combines all relevant topology transitions: insertion, removal, remeshing, capacity replacement, and Platen workspace resize, with the pre-generated Gaussian pool left untouched.

With NVFORTRAN 24.3, the CPU refinement events occur at steps 14,268, 14,971, and 15,691; NVFORTRAN 25.5 gives the same steps and removals. The first collector removal occurs at step 15,656, and the CPU makes nine removals and ends with 526 active elements. The native NVIDIA A30 path and the complete force oracle reproduce the CPU event steps and removal schedule exactly; all 81 statistics rows agree with the CPU within 8.0×10^{-10} and 6.1×10^{-8} relatively, including the rows written after the leading bead has reached the collector. All paths preserve 81 active anchors, and capacity grows from 420 to 481, 517, and 556.

The CPU event and removal sequence is not a CPU/GPU identity requirement. The device kernels evaluate in a different floating-point order, and the stochastic bending trajectory can amplify that difference near the collector. A build that evaluates only the direct Coulomb sum on the host writes a statistics file byte-identical to the native one. Validation instead requires collector removal both before and after the final remesh, finite output, ordered coordinates, unchanged anchor fields, and conservation of reference volume, evaporated volume, mass, and charge at every Akima event.

Removal is performed by the device topology primitive and does not download the full jet. Complete state transfers remain limited to accepted refinement events and final shutdown. Host code walks the mass boundaries, constructs the target mesh, keeps the anchor bookkeeping and final assembly, applies the conservative radius-floor and evaporation-limit corrections, and checks the invariants; Akima coefficient construction, all

11 field interpolations, the volume reconstruction with its conservation rescale, and the conversion of mass and charge densities execute on the GPU.

An NVIDIA A30 transfer audit records four complete state downloads, namely the three accepted Akima events and final shutdown, together with exactly three topology rebinds and three evaporation-state rebinds; the Gaussian pool is uploaded once at startup. Each ordinary step returns one 20-byte topology record with the insertion and removal decisions, against seven separate scalar transfers before 1 October 2026; an accepted removal transfers and clears only the collected point.

A dedicated A/B build evaluates the historical host Akima routines and the device implementation at all three events. Across 33 field/event comparisons, the maximum coefficient relative difference is 1.03×10^{-14} , while interpolated values differ by at most 1.82×10^{-12} absolutely and 1.89×10^{-15} relatively. Independent device and host-Akima-oracle runs produce byte-identical statistics files and the same event and removal topology.

The commands `tests/refinement/run_test23.sh nvfortran`, `tests/refinement/run_test23.sh openacc`, and `tests/refinement/run_test23.sh force-oracle` run the CPU, native GPU, and force-oracle validations. Arguments `host-akima` and `akima-compare` select the historical spline oracle and the direct host/device spline comparison, respectively. Argument `refinement-compare` (target `nvfortran-openacc-compare-refinement`) evaluates the host and device reconstruction of volume, mass, charge, and evaporated volume at every event; all three events agree at roundoff, at worst 8.9×10^{-16} absolutely and 1.4×10^{-16} relatively.

4.23 Test Case 24: long run to a stationary bead count

Test Case 24 is a long production run whose purpose is to reach a statistically stationary active-bead count. Once the jet has bridged the 16 cm nozzle-to-collector gap with a developed bending instability, insertion at the nozzle and removal at the collector are expected to balance, so that the bead count oscillates around a mean instead of drifting. Unlike Test Cases 21–23 it is not designed to finish within a smoke-test timeframe: it integrates `final time 0.5d0` seconds with `timestep 5.d-9` seconds, about 100 million steps.

The case is assembled from three sources. Its electrostatic parameters are JETSPIN’s canonical set, `density charge 44000`, `collector distance 16`, and `external potential 30.02076857`, shared with Examples 3, 4 and 5 and with Test Cases 9–20. Rheology, air drag, insertion and removal are those of Test Case 23: Maxwell rheology, stochastic Platen air drag, nozzle insertion, and collector removal, with its anchor spacing, `dynamic refinement anchor 0.10 cm`. Initialization and print directives are those of Example 3, so the jet grows from a single nozzle bead (`points 1`) rather than from the pre-extended 400-element mesh used to validate the refinement algorithm. This last point is what distinguishes the case: every other anchored stochastic-Platen refinement test (Test Cases 21–23) starts from an already extended jet, so the young-jet growth regime of this configuration had not been exercised. Example 5 also grows from one bead with refinement, but with RK4, multiple-step Coulomb sums, and no anchors.

Yarin evaporation, which Test Case 23 uses, is not enabled here, and this is a definition rather than a temporary state; the evaporation directives of the input are read but unused. The evaporative configuration is Test Case 25 (Sec. 4.24), whose input differs from this one by two lines, `evaporation yes` and `evaporation polymer frac 0.50d0`. With the 6% polymer fraction of Ref. [36] the evaporative configuration stopped on a numerical instability near step 1.07 million in runs of a build of 29 September 2026, because the dried jet keeps its charge while losing fifteen times its mass and cross-section; Test Case 25 carries that analysis.

The dynamic-refinement cadence is taken from Example 5, `dynamic refinement threshold 0.4 cm` and `dynamic refinement every 1.d-3 s`, rather than Test Case 23’s tighter `0.10 cm` and `1.d-5 s`. This choice is load-bearing rather than cosmetic. With the tighter cadence the jet accumulates dozens of accepted refinement events within fewer than 200,000 steps, each one re-fitting the cross-section radius from the previous event’s own output. The result is a nonphysical thinning compounding far beyond the order-of-magnitude reduction that electrospinning genuinely produces, collapsing bead mass toward zero until the resulting acceleration overflows the stress equation. The advisory `warning(108)` is raised whenever a refinement threshold is set below twenty times the base discretization resolution. It is informational and not enforced: Example 5 and Test Cases 24 and 25 sit exactly at that ratio (0.4 cm over a resolution of 0.02 cm) and raise no warning, while Test Cases 21–23, at five times the resolution, are below it and remain valid short-window references. Refinement also starts late: `dynamic refinement start 2.5d-4`, a directive that neither Example 5 nor Test Case 23 sets, allows no refinement during the first 2.5×10^{-4} s (50,000 steps).

Since 5 October 2026 the case follows Test Case 25: CPU and GPU builds read the same pre-generated Gaussian pool (Sec. 2.5.1), and the GPU runs the device-resident path once the jet holds 100 beads. The reference run uses the NVFORTRAN 24.3 CPU build and is truncated at 5,000,000 steps, 5% of the target duration, by `final time 2.5d-2` and `print time 1.d-4` in a copy of the input; it closes correctly. The first bead reaches the collector at step 2,541,113. Between 4 and 5 million steps the collector velocity is 1963 cm/s, the off-axis distance of the leading bead 3.8 cm, the cone angle 26.6 degrees, the active-bead count between 471 and 553 around a mean of 511, and the path length 211 cm, so the balanced insertion and removal regime is reached; four more CPU seeds give mean active counts between 511.3 and 512.0. One NVIDIA A30 runs the same 5 million steps in 596 s, with the process bound to the NUMA node of its GPU: it reproduces the CPU run exactly until step 1,379,105, where it engages the device-resident path, follows the CPU trajectory afterwards (with seed 317 the active-bead count first differs at step 4.28 million; until 7 October 2026 the bead inserted at the array reallocation of step 1,963,389 came one step late on the GPU, and the counts separated at 2.66 million steps), and five seeds give between 511 and 512 active beads and a path length of 211 cm over the same window. The A30 step then costs between about 105 and 145 μ s from 100 to 600 beads, against 4 to 5 ms for the CPU build with about 500 beads; the standard GPU build of the morning of 5 October 2026, which kept this case on the host with only the Coulomb sums on the GPU, needed 4546 s. With NVHPC 25.5 the run takes 630 s, against 597 s with NVHPC 24.3, the reference compiler, in the same session, and gives the same insertion and removal totals and 511.2 active beads between 4 and 5 million steps, as NVHPC 24.3 does. On one A30 the full 100 million steps took 14,906 s, about four hours, with a device-resident build of 5 October 2026, whose output equals the current build’s up to the reallocation at step 1,963,389, where that build inserted one step late: from 10 to 100 million steps the jet keeps 517 active beads on average (474 to 560), a path length of 214 cm, an off-axis distance of 3.8 cm and a cone angle of 26.9 degrees, about 1% above the values between 4 and 5 million steps, where the jet is still settling. These long-run statistics rest on periodically repeating noise: with the default pool of 10^8 values (Sec. 2.5.1) the noise repeats every $10^8/(6 n_{\text{active}})$ steps, about 32,000 steps at 517 beads, so the 100 million steps go through the pool about 3,100 times.

Track the printed `n` column over time to observe the stationary oscillation. The input is stored in `examples/input-24/input.dat`. The investigation into refinement robustness under repeated remeshing, which produced the cadence used here, is recorded in `docs/refinement-robustness-investigation.md`.

4.24 Test Case 25: evaporative counterpart of Test Case 24

Test Case 25 is Test Case 24 with Yarin evaporation enabled. The two input files differ by two directives, `evaporation yes` and `evaporation polymer frac 0.50d0`. Everything else is shared: the canonical electrostatic parameters (`density charge 44000`, `collector distance 16`, `external potential 30.02076857`), Maxwell rheology with $\mu_0 = 20$ P and $G_0 = 5 \times 10^4$ dyn/cm², stochastic Platen air drag, nozzle insertion and collector removal from Test Case 23 with its anchor spacing (`dynamic refinement anchor 0.10 cm`), the single-nozzle-bead start and print directives from Example 3, Example 5’s refinement cadence, and `dynamic refinement start 2.5d-4`, which allows no refinement during the first 50,000 steps. The evaporation law of Sec. 2.8 is used with the coefficients of Ref. [36] unchanged, $B = 7$, $m = 0.1$, $t_{\text{ev}} = 1$, and a relative humidity of 0.165 at 293.15 K; only the initial polymer mass fraction is raised from 0.06 to 0.50.

With $c_{p,0} = 0.50$ the solidification cutoff $c_p = 0.9$ is reached at $V/V_0 = 0.556$, and every bead collected in the reference run has reached it: at the collector the jet is 90% polymer by mass, 89% of the initial solvent has evaporated, the viscosity has grown by a factor of 2.49 (to 49.7 P), the relaxation time by 1.80, and the elastic modulus by 1.38.

The initial polymer fraction of Ref. [36] does not survive this configuration. With $c_{p,0} = 0.06$ a build of 29 September 2026, which still drew this case’s noise step by step and, on the GPU, integrated on the host with only the Coulomb sums offloaded, stopped the run on the code’s numerical instability check at step 1,078,685 on an NVIDIA A30 (step 1,067,695 with the NVFORTRAN CPU build), with a stress NaN. These runs and the diagnostic variants below were not repeated with the current build, in which the CPU and the A30 give identical results below 100 beads. The cause lies in the parameter regime. An evaporating element keeps its charge while its mass and cross-section decrease with its volume, down to $V/V_0 = c_{p,0}/0.9$, that is one fifteenth; its charge-to-mass ratio therefore grows fifteenfold. Because $\theta \propto c_p$, the elastic modulus grows only 2.9 times, so the elastic resistance $G \pi a^2$ of a dried element is about five times weaker than that

of the fresh jet. The dried head of the jet is then pushed laterally by Coulomb repulsion, its off-axis distance growing from 3 to 20 cm within 40,000 steps; the next accepted refinement event has to remesh a jet that has left the physical regime, and the stress NaN follows. Diagnostic variants of 1.5 million steps on the CPU confirm that the outcome follows the elastic resistance of the dried jet: freezing viscosity, relaxation time and modulus leads to an earlier failure; keeping the relaxation time constant, so that the modulus grows with the viscosity, fails when the growth at the cutoff is 15 or less (with 15 the jet reaches the collector, then fails at step 1,430,536) and runs cleanly to 1.5 million steps when it is 30 or more. With the relative humidity set to one, so that no net evaporation occurs, the evaporative code path reproduces Test Case 24. Raising the initial polymer fraction bounds the mass loss without altering the concentration law, whose coefficients B and m were fitted by Yarin et al. to the envelope cone of a much more viscous 6% PEO solution ($\mu_0 = 10^4$ P, $\theta_0 = 10$ ms).

The reference run uses the NVFORTRAN 24.3 CPU build and is truncated at 5,000,000 steps, 5% of the target duration, by `final time 2.5d-2` and `print time 1.d-4` in a copy of the input; it closes correctly. The jet first reaches the collector at step 2,120,839. Between 4 and 5 million steps the collector velocity is 2535 cm/s, the off-axis distance of the leading bead 2.8 cm, the cone angle 19.7 degrees, the active-bead count between 242 and 295, the path length 112 cm, and the fibre radius at the collector 2.8 μm against a nozzle radius of 50 μm . The envelope cone opens from the nozzle with a local half-angle growing from about 8 to 12 degrees. Four more CPU seeds give mean active counts between 267.8 and 268.4 and the same path length, so the stationary regime does not depend on the noise realization. On one NVIDIA A30 the time-integration loop of the same 5 million steps takes 666 s, with the process bound to the NUMA node of its GPU, against 5863 s for the CPU build; the A30 reads the same Gaussian pool, follows the CPU trajectory after it engages the device-resident path (with seed 317 the active-bead count first differs at step 4.64 million), and gives the same stationary values. With NVHPC 25.5 the run takes 706 s, against 664 s with NVHPC 24.3, the reference compiler, in the same session, and gives the same insertion and removal totals and 267.5 active beads between 4 and 5 million steps (267.9 with NVHPC 24.3). Compared with an NVFORTRAN CPU run of Test Case 24 over the same interval (3.8 cm, 27 degrees), evaporation and the stiffening it causes make the bending loops smaller, as reported in Ref. [36]. On one A30 the full 100 million steps took 15,192 s, about four hours, with a device-resident build of 5 October 2026, whose output equals the current build's over the first 5 million steps: from 10 to 100 million steps the jet keeps 268 active beads on average (241 to 297), a path length of 112 cm, an off-axis distance of 2.8 cm and a cone angle of 19.7 degrees, the values of the window between 4 and 5 million steps. As for Test Case 24, these statistics rest on periodically repeating noise: with the default pool of 10^8 values (Sec. 2.5.1) the noise repeats about every 62,000 steps at 268 beads, so the 100 million steps go through the pool about 1,600 times.

The A30 run has two phases. Until the jet holds 100 beads (step 1,446,413 with seed 317, right after an accepted refinement event) the whole step runs on the host, which also computes the Coulomb sums while the jet has fewer than 128 active beads, and the run reproduces the CPU build byte for byte. Afterwards the step runs on the device-resident state, and an ordinary step returns only a 20-byte record of topology decisions to the host. With each process bound to the NUMA node of its GPU, the first phase takes 156 s, the time of the CPU build; until 2026-10-05 the three Coulomb sums of each step were offloaded, each call copying 13 arrays to the device and 2 back, and the phase took 430 s, 79% of it in those calls. The second phase takes about 500 s, 120 to 145 μs per step from 120 to 300 beads against 0.5 to 1.8 ms for the CPU build: its kernels, 12 to 15 per step after fusing the small ones, run on one asynchronous queue and the host waits once per step, for the topology record. With a synchronization after every kernel, as until 2026-10-05, the phase took 1418 s and the device was idle for 40 to 45% of each step. Without binding, placement of the host process alone changes the timings by up to 8%.

The case is excluded from the short smoke and regression matrices because of its length. The input is stored in `examples/input-25/input.dat`, and `docs/examples/test-25.md` lists the diagnostic variants and the time budget of the two phases in detail.

Chapter 5

Tips

Scope of Chapter

This chapter describes few useful tips which can be used to enhance results obtained by JETSPIN.

5.1 How to visualize a PDB file in VMD software package

The PDB file written by JETSPIN can be easily visualized in VMD software package. The first step is to download and install VMD which can be found at website: <http://www.ks.uiuc.edu/Research/vmd/>. As VMD is installed, the PDB file can be opened using VMD: "vmd frame000001.pdb" in the terminal (frame000001 is the first frame of a new run), or, together with its bonds, "vmd -psf frame000001.psf -pdb frame000001.pdb"; alternatively, click on "file" in the main white box and then on "load molecule" (see Fig 5.1), browse for your PDB file frame000001.pdb and load it. In order to load the topological structure of the jet, it is possible to load the PSF file written by JETSPIN alongside the PDB file: right click on the just uploaded file (e.g. "frame000001.pdb") in the white box in the middle of the "MD Main" window, and select "Load Coordinates into Molecule" pull-down menu option (see Fig 5.2). Hence, browse for the PSF file (frame000001.psf) defining the bonds for each pair of consecutive jet beads, and load it. Finally, the jet will be displayed in the graphics window.

The developer binary trajectory `traj.dat` (directive `print binary`, Tab 3.1) can be converted for VMD by the tool `tools/traj2dcd.f90`, which reads the parameter header and the frames of any `style` (1–9) written by JETSPIN, converts the coordinates to centimeters, and writes `trajout.xyz` (a single frame with the largest number of beads, used as structure) and the trajectory `trajout.dcd`. It is compiled and run in the directory holding `traj.dat`, e.g. "gfortran -O2 -o traj2dcd.x traj2dcd.f90; ./traj2dcd.x", and the result is opened by "vmd trajout.xyz -dcd trajout.dcd".

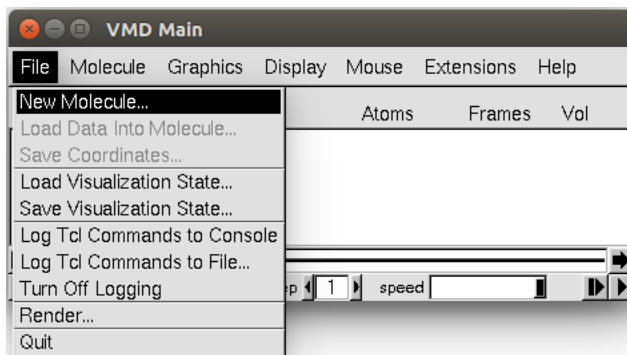


Figure 5.1: Snapshot of the main white box of VMD.

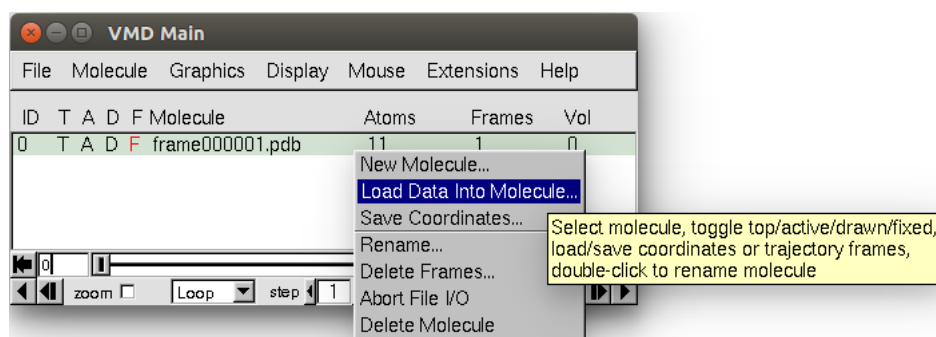


Figure 5.2: Snapshot of the "Load Coordinates into Molecule" selection in VMD.

Bibliography

- [1] D. Li, Y. Wang, Y. Xia, Electrospinning nanofibers as uniaxially aligned arrays and layer-by-layer stacked films, *Advanced Materials* 16 (4) (2004) 361–366. [1.1](#)
- [2] A. Greiner, J. H. Wendorff, Electrospinning: a fascinating method for the preparation of ultrathin fibers, *Angewandte Chemie International Edition* 46 (30) (2007) 5670–5703. [1.1](#)
- [3] C. P. Carroll, E. Zhmayev, V. Kalra, Y. L. Joo, Nanofibers from electrically driven viscoelastic jets: modeling and experiments, *Korea-Aust Rheol J* 20 (2008) 153–164. [1.1](#)
- [4] Z.-M. Huang, Y.-Z. Zhang, M. Kotaki, S. Ramakrishna, A review on polymer nanofibers by electrospinning and their applications in nanocomposites, *Composites science and technology* 63 (15) (2003) 2223–2253. [1.1](#)
- [5] L. Persano, A. Camposeo, C. Tekmen, D. Pisignano, Industrial upscaling of electrospinning and applications of polymer nanofibers: a review, *Macromolecular Materials and Engineering* 298 (5) (2013) 504–520. [1.1](#), [4.3](#)
- [6] A. L. Yarin, B. Pourdeyhimi, S. Ramakrishna, *Fundamentals and Applications of Micro and Nanofibers*, Cambridge University Press, 2014. [1.1](#), [2.1](#), [4.3](#)
- [7] J. H. Wendorff, S. Agarwal, A. Greiner, *Electrospinning: materials, processing, and applications*, John Wiley & Sons, 2012. [1.1](#)
- [8] D. Pisignano, *Polymer Nanofibers: Building Blocks for Nanotechnology*, Royal Society of Chemistry, 2013. [1.1](#), [4.3](#)
- [9] Y. Zeng, Y. Wu, Z. Pei, C. Yu, Numerical approach to electrospinning, *International Journal of Nonlinear Sciences and Numerical Simulation* 7 (4) (2006) 385–388. [1.1](#)
- [10] J. Feng, Stretching of a straight electrically charged viscoelastic jet, *Journal of Non-Newtonian Fluid Mechanics* 116 (1) (2003) 55–70. [1.1](#), [2.1](#)
- [11] D. H. Reneker, A. L. Yarin, H. Fong, S. Kooimbhongse, Bending instability of electrically charged liquid jets of polymer solutions in electrospinning, *Journal of Applied physics* 87 (9) (2000) 4531–4547. [1.1](#), [2.1](#), [2.7](#), [2.7](#), [4.1](#), [4.3](#)
- [12] A. L. Yarin, S. Kooimbhongse, D. H. Reneker, Taylor cone and jetting from liquid droplets in electrospinning of nanofibers, *Journal of Applied Physics* 90 (9) (2001) 4836–4846. [1.1](#)
- [13] S. V. Fridrikh, H. Y. Jian, M. P. Brenner, G. C. Rutledge, Controlling the fiber diameter during electrospinning, *Physical review letters* 90 (14) (2003) 144502. [1.1](#)
- [14] S. Theron, E. Zussman, A. Yarin, Experimental investigation of the governing parameters in the electrospinning of polymer solutions, *Polymer* 45 (6) (2004) 2017–2030. [1.1](#)
- [15] C. Lu, P. Chen, J. Li, Y. Zhang, Computer simulation of electrospinning. part i. effect of solvent in electrospinning, *Polymer* 47 (3) (2006) 915–921. [1.1](#)

- [16] X. Wang, Y. Liu, C. Zhang, Y. An, X. He, W. Yang, Simulation on electrical field distribution and fiber falls in melt electrospinning, *Journal of nanoscience and nanotechnology* 13 (7) (2013) 4680–4685. [1.1](#)
- [17] I. Coluzza, D. Pisignano, D. Gentili, G. Pontrelli, S. Succi, Ultrathin fibers from electrospinning experiments under driven fast-oscillating perturbations, *Physical Review Applied* 2 (5) (2014) 054011. [1.1](#), [2.2](#)
- [18] W. Smith, Molecular dynamics on hypercube parallel computers, *Computer Physics Communications* 62 (2) (1991) 229–248. [1.3](#)
- [19] D. Brown, J. H. Clarke, M. Okuda, T. Yamazaki, A domain decomposition parallelization strategy for molecular dynamics simulations on distributed memory machines, *Computer Physics Communications* 74 (1) (1993) 67–80. [1.3](#)
- [20] M. Lauricella, G. Pontrelli, I. Coluzza, D. Pisignano, S. Succi, Jetspin: a specific-purpose open-source software for simulations of nanofiber electrospinning, *Computer Physics Communications* 197 (2015) 227–238. [1.3](#), [3.1](#)
- [21] J. Racine, The cygwin tools: a gnu toolkit for windows, *Journal of Applied Econometrics* 15 (3) (2000) 331–341. [1.4](#)
- [22] G. Pontrelli, D. Gentili, I. Coluzza, D. Pisignano, S. Succi, Effects of non-linear rheology on the electrospinning process: a model study, *Mechanics Research Communications* 61 (2014) 41–46. [2.1](#), [3.1](#), [4.1](#)
- [23] H. A. Barnes, J. F. Hutton, K. Walters, *An introduction to rheology*, Vol. 3, Elsevier, 1989. [2.1](#)
- [24] T. A. Kowalewski, S. Barral, T. Kowalczyk, Modeling electrospinning of nanofibers, in: *IUTAM symposium on modelling nanomaterials and nanosystems*, Springer, 2009, pp. 279–292. [2.1](#), [2.1](#), [2.4](#)
- [25] J. Feng, The stretching of an electrified non-newtonian jet: A model for electrospinning, *Physics of Fluids (1994-present)* 14 (11) (2002) 3912–3926. [2.1](#)
- [26] M. M. Hohman, M. Shin, G. Rutledge, M. P. Brenner, Electrospinning and electrically forced jets. i. stability theory, *Physics of Fluids (1994-present)* 13 (8) (2001) 2201–2220. [2.1](#)
- [27] T. Kowalewski, S. NSKI, S. Barral, Experiments and modelling of electrospinning process, *Technical Sciences* 53 (4). [2.1](#)
- [28] A. Ziabicki, H. Kawai, *High-Speed Fiber Spinning: Science and Engineering Aspects*, Krieger Publishing Co, 1991. [2.1](#)
- [29] M. Lauricella, G. Pontrelli, D. Pisignano, S. Succi, Dynamic mesh refinement for discrete models of jet electro-hydrodynamics, *Journal of Computational Science*. [2.1](#), [2.4](#), [2.4](#)
- [30] M. Lauricella, G. Pontrelli, I. Coluzza, D. Pisignano, S. Succi, Different regimes of the uniaxial elongation of electrically charged viscoelastic jets due to dissipative air drag, *Mechanics Research Communications* 69 (2015) 97–102. [2.1](#)
- [31] M. Lauricella, G. Pontrelli, D. Pisignano, S. Succi, Nonlinear langevin model for the early-stage dynamics of electrospinning jets, *Molecular Physics* 113 (17-18) (2015) 2435–2441. [2.1](#)
- [32] A. L. Yarin, *Free liquid jets and films: hydrodynamics and rheology*, Longman Scientific & Technical Harlow, 1993. [2.1](#), [4.3](#)
- [33] M. Lauricella, D. Pisignano, S. Succi, Three-dimensional model for electrospinning processes in controlled gas counterflow, submitted to *Journal of Physical Chemistry A*. [2.1](#)
- [34] H. Akima, A new method of interpolation and smooth curve fitting based on local procedures, *Journal of the ACM (JACM)* 17 (4) (1970) 589–602. [2.4](#)

- [35] H. Akima, A method of univariate interpolation that has the accuracy of a third-degree polynomial, *ACM Transactions on Mathematical Software (TOMS)* 17 (3) (1991) 341–366. [2.4](#), [2.4](#)
- [36] A. Yarin, S. Koombhongse, D. H. Reneker, Bending instability in electrospinning of nanofibers, *Journal of Applied Physics* 89 (5) (2001) 3018–3026. [2.4.1](#), [2.8](#), [2.8](#), [2.8.2](#), [2.8.3](#), [2.8.5](#), [4.7](#), [4.8](#), [4.23](#), [4.24](#)
- [37] D. T. Gillespie, E. Seitaridou, *Simple Brownian Diffusion: An Introduction to the Standard Theoretical Models*, Oxford University Press, 2012. [2.5](#)
- [38] M. Lauricella, F. Cipolletta, G. Pontrelli, D. Pisignano, S. Succi, Effects of orthogonal rotating electric fields on electrospinning process, *Physics of Fluids* 29 (8) (2017) 082003. [2.6](#)
- [39] W. H. Press, *Numerical recipes 3rd edition: The art of scientific computing*, Cambridge university press, 2007. [2.9](#)
- [40] E. Platen, Derivative free numerical methods for stochastic differential equations, in: *Stochastic Differential Systems*, Springer, 1987, pp. 187–193. [2.9](#)
- [41] P. E. Kloeden, E. Platen, *Numerical solution of stochastic differential equations*, Vol. 23, Springer, 1992. [2.9](#)
- [42] E. Platen, N. Bruti-Liberati, *Numerical solution of stochastic differential equations with jumps in finance*, Vol. 64, Springer, 2010. [2.9](#)
- [43] W. Streett, D. Tildesley, G. Saville, Multiple time-step methods in molecular dynamics, *Molecular Physics* 35 (3) (1978) 639–648. [2.10](#)
- [44] M. P. Allen, D. J. Tildesley, *Computer simulation of liquids*, Oxford university press, 1989. [2.10](#)
- [45] C. P. Carroll, Y. L. Joo, Discretized modeling of electrically driven viscoelastic jets in the initial stage of electrospinning, *Journal of Applied Physics* 109 (9) (2011) 094315. [3.1](#)
- [46] W. Humphrey, A. Dalke, K. Schulten, Vmd: visual molecular dynamics, *Journal of molecular graphics* 14 (1) (1996) 33–38. [3.2](#)
- [47] E. F. Pettersen, T. D. Goddard, C. C. Huang, G. S. Couch, D. M. Greenblatt, E. C. Meng, T. E. Ferrin, Ucsf chimera: a visualization system for exploratory research and analysis, *Journal of computational chemistry* 25 (13) (2004) 1605–1612. [3.2](#)
- [48] M. Montinaro, V. Fasano, M. Moffa, A. Camposeo, L. Persano, M. Lauricella, S. Succi, D. Pisignano, Sub-ms dynamics of the instability onset of electrospinning, *Soft Matter* 11 (2015) 3424. [4.3](#)
- [49] N. Yuya, W. Kai, B.-S. Kim, I. Kim, Morphology controlled electrospun poly(vinyl pyrrolidone) fibers: effects of organic solvent and relative humidity, *Journal of Materials Science and Engineering with Advanced Technology* 2 (1) (2010) 97–112. [4.3](#)
- [50] V. Bühler, *Polyvinylpyrrolidone excipients for pharmaceuticals: povidone, crospovidone and copovidone*, Springer Science & Business Media, 2005. [4.3](#)
- [51] V. N. Morozov, A. Y. Mikheev, Water-soluble polyvinylpyrrolidone nanofilters manufactured by electrospay-neutralization technique, *Journal of Membrane Science* 403 (2012) 110–120. [4.3](#)